计算机系统 I

期末复习讲义

知识点讲解 + 分章真题巩固

使用说明

- 全书按 Lec01–Lec08 组织为 9 章，每章先系统梳理知识点，再安排配套题目。
- 复习时先看“本章考点与复习路线”，再学习详细知识；不建议直接背结论。
- 练习标签：**[Exam]** 本课程期末卷；**[Quiz]** 本课程小测；**[Recall]** 回忆题；**[Class]** 课堂题；**[NumLogic]** 数逻真题；**[Extra]** 计组/数逻同考点补充题。
- 真题巩固部分不附答案；含电路图的题目已保留题图。

复习主线：数据如何表示 → 逻辑如何计算 → 状态如何保存

指令如何定义 → 处理器如何执行

目录

全书复习路线 **iii**

第一章 信息表示 (Information Representation) **1**

1.1 信息表示基础 (Foundations of Information Representation) 1

1.1.1 信息与信号 (Information & Signals) 1

1.1.2 外部信息与内部数据 (External Information & Internal Data) 1

1.2 数制与整数表示 (Radix Systems & Integer Representation) 2

1.2.1 数制与转换 (Radix Number Systems & Conversions) 2

1.2.2 整数的内部表示 (Internal Representation of Integers) 2

1.3 浮点数表示 (Floating-Point Representation) 4

1.3.1 二进制小数基础 (Fractional Binary Numbers) 4

1.3.2 IEEE 754 标准 (IEEE 754 Standard) 4

1.3.3 浮点数的分类 (Categories of Floating-Point Values) 5

1.3.4 浮点数编码与运算特性 (Encoding and Arithmetic Properties) 6

1.4 非数值数据与存储 (Non-numeric Data & Storage) 7

1.4.1 非数值数据的表示 (Representation of Non-numeric Data) 7

1.4.2 数据宽度与单位 (Data Width and Unit) 8

1.4.3 主存容量与频率单位 (Units of Capacity and Frequency) 8

1.4.4 数据存储的字节序 (Byte Ordering) 9

1.5 全章总结 (Summary) 9

1.6 真题与巩固练习 10

第二章 布尔代数与数字逻辑 (Boolean Algebra) **12**

2.1 数字逻辑导论 (Introduction to Digital Logic) 12

2.1.1 模拟信号与数字信号 (Analog vs. Digital Signals) 12

2.1.2 为什么采用数字系统 (Why Digital?) 12

2.2 二值逻辑与逻辑门 (Binary Logic and Gates) 12

2.2.1 二值变量与基本运算 (Binary Variables & Basic Operations) 12

2.2.2 真值表 (Truth Tables) 13

2.2.3 用开关建模逻辑函数 (Logic Functions with Switches) 13

2.2.4 逻辑门类型 (Logic Gate Types) 13

2.2.5 常见逻辑门真值表 (Common Gate Truth Tables) 14

2.3 晶体管与逻辑门实现 (Transistors and Gate Implementation) . . . 14
2.3.1 从继电器到晶体管 (Relays, Vacuum Tubes, Transistors) . . . 14
2.3.2 Robert Noyce 与集成电路 (Integrated Circuit) . . . 14
2.3.3 硅与掺杂 (Silicon and Doping) . . . 14
2.3.4 IC 逻辑家族与 CMOS 晶体管 (TTL and CMOS) . . . 15
2.3.5 用晶体管实现门电路 (Gate Implementation) . . . 15
2.4 IC 参数与数字抽象 (IC Parameters and Digital Abstraction) . . . 15
2.4.1 电源电压与逻辑电平 (VCC and Logic Levels) . . . 15
2.4.2 噪声与噪声容限 (Noise and Noise Margins) . . . 15
2.4.3 常见逻辑家族参数 (Logic Family Examples) . . . 16
2.4.4 传播延迟与延迟模型 (Propagation Delay and Delay Models) . . . 16
2.4.5 转换时间、功耗、扇入扇出 (Transition, Power, Fan-in/out) . . . 16
2.4.6 门成本 (Gate Cost) . . . 17
2.5 布尔代数 (Boolean Algebra) . . . 17
2.5.1 George Boole 与布尔代数 . . . 17
2.5.2 公理 (Axioms) . . . 17
2.5.3 单变量定理 (Single-Variable Theorems) . . . 18
2.5.4 二至三变量定理 (2–3 Variable Theorems) . . . 18
2.5.5 n 变量定理、德摩根定理和香农展开 . . . 18
2.5.6 运算优先级 (Boolean Operator Precedence) . . . 19
2.5.7 对偶性 (Duality) . . . 19
2.5.8 布尔代数证明示例 (Algebraic Proof Examples) . . . 19
2.6 逻辑函数表示与标准形式 (Logic Function Representation) . . . 20
2.6.1 逻辑函数、真值表与波形 (Functions, Truth Tables, Waveforms) . . . 20
2.6.2 表达式不唯一 (Boolean Expressions Are Not Unique) . . . 20
2.6.3 函数补 (Complement of a Function) . . . 21
2.6.4 文字项、最小项、最大项 (Literals, Minterms, Maxterms) . . . 21
2.6.5 最小项与最大项编号规则 . . . 21
2.6.6 规范形式 (Canonical Forms) . . . 22
2.6.7 标准形式 (Standard Forms) . . . 22
2.7 逻辑函数化简 (Simplification of Logic Functions) . . . 23
2.7.1 化简目标与成本准则 (Optimization and Cost Criteria) . . . 23
2.7.2 成本计算示例 (Cost Examples) . . . 23
2.7.3 代数化简 (Algebraic Manipulation) . . . 23
2.7.4 卡诺图基础 (Karnaugh Maps) . . . 24
2.7.5 卡诺图排列 (K-map Layouts) . . . 25
2.7.6 卡诺图优化原则 (Principles and Rules) . . . 25
2.7.7 蕴含项、主蕴含项、必要主蕴含项 (Implicants) . . . 25
2.7.8 三变量卡诺图例题 . . . 26
2.7.9 四变量卡诺图例题 . . . 26

2.7.10 带不关心项的例题 (Don't Cares) . . . 26
2.7.11 系统化化简示例 (Systematic Simplification) . . . 26
2.7.12 结果可能不唯一 (Results May Not Be Unique) . . . 27
2.7.13 德摩根与气泡推动 (DeMorgan and Bubble Pushing) . . . 27
2.7.14 多级优化 (Multiple-Level Optimization) . . . 28
2.8 附加门与电路 (Additional Gates and Circuits) . . . 28
2.8.1 异或与同或 (XOR and XNOR) . . . 28
2.8.2 XOR 的实现与恒等式 . . . 28
2.8.3 XOR/XNOR 的应用 . . . 29
2.8.4 奇偶校验生成器与检查器 (Parity Generators and Checkers) . . . 29
2.8.5 高阻输出与三态缓冲器 (Hi-Z and 3-State Buffer) . . . 29
2.8.6 多路复用线与传输门 (Multiplexed Line and Transmission Gate) . . . 30
2.9 全章总结 (Summary) . . . 30
2.10 真题与巩固练习 . . . 31

第三章 组合逻辑设计 (Combinational Logic) 34
3.1 Verilog HDL 简介 (Introduction to Verilog HDL) . . . 34
3.1.1 背景与 HDL 概念 (Background & What is HDL) . . . 34
3.1.2 基于 HDL 的设计流程 (HDL-based Design Flow) . . . 34
3.1.3 Verilog 词法规则 (Lexical Conventions) . . . 35
3.1.4 基本语法 (Basic Syntax) . . . 36
3.1.5 建模方法 (Modeling Methods) . . . 37
3.2 组合逻辑电路基础 (About Combinational Logic Circuits) . . . 39
3.2.1 数字逻辑电路 (Digital Logic Circuit) . . . 39
3.2.2 组合逻辑电路定义 . . . 39
3.2.3 组合逻辑与时序逻辑的区别 . . . 39
3.2.4 组合逻辑电路的结构约束 . . . 40
3.2.5 组合电路的设计选择 . . . 40
3.2.6 组合逻辑设计流程 (Design Procedure) . . . 40
3.3 组合逻辑设计示例 (Design Examples) . . . 40
3.3.1 示例 1：三开关控制一盏灯 . . . 40
3.3.2 示例 2：BCD 到余 3 码转换器 . . . 41
3.4 经典组合逻辑功能块 (Classic/Basic Designs) . . . 43
3.4.1 功能与功能块 (Functions and Functional Blocks) . . . 43
3.4.2 基本逻辑功能 (Rudimentary Logic Functions) . . . 43
3.4.3 使能功能 (Enabling Function) . . . 44
3.4.4 译码器 (Decoder) . . . 44
3.4.5 用译码器和 OR 门实现组合逻辑 . . . 46
3.4.6 七段显示译码器 (Seven-Segment Display Decoder) . . . 46
3.4.7 编码器 (Encoder) . . . 47

3.4.8 多路选择器 (Multiplexer) . . . 48
3.4.9 用 MUX 实现组合逻辑 . . . 49
3.4.10 半加器与全加器补充 (Half Adder & Full Adder) . . . 50
3.5 时序分析 (Timing Analysis) . . . 50
3.5.1 电路延迟 (Circuit Delay) . . . 50
3.5.2 传播延迟与污染延迟 . . . 50
3.5.3 关键路径与最短路径 . . . 51
3.5.4 竞争冒险与毛刺 (Race Hazard & Glitch) . . . 51
3.6 逻辑电路分析 (Analysis of Logic Circuits) . . . 51
3.6.1 分析步骤 . . . 51
3.6.2 示例：分析一个组合逻辑电路 . . . 51
3.7 复习小结 (Summary) . . . 52
3.8 真题与巩固练习 . . . 53

第四章 算术运算单元 (Arithmetic Unit) 55
4.1 运算单元总览与迭代组合电路 (Overview & Iterative Combinational Circuits) . . . 55
4.1.1 本讲主线 (Lecture Roadmap) . . . 55
4.1.2 迭代组合电路 (Iterative Combinational Circuits) . . . 55
4.2 加法器 (Adders) . . . 56
4.2.1 半加器 (Half-Adder) . . . 56
4.2.2 全加器 (Full Adder) . . . 56
4.2.3 多位进位传播加法器 (Carry Propagate Adders, CPA) . . . 57
4.2.4 行波进位加法器 (Ripple-Carry Adder, RCA) . . . 57
4.2.5 超前进位加法器 (Carry Lookahead Adder, CLA) . . . 57
4.2.6 组超前进位 (Group Carry Lookahead) . . . 58
4.2.7 前缀加法器 (Prefix Adder) . . . 58
4.2.8 Carry Skip 与 Carry Select . . . 59
4.2.9 32 位 CLA 与延迟比较 . . . 59
4.3 减法、溢出与标志位 (Subtraction, Overflow & Flags) . . . 59
4.3.1 无符号补码减法 (Unsigned 2's Complement Subtraction) . . . 59
4.3.2 统一加减法硬件 (Adder-Subtractor) . . . 60
4.3.3 进位与溢出 (Carry & Overflow) . . . 60
4.3.4 带符号溢出检测 (Detecting Signed Overflow) . . . 60
4.3.5 重要标志位 (Important Flags) . . . 61
4.3.6 半减器与全减器 (Half/Full Subtractor) . . . 61
4.4 算术逻辑单元与移位器 (ALU & Shifters) . . . 61
4.4.1 什么是 ALU? . . . 61
4.4.2 简单 1 位 ALU . . . 62
4.4.3 N 位 8 操作 ALU . . . 62
4.4.4 SLT 指令示例 (Set Less Than) . . . 62

4.4.5 移位器的用途 (Various Use of Shifters) . . . 63
4.4.6 移位作为乘除法 (Shifters as Multipliers and Dividers) . . . 63
4.4.7 组合移位器与桶形移位器 (Combinational and Barrel Shifters) . . . 63
4.4.8 数据通路中的 ALU (Datapath and ALU) . . . 64
4.5 乘法器 (Multipliers) . . . 64
4.5.1 二进制乘法基础 . . . 64
4.5.2 直接竖式算法 (Straightforward Algorithm) . . . 64
4.5.3 逐次乘法硬件实现 1 (Implementation 1) . . . 65
4.5.4 逐次乘法硬件实现 2 与 3 . . . 65
4.5.5 有符号乘法与 Booth 算法 . . . 66
4.5.6 Booth 算法为什么成立? . . . 67
4.5.7 Booth 示例与易错点 . . . 67
4.5.8 阵列乘法器与更快乘法 (Array Multiplier & Faster Multiplication) . . . 68
4.6 除法器 (Dividers) . . . 68
4.6.1 无符号除法与恢复除法 (Unsigned and Restoring Division) . . . 68
4.6.2 恢复除法实现 1 . . . 68
4.6.3 恢复除法实现 2 与改进 . . . 68
4.6.4 非恢复除法 (Non-Restoring Division) . . . 69
4.6.5 有符号除法 (Signed Division) . . . 69
4.6.6 4 x 4 阵列除法器 . . . 69
4.7 浮点运算单元 (Floating-Point Operations) . . . 70
4.7.1 浮点加法步骤 (Floating-Point Addition) . . . 70
4.7.2 浮点加法硬件 . . . 71
4.7.3 浮点乘法步骤 (Floating-Point Multiplication) . . . 71
4.7.4 浮点乘法、除法与硬件复杂度 . . . 73
4.7.5 浮点除法概要 (Floating-Point Division) . . . 73
4.8 全章总结 (Summary) . . . 73
4.9 真题与巩固练习 . . . 74

第五章 时序逻辑 (Sequential Logic) 75
5.1 时序逻辑基础 (Foundations of Sequential Logic) . . . 75
5.1.1 时序电路的组成 (Sequential Circuit Model) . . . 75
5.1.2 同步与异步时序电路 (Synchronous vs. Asynchronous) . . . 75
5.1.3 门延迟、反馈与状态 (Delay, Feedback and State) . . . 76
5.2 基本存储元件 (Basic Storage Elements) . . . 76
5.2.1 S-R 锁存器 (S-R Latches) . . . 76
5.2.2 带时钟的 S-R 锁存器 (Clocked S-R Latch) . . . 77
5.2.3 D 锁存器 (D Latch) . . . 77
5.2.4 锁存器的时序问题 (Latch Timing Problem) . . . 77
5.2.5 主从触发器 (Master-Slave Flip-Flop) . . . 78

5.2.6 边沿触发 D 触发器 (Edge-Triggered D Flip-Flop) . . . 78
5.2.7 直接输入与初始化 (Direct Inputs) . . . 78
5.2.8 触发器时序参数 (Flip-Flop Timing Parameters) . . . 79
5.2.9 J-K 与 T 触发器 (J-K and T Flip-Flops) . . . 79
5.3 同步时序逻辑分析 (Synchronous Sequential Logic Analysis) . . . 79
5.3.1 分析任务与一般流程 . . . 79
5.3.2 状态表与状态图 (State Table and State Diagram) . . . 80
5.3.3 分析例 1：两个 D 触发器电路 . . . 80
5.3.4 Moore、Mealy 与混合输出 . . . 80
5.3.5 等价状态 (Equivalent States) . . . 81
5.3.6 分析例 2：三触发器无输入电路 . . . 81
5.3.7 系统级时序约束 (Circuit and System Level Timing) . . . 81
5.4 同步时序逻辑设计 (Synchronous Sequential Logic Design) . . . 82
5.4.1 设计流程 (Design Procedure) . . . 82
5.4.2 状态图构造：序列识别器 (Sequence Recognizer) . . . 83
5.4.3 例：识别序列 1101 的 Mealy 模型 . . . 83
5.4.4 例：识别 1101 的 Moore 模型 . . . 83
5.4.5 状态编码 (State Assignment) . . . 84
5.4.6 状态编码例：Counting vs. Gray . . . 84
5.4.7 状态编码规则例 . . . 85
5.4.8 设计例 3：模 3 累加器 . . . 85
5.4.9 未用状态与自启动 (Unused States and Self-Starting) . . . 86
5.5 经典时序逻辑电路 (Classic Sequential Logic Circuits) . . . 86
5.5.1 寄存器 (Registers) . . . 86
5.5.2 使能与时钟门控 (Enable and Clock Gating) . . . 87
5.5.3 寄存器传送与微操作 (Register Transfers and Microoperations) . . . 87
5.5.4 寄存器传送结构 (Register Transfer Structures) . . . 88
5.5.5 移位寄存器 (Shift Registers) . . . 88
5.5.6 计数器 (Counters) . . . 89
5.5.7 波纹计数器 (Ripple Counters) . . . 89
5.5.8 同步计数器 (Synchronous Counters) . . . 89
5.5.9 模 N 计数器与 BCD 计数器 . . . 90
5.5.10 模 N 计数的实现方法 . . . 91
5.6 全章总结 (Summary) . . . 92
5.7 真题与巩固练习 . . . 93

第六章 指令集体系结构与寻址 (ISA & Addressing) 98
6.1 ISA 基础 (Foundations of ISA) . . . 98
6.1.1 ISA 在计算机系统中的位置 (System Stack) . . . 98
6.1.2 什么是指令 (What is an Instruction) . . . 99

6.1.3 操作数的位置 (Where Are the Operands) . . . 99
6.1.4 指令格式与指令集 (Instruction Formats & Instruction Set) . . . 99
6.1.5 ISA 的定义与机器状态 (ISA Specifies State Change) . . . 100
6.1.6 ISA 像“方言”(Different ISAs as Dialects) . . . 100
6.1.7 指令格式设计思路 (Thoughts of Instruction Formats) . . . 100
6.1.8 四条设计原则 (Design Principles) . . . 100
6.2 ISA 设计问题 (Issues in ISA Design) . . . 101
6.2.1 基本设计问题 (Basic Design Issues) . . . 101
6.2.2 设计权衡决策 (Design Tradeoff Decisions) . . . 101
6.3 操作数个数 (Number of Operands) . . . 102
6.3.1 为什么很少见四地址指令 . . . 102
6.3.2 三操作数指令 (Three-Operand Instructions) . . . 102
6.3.3 二操作数指令 (Two-Operand Instructions) . . . 102
6.3.4 一操作数指令 (One-Operand Instructions) . . . 103
6.3.5 零操作数指令与栈 (Zero-Operand Instructions and Stack) . . . 103
6.3.6 操作数个数的取舍 . . . 104
6.4 寻址方式 (Addressing Modes) . . . 104
6.4.1 有效地址 (Effective Address) . . . 104
6.4.2 立即寻址 (Immediate Addressing) . . . 104
6.4.3 直接寻址 (Direct Addressing) . . . 104
6.4.4 间接寻址 (Indirect Addressing) . . . 105
6.4.5 寄存器直接与寄存器间接 (Register Direct / Indirect) . . . 105
6.4.6 相对、变址与基址寻址 (Relative, Indexed and Based) . . . 105
6.4.7 寻址方式总结表 (Summary of Addressing Modes) . . . 105
6.4.8 寻址示例 (Addressing Example) . . . 106
6.5 操作类型与指令编码 (Operations and Encodings) . . . 106
6.5.1 操作类型 (Types of Operations) . . . 106
6.5.2 典型操作集合 (Typical Operations) . . . 106
6.5.3 编码类型 (Types of Encodings) . . . 107
6.6 ISA 演化与 CISC/RISC (Evolution and CISC/RISC) . . . 107
6.6.1 指令集的演化 (Evolution of Instruction Sets) . . . 107
6.6.2 CISC：复杂指令集计算机 . . . 108
6.6.3 RISC：精简指令集计算机 . . . 109
6.6.4 典型 RISC 特征 (A Typical RISC) . . . 109
6.6.5 MIPS 指令格式示例 . . . 109
6.6.6 CISC 与 RISC 对比 . . . 110
6.6.7 CISC/RISC 的现实复杂性 . . . 110
6.6.8 现代 x86 与近期趋势 . . . 110
6.7 ISA 分类与操作数位置 (Classification of ISAs) . . . 111
6.7.1 按操作数位置分类 . . . 111

6.7.2 代码序列：C = A + B 在四类指令集中的表示 . . . 111
6.7.3 为什么几乎所有新架构都使用 GPR . . . 112
6.7.4 栈架构 (Stack Architectures) . . . 112
6.7.5 累加器架构 (Accumulator Architectures) . . . 112
6.7.6 内存-内存架构 (Memory-Memory Architectures) . . . 113
6.7.7 寄存器-内存架构 (Register-Memory Architectures) . . . 113
6.7.8 装载-存储架构 (Load-Store Architectures) . . . 113
6.7.9 寄存器的优缺点 (Registers: Advantages and Disadvantages) . . . 114
6.8 复习抓手与补充知识 (Review Notes) . . . 114
6.8.1 ISA 与微体系结构的区别 . . . 114
6.8.2 评估 ISA 的常用指标 . . . 114
6.8.3 常见易混点 . . . 115
6.9 全章总结 (Summary) . . . 115
6.10 真题与巩固练习 . . . 116

第七章 RISC-V 指令、汇编与机器码 117

7.1 RISC-V ISA 概览 (RISC-V ISA Overview) . . . 117
7.1.1 什么是 RISC-V? . . . 117
7.1.2 RISC-V 的设计目标与原则 . . . 117
7.1.3 用户级 ISA 与标准扩展 . . . 118
7.1.4 基础 RISC-V 与处理器状态 . . . 118
7.1.5 RV32I 指令速览 (RV32I Instruction Quick Reference) . . . 119
7.1.6 动态指令混合 (Dynamic Instruction Mix) . . . 120
7.2 指令编码与格式 (Instruction Encoding and Formats) . . . 121
7.2.1 混合长度编码 . . . 121
7.2.2 核心字段位置 . . . 121
7.2.3 六类常用指令格式 . . . 122
7.2.4 立即数编码变体 . . . 122
7.2.5 RISC-V 寻址方式 . . . 122
7.3 RV32I 指令系统 (RV32I Instruction Set) . . . 123
7.3.1 RV32I 指令类别总览 . . . 123
7.3.2 ALU 指令：R-Type . . . 123
7.3.3 按位逻辑与移位 . . . 123
7.3.4 ALU 指令：I-Type . . . 124
7.3.5 ALU 指令：U-Type 与大立即数 . . . 124
7.4 内存访问、字节序与对齐 (Memory Access, Endianness and Alignment) . . . 125
7.4.1 Load/Store 架构 . . . 125
7.4.2 常用 Load/Store 指令 . . . 125
7.4.3 字节序 (Endianness) . . . 125
7.4.4 内存对齐 (Memory Alignment) . . . 126

7.4.5 数组寻址例：`g = h + A[i]` . . . 126
7.5 控制转移指令 (Control Transfer Instructions) . . . 126
7.5.1 跳转与分支的共同特点 . . . 126
7.5.2 `jal` 与 `jalr` . . . 127
7.5.3 条件分支 B-Type . . . 127
7.5.4 编译 if 语句 . . . 127
7.5.5 更多条件与边界检查技巧 . . . 128
7.5.6 编译 while 循环 . . . 128
7.5.7 远距离分支 . . . 128
7.5.8 switch 与跳转表 . . . 128
7.5.9 NOP 与机器码解码 . . . 129
7.5.10 立即数编码变体 (Immediate Encoding Variants) . . . 129
7.6 指令编码速查 (Encoding Reference) . . . 131
7.6.1 常用 R-Type 与 I-Type 编码 . . . 131
7.6.2 Load/Store、Branch、U/J-Type 编码 . . . 131
7.7 特权 ISA 与软件栈 (Privileged ISA and Software Stack) . . . 132
7.7.1 特权模式 . . . 132
7.7.2 实现层次与特权软件栈 . . . 132
7.8 汇编语言与调用约定 (Assembly Language and Calling Convention) . . . 132
7.8.1 函数调用角色 . . . 132
7.8.2 函数调用六步 . . . 133
7.8.3 函数入口与出口 . . . 133
7.8.4 寄存器约定 . . . 133
7.8.5 栈与栈指针 . . . 134
7.8.6 叶函数示例 . . . 134
7.8.7 非叶函数与递归 . . . 134
7.8.8 递归的缺点 . . . 135
7.8.9 调用前后保持与不保持的内容 . . . 135
7.8.10 C 变量存储类别与内存布局 . . . 136
7.8.11 栈帧 (Stack Frame) . . . 136
7.9 从源代码到运行程序 (From Source Code to A Running Program) . . . 136
7.9.1 整体流程 . . . 136
7.9.2 编译器与预处理器 . . . 137
7.9.3 汇编器与伪指令 . . . 137
7.9.4 汇编指示符 (Assembler Directives) . . . 137
7.9.5 Hello World 汇编结构 . . . 137
7.10 ELF、链接与装载 (ELF, Linking and Loading) . . . 138
7.10.1 ELF 基础 . . . 138
7.10.2 三类目标文件 . . . 138
7.10.3 可重定位目标文件 . . . 139

7.10.4 链接器的作用 . . . 139
7.10.5 地址修正与符号解析 . . . 139
7.10.6 可执行文件格式 . . . 140
7.10.7 静态链接与动态链接 . . . 140
7.10.8 静态库与共享库 . . . 140
7.10.9 装载器 (Loader) . . . 140
7.10.10 动态装载、PIC 与 GOT . . . 141
7.10.11 Lazy Linkage . . . 141
7.11 补充主题与课件结尾 (Additional Topics) . . . 141
7.11.1 编程语言演进与自举 . . . 141
7.11.2 Reflections on Trusting Trust . . . 142
7.11.3 真实入口点：`_start` 与 `crt0` . . . 142
7.11.4 Quiz：符号放在哪里? . . . 142
7.12 全章总结 (Summary) . . . 143
7.13 真题与巩固练习 . . . 143

第八章 处理器性能与单周期 CPU 146
8.1 计算机体系结构概览 (Computer Architecture Overview) . . . 146
8.1.1 层次结构 (System Stack) . . . 146
8.1.2 冯 · 诺依曼结构 (Von Neumann Structure) . . . 146
8.1.3 多核处理器示例 (Multicore Example: AMD Barcelona) . . . 147
8.2 CPU 内部结构 (Inside the Processor) . . . 147
8.2.1 CPU 的三大核心部件 (Datapath, Control & Cache) . . . 147
8.2.2 三种设计方法论 (Three Design Methodologies) . . . 147
8.2.3 可编程与非可编程系统 (Programmable vs Non-programmable) . . . 148
8.2.4 CPU 的本质 (CPU as a General Digital System) . . . 148
8.3 CPU 关键部件详解 (Key CPU Components) . . . 148
8.3.1 算术逻辑运算单元 (ALU) . . . 148
8.3.2 存储器 (Memory) . . . 148
8.3.3 寄存器 (Register) . . . 149
8.3.4 寄存器文件 (Register File) . . . 149
8.3.5 立即数生成单元 (Immediate Generation Unit, ImmGen) . . . 149
8.3.6 RISC-V 寄存器约定 (RISC-V Register Conventions) . . . 150
8.3.7 RISC-V 指令字段 (RISC-V Instruction Fields) . . . 150
8.3.8 RISC-V 汇编语言概览 (RISC-V Assembly Language) . . . 151
8.3.9 指令编码示例 (Instruction Encoding Examples) . . . 151
8.3.10 逻辑设计约定 (Logic Design Conventions) . . . 151
8.4 CPU 性能分析 (CPU Performance Analysis) . . . 151
8.4.1 基本性能指标 (Key Metrics) . . . 151
8.4.2 CPU 时钟与时间公式 (CPU Clock & Time Formula) . . . 152

8.5 性能提升与三堵墙 (Performance Improvement & Three Walls) . . . 153
8.5.1 性能改进方向 (Performance Improvement Directions) . . . 153
8.5.2 三堵墙 (Three Walls) . . . 154
8.6 多处理器与 Amdahl 定律 (Multiprocessors & Amdahl's Law) . . . 154
8.6.1 多处理器 (Multiprocessors) . . . 154
8.6.2 Amdahl 定律 (Amdahl's Law) . . . 154
8.7 构建数据通路 (Building the Datapath) . . . 156
8.7.1 本课程简化 RISC-V CPU 所支持的指令集 . . . 156
8.7.2 指令执行的五个阶段 (Five Stages of Instruction Execution) . . . 156
8.7.3 取指单元 (Instruction Fetch Unit) . . . 156
8.7.4 各类型指令的数据流 (Data Paths by Instruction Type) . . . 157
8.7.5 组合数据通路的原则 (Composing the Elements) . . . 158
8.7.6 完整单周期数据通路 (Full Single-Cycle Datapath) . . . 158
8.8 控制器设计 (Controller Design) . . . 159
8.8.1 控制器的设计思路 (Design Approach) . . . 159
8.8.2 控制信号总览 (Control Signals Overview) . . . 159
8.8.3 两级译码结构 (Two-Level Decoder) . . . 159
8.8.4 各指令的控制信号真值表 (Control Signals Truth Table) . . . 160
8.8.5 控制器 Verilog 实现 (Controller Verilog Implementation) . . . 160
8.9 单周期实现的性能分析 (Single-Cycle Implementation Performance) . . . 161
8.9.1 关键路径分析 (Critical Path Analysis) . . . 161
8.9.2 单周期实现的缺陷 (Limitations of Single-Cycle) . . . 162
8.10 全章总结 (Summary) . . . 162
8.11 真题与巩固练习 . . . 163

第九章 多周期处理器 (Multi-Cycle Processor) 166
9.1 为什么需要多周期? (Why Multi-Cycle?) . . . 166
9.1.1 提升程序性能的三个方向 . . . 166
9.1.2 单周期 vs. 多周期对比 . . . 166
9.1.3 多周期 CPU 的基本设计原则 . . . 167
9.2 多周期数据通路 (Multi-Cycle Datapath) . . . 167
9.2.1 数据通路总体变化 . . . 167
9.2.2 新增的临时寄存器 (New Temporary Registers) . . . 167
9.2.3 控制信号汇总 (Control Signals Summary) . . . 168
9.3 指令执行的各阶段 (Instruction Execution Stages) . . . 169
9.3.1 阶段一：取指 (IF, Instruction Fetch) . . . 169
9.3.2 阶段二：译码与读取操作数 (ID, Instruction Decode & Register Fetch) . . . 169
9.3.3 阶段三：执行 (EX, Execute) . . . 170
9.3.4 阶段四：访存 (MEM, Memory Access) . . . 170
9.3.5 阶段五：写回 (WB, Write Back) . . . 170

9.3.6 各类指令的阶段数汇总 . . . 170
9.4 控制器设计 (Controller Design) . . . 170
9.4.1 控制器设计的分析框架 . . . 170
9.4.2 为什么单周期的控制器设计方式不适用? . . . 171
9.4.3 两种控制器实现方案 . . . 172
9.5 硬连线控制器：有限状态机 (Hardwired Controller: FSM) . . . 172
9.5.1 FSM 设计思路 . . . 172
9.5.2 各状态定义与转移 . . . 172
9.5.3 硬连线控制器的电路实现 . . . 174
9.6 微程序控制器 (Micro-Program Controller) . . . 174
9.6.1 历史背景 . . . 174
9.6.2 基本原理与术语 . . . 175
9.6.3 微程序控制器的实现架构 . . . 175
9.6.4 微程序控制器的两个核心问题 . . . 175
9.6.5 优缺点总结 . . . 176
9.7 多周期 CPU 的性能分析 (Performance Analysis) . . . 176
9.7.1 CPI 计算 . . . 176
9.7.2 关键路径与时钟周期 . . . 176
9.7.3 性能对比计算 . . . 177
9.7.4 多周期设计的意义 . . . 177
9.8 全章总结 (Summary) . . . 177
9.9 真题与巩固练习 . . . 178

考前检查清单 **180**

全书复习路线

三轮复习建议

1. **第一轮：建框架。**按章掌握概念、数据流和结构关系；对每个公式都要知道适用条件。
2. **第二轮：按题型训练。**重点练习编码/解码、K 图、组合/时序电路设计、RISC-V 机器码、CPU 控制与性能计算。
3. **第三轮：限时混合卷。**每道题先定位章节和题型，做完后记录错因，而不只是记答案。

第一章　信息表示 (Information Representation)

本章考点

高频任务：进制转换、原/反/补/移码、符号扩展与截断、IEEE 754 编码与解码、大小端。**复习路线**：先确认位宽与解释方式，再进行数值运算；遇到浮点题固定拆成 sign–exponent–fraction 三步。

1.1　信息表示基础 (Foundations of Information Representation)

1.1.1　信息与信号 (Information & Signals)

- **数字系统 (Digital Systems)**：变量取离散值。二进制值（0 和 1，False/True，Low/High）是数字系统中最常见的表示形式。
- **采用二进制的原因**：物理器件容易实现（例如高低电平、磁场方向、表面凹凸、电荷有无），抗干扰能力强，易于进行逻辑运算。

信号的分类 (Classification of Signals)

按照取值与时间是否连续，信号可分为：

- **模拟信号 (Analog)**：取值和时间均连续。
- **异步数字信号 (Asynchronous)**：取值离散，但时间连续。
- **同步数字信号 (Synchronous)**：取值和时间均离散（数字系统的目标形态）。

在物理实现上，电压信号存在一个**阈值区 (Threshold Region)**，高于/低于阈值分别判定为 1/0，阈值区的存在正是二进制抗干扰能力强的根本原因。

1.1.2　外部信息与内部数据 (External Information & Internal Data)

从不同视角看，计算机处理的数据可分层理解：

- **外部信息 (External Information)**：文本、图像、声音、视频等，通过输入/输出设备与用户交互，在高级程序员视角下还包括图、树、链表、队列等**结构化数据**，最终都以二进制编码存储。

- **内部数据 (Internal Data)**：从底层程序员／硬件设计者视角看，基本类型 (Primitive Types) 分为两大类：
  - **数值型 (Numeric)**：整数（定点数，又分无符号 Unsigned 与有符号 Signed）和实数（浮点数）。
  - **非数值型 (Non-numeric)**：位串 (Bits)、字符／字符串 (Character/String)。

1.2　数制与整数表示 (Radix Systems & Integer Representation)

1.2.1　数制与转换 (Radix Number Systems & Conversions)

计算机中常见进制包括：二进制 (Base 2)、八进制 (Base 8)、十进制 (Base 10) 和十六进制 (Base 16)。

任意进制到十进制 (R-base to Decimal)

按权展开求和：

$$(Number)_r = \sum_{i=0}^{n-1} A_i \cdot r^i + \sum_{j=-m}^{-1} A_j \cdot r^j$$

例如：$(100101.01)_2 = 1 \times 2^5 + 1 \times 2^2 + 1 \times 2^0 + 1 \times 2^{-2} = 32 + 4 + 1 + 0.25 = (37.25)_{10}$

十进制到任意进制 (Decimal to R-base)

- **整数部分**：除基取余法（Repeated Division），逆序排列。
- **小数部分**：乘基取整法（Repeated Multiplication），顺序排列。

注：小数部分的转换可能会产生无限循环（如将十进制的 *0.65* 转换为二进制），实际应用中需根据位宽进行截断 *(Truncate)* 或舍入 *(Round)*。

二进制与八进制/十六进制的互相转换

- **二进制 ↔ 八进制**：以小数点为界，向两侧每 3 位分为一组，不足补零。
- **二进制 ↔ 十六进制**：以小数点为界，向两侧每 4 位分为一组，不足补零。

1.2.2　整数的内部表示 (Internal Representation of Integers)

机器数与真值 (Machine Number and True Value)

机器数是数据在计算机内部的二进制编码，真值是其对应的实际数值。对于带符号整数，最高位为符号位（0 正 1 负）。

整数的编码方式 (Encoding Methods for Integers)

四种主要的编码方式

1. **原码 (Sign-and-Magnitude)**：最高位为符号位，其余位表示数值的绝对值。存在 $+0$ 和 -0 两种表示。
2. **反码 (One's Complement)**：正数的反码是其本身；负数的反码是符号位保持不变，其余各位按位取反。
3. **补码 (Two's Complement)**：广泛使用的表示法。正数补码是其本身；负数补码是其**反码加 1**。
   - 只有唯一的 0 表示。
   - n 位补码表示范围：$-2^{n-1} \le X \le 2^{n-1}-1$。负数比正数多一个（即 -2^{n-1} 没有对应的正数）。
   - 取反加一等式：$\sim x + 1 = -x$。其原理是 $\sim x + x = 111\ldots1 = -1$。
4. **移码 (Frameshift Code / Biased Notation)**：常用于浮点数的阶码表示。给真值加上一个偏置常数（Bias）得到，等同于补码符号位取反。

补码的数学定义与求补码的快捷方法 (Definition & Shortcut)

从"基的补"角度，补码有两种形式化定义（设 N 为 n 位二进制数）：

- **反基补码 (Diminished Radix Complement)**：对基数 r 即 $(r-1)$ 的补，二进制下为 1's complement，定义为 $(r^n-1)-N$。由于 r^n-1 全为 1，且 $1-0=1$，$1-1=0$，因此等价于**逐位取反**。
- **基补码 (Radix Complement)**：对基数 r 即 r 的补，二进制下为 2's complement，定义为 r^n-N，即反码加 1。

求补码的快捷方法（从最低位向最高位扫描）：保留末尾所有的 0，保留遇到的第一个 1，此后的各位**全部取反**。例如 10010100 的补码为 01101100（下划线 100 保留，左侧取反）。

符号扩展与截断 (Sign Extension and Truncation)

- **符号扩展 (Sign Extension)**：当小字长转换为大字长时。无符号数高位补 0；有符号数高位补符号位（0 或 1）。
- **截断 (Truncating)**：大字长转换为小字长时，直接丢弃高位，保留低位。可能会改变原有数值，相当于对 2^k 取模。

1.3　浮点数表示 (Floating-Point Representation)

1.3.1　二进制小数基础 (Fractional Binary Numbers)

分数二进制的按权展开：

$$\sum_{k=-j}^{i} b_k \times 2^k$$

小数点右侧的位代表 $2^{-1}(1/2)$, $2^{-2}(1/4)$, $2^{-3}(1/8)$ 等。其局限性在于只能精确表示形如 $x/2^k$ 的有理数，其他有理数（如 1/3, 1/10）会产生无限循环位表示。这引出了后续将学习的 IEEE 754 浮点数标准。

1.3.2　IEEE 754 标准 (IEEE 754 Standard)

为了统一计算机内部小数的表示，IEEE 制定了 754 浮点数标准。浮点数在底层被表示为科学计数法的形式：

$$V = (-1)^s \times M \times 2^E$$

- **符号 (Sign, s)**：决定数值的正负。$s = 0$ 为正，$s = 1$ 为负。
- **尾数 (Significand/Mantissa, M)**：一个二进制小数，决定了数值的精度。
- **阶码 (Exponent, E)**：对基数 2 进行加权，决定了数值的范围。

在计算机存储中，浮点数被划分为三个相连的位字段：

1. **符号位 (`s`)**：直接编码符号 s。
2. **阶码字段 (`exp`)**：编码阶码 E。
3. **小数字段 (`frac`)**：编码尾数 M。

主要精度类型与相对精度 (Precision Types & Relative Precision)

- **单精度 (Single Precision, `float`)**：共 32 位。分配为：1 位符号位，8 位阶码，23 位小数。
  - **偏置常数 (Bias)**：$2^{8-1} - 1 = 127$。
  - **相对精度**：最小分辨率约 2^{-23}，等效于 $23 \times \log_{10} 2 \approx 23 \times 0.3 \approx 7$ 位十进制有效数字。
- **双精度 (Double Precision, `double`)**：共 64 位。分配为：1 位符号位，11 位阶码，52 位小数。
  - **偏置常数 (Bias)**：$2^{11-1} - 1 = 1023$。
  - **相对精度**：最小分辨率约 2^{-52}，等效于 $52 \times \log_{10} 2 \approx 52 \times 0.3 \approx 16$ 位十进制有效数字。

关于偏置常数的设计目的 (Why Bias?)

引入偏置常数的核心目的是将带正负号的实际指数 e 映射为**无符号整数**阶码 E ($E = e + \text{Bias}$)。这种设计避免了使用补码表示指数所带来的大小比较复杂性。计算公式采取 $2^{k-1} - 1$（取了剔除全 0 和全 1 特殊值后的中间区间），使得指数分布更平衡，并具有符合科学计算需求的轻微正数倾向性。

1.3.3　浮点数的分类 (Categories of Floating-Point Values)

根据阶码字段 (`exp`) 的位模式，浮点数被解析为以下三种不同的情况：

规格化值 (Normalized Values)

这是最普遍的情况。当 `exp` 的位模式**既不全为 0，也不全为 1** 时，该数值为规格化数。

- **真实阶码** E：$E = \exp - \text{Bias}$。
- **真实尾数** M：$M = 1 + f$。这里隐含了一个开头的"1"（因为非零二进制数的最高有效位总是 1，所以不需要存），尾数值相当于 $1.f_{n-1}\ldots f_0$。

非规格化值 (Denormalized Values)

当 `exp` 的位模式**全为 0** 时，该数值为非规格化数。主要用于表示 0 以及非常接近 0 的数（渐进式下溢）。

- **真实阶码** E：$E = 1 - \text{Bias}$（注意这里不是 $0 - \text{Bias}$，这种设计是为了与规格化数平滑过渡）。
- **真实尾数** M：$M = f$。不再隐含开头的 1，即 $0.f_{n-1}\ldots f_0$。
- **零的表示**：当 `exp` 全为 0 且 `frac` 全为 0 时，表示零。根据符号位的不同，有 $+0.0$ 和 -0.0 之分。

特殊值 (Special Values)

当 `exp` 的位模式**全为 1** 时，表示特殊值。

- **无穷大 (Infinity)**：当 `frac` 全为 0 时，表示无穷大。$s = 0$ 为 $+\infty$，$s = 1$ 为 $-\infty$。通常产生于除以零或溢出等操作。
- **NaN (Not a Number)**：当 `frac` 不全为 0 时，表示"不是一个数"。通常产生于无意义的数学运算，如 $\sqrt{-1}$ 或 $\infty - \infty$。

微型浮点数示例 (Tiny Floating Point Example)

为便于手算理解三类取值的分界，常用一个 8 位微型浮点格式：1 位符号、4 位阶码（Bias $= 2^{4-1} - 1 = 7$）、3 位小数。它与 IEEE 标准的结构完全相同，同样包含规格化、非规格化、0、∞ 与 NaN。

- 最小非规格化正数：`0 0000 001`，$E = 1 - 7 = -6$，$M = 1/8$，值为 $\frac{1}{8} \times 2^{-6} = 2^{-9}$。
- 最大非规格化数与最小规格化数在数轴上**平滑衔接**（这正是非规格化数取 $E = 1 - \text{Bias}$ 的目的）。
- 最大规格化数：`0 1110 111`，$E = 14 - 7 = 7$，$M = 1.875$，值为 $1.875 \times 2^7 = 240$。

1.3.4　浮点数编码与运算特性 (Encoding and Arithmetic Properties)

IEEE 754 在设计位排布时采用了 [符号位 | 阶码 | 尾数] 的固定顺序，这赋予了浮点数在硬件底层处理上的极大优势：

- **零的统一 (FP Zero Same as Integer Zero)**：浮点数 +0.0 的所有二进制位全为 0，这与无符号整数 0 的二进制表示完全一致。因此在进行内存级操作（如 C 语言中的 `memset` 清零）时，可以直接按整数逻辑处理。
- **无符号整数比较优化 (Unsigned Integer Comparison)**：由于阶码位于高位，且已被 Bias 机制转换为无符号数，绝大部分浮点数的大小逻辑（如非规格化数与规格化数、规格化数与无穷大之间）与其二进制按无符号整数解析出的大小逻辑是一致的。这意味着 CPU 可以直接复用**整数比较器电路**来比较同号浮点数，极大优化了硬件性能（需特殊处理的仅有符号位、$-0.0 = 0.0$ 判定以及 NaN）。

舍入 (Rounding)

由于二进制浮点数的表示精度有限，当某个实数无法精确表示时，必须采取舍入操作。IEEE 754 定义了四种舍入模式，其中默认且最常用的是**向偶数舍入 (Round-to-Even)**。

它试图找到最接近的匹配值。这种方法可以避免传统的“逢五进一”在统计中产生系统性的偏大偏差。具体规则如下（假设要舍入到保留位数的最后一位）：

- **小于一半** ($< 1/2$)：直接向下舍入（舍弃后续位）。
- **大于一半** ($> 1/2$)：向上舍入（保留位的最低有效位加 1）。
- **恰好一半** ($= 1/2$)：即待舍弃部分恰好是 $100\ldots_2$ 的形式。此时观察保留部分的**最低有效位**：
  - 若为 **1 (奇数)**，则向上进位，使其变为 0。
  - 若为 **0 (偶数)**，则直接舍弃，保持不变。
  - 核心原则：最终结果的最低有效位必然为 0（即“偶数”）。

示例：舍入到最近的 $1/4$（保留二进制小数点后 2 位）

为了清晰说明，我们观察以下数值如何舍入到二进制小数点后两位（即根据小数点后第三位及其之后的位来决定动作）。

原始值	二进制表示	舍弃部分分析	舍入动作	舍入结果
$2\frac{3}{32}$	$10.00\mathbf{011}_2$	011_2 (小于 100_2)	向下舍入 (Down)	10.00_2 ($= 2$)
$2\frac{3}{16}$	$10.00\mathbf{110}_2$	110_2 (大于 100_2)	向上进位 (Up)	10.01_2 ($= 2\frac{1}{4}$)
$2\frac{7}{8}$	$10.11\mathbf{100}_2$	100_2 (等于 100_2)	**向偶数舍入** (末位是 1, 进位变 0)	11.00_2 ($= 3$)
$2\frac{5}{8}$	$10.10\mathbf{100}_2$	100_2 (等于 100_2)	**向偶数舍入** (末位是 0, 直接舍弃)	10.10_2 ($= 2\frac{1}{2}$)

浮点数运算的特性：缺乏结合律 (Lack of Associativity)

由于浮点数存在精度限制（Precision Limit），其加法运算在计算机底层不满足数学上的结合律：

$$(x+y)+z \neq x+(y+z)$$

示例分析

设 $x=-1.5\times10^{38}$, $y=1.5\times10^{38}$, $z=1.0$。

- **计算** $(x+y)+z$：先计算 $x+y=0.0$，再加上 z，结果为 **1.0**。
- **计算** $x+(y+z)$：由于单精度浮点数仅有约 7 位有效数字，在计算 $y+z$ 时，1.0 相对于 1.5×10^{38} 过于微小，在对阶操作中会被舍弃（Rounding off），导致 $y+z\approx1.5\times10^{38}$。最后加上 x，结果为 **0.0**。

结论与影响：在高性能计算和并行计算（Parallel Computing）中，由于计算任务常被拆分并行执行，操作顺序的不确定性可能导致微小的计算误差。在编写涉及大规模数据求和的算法（如深度学习中的梯度累加）时，需特别注意此类精度风险。

数值误差引发的真实灾难 (Disasters Attributable to Bad Numerics)

浮点／定点误差并非纯理论问题，历史上曾酿成严重事故：

- **爱国者导弹拦截失败 (1991)**：海湾战争期间，爱国者导弹系统因把开机以来的时间用浮点累加（0.1 秒无法精确表示），长时间运行后**累积时间误差**，导致未能拦截飞毛腿导弹，造成 28 名士兵阵亡。
- **阿丽亚娜 5 号火箭爆炸 (1996)**：惯性参考系统将一个 64 位浮点数转换为 16 位有符号整数时**溢出**，引发连锁故障，火箭升空 40 秒后解体，损失约 5 亿美元。

1.4　非数值数据与存储 (Non-numeric Data & Storage)

1.4.1　非数值数据的表示 (Representation of Non-numeric Data)

计算机不仅需要处理数值，还需要处理逻辑值、字符等非数值型数据。这些数据在底层同样必须被编码为二进制形式。

十进制数的二进制编码 (Binary Coded Decimal, BCD)

为了方便计算机处理十进制数据，通常使用 BCD 码来表示十进制数字。

- **有权 BCD 码 (Weighted BCD Code)**：每一位二进制数都有一个固定的权重。最常用的是 **8421 码**。
- **无权 BCD 码 (No weight BCD Code)**：没有固定权重的编码方案，常用的有 **余 3 码 (Excess-3 code)** 和 **格雷码 (Gray code)**。

逻辑值的表示 (Representation of Logical Values)

在通常情况下，一个字（Word）或其他可寻址的数据单元被视为一个整体。但在某些逻辑操作中，我们需要将一个 n 位的数据视为由 n 个独立的 1 位数据组成，每一位都是 0 或 1，从而表示 n 个逻辑值。

字符与字符串 (Alphanumeric Codes)

- **ASCII 码**：美国信息交换标准代码。它使用 7 位二进制来表示字符，包含 94 个图形打印字符（如大写字母、小写字母、数字 0-9 等）和 34 个非打印控制字符（如退格、回车等）。在 ASCII 中，大小写字母之间的转换只需要翻转第 6 位比特即可。
- **Unicode**：将 ASCII 扩展到了 16 位，提供了 65,536 个通用字符代码空间，主要用于统一编码世界上不同语言的文字。
- **汉字编码**：汉字系统包含输入码（用于键盘输入）、机内码（用于系统内部存储与处理，通常分为区码和位码）以及字形码（用于显示和打印）。

1.4.2　数据宽度与单位 (Data Width and Unit)

- **比特 (Bit)**：构成二进制数的最小单位。
- **字节 (Byte)**：1 字节等于 8 比特。
- **字长 (Word Size)**：机器处理整数型数据（包括内存地址）的标称大小。
  - 32 位机器的字长为 4 字节，这限制了其最大寻址空间为 4GB。
  - 64 位机器的高端系统使用 8 字节字长，如 x86-64 架构支持 48 位地址线，寻址空间可达 256 TB。

在 C 语言中，不同的数据类型在不同架构下的宽度有所区别。例如，在典型的 32 位和 64 位机器上：

- `char`：1 字节。
- `int` 和 `float`：通常为 4 字节。
- `double`：通常为 8 字节。
- 指针 `(pointer)`：在 32 位机器上为 4 字节，在 64 位机器上为 8 字节。

特别注意二进制单位（基于 $2^{10} = 1024$）与十进制单位（基于 $10^3 = 1000$）的区别：

- KB (10^3) vs KiB (2^{10}), MB (10^6) vs MiB (2^{20}), GB (10^9) vs GiB (2^{30})
- 更高单位：T (Tera, 2^{40}), P (Peta, 2^{50}), E (Exa, 2^{60}), Z (Zetta, 2^{70}), Y (Yotta, 2^{80})

1.4.3　主存容量与频率单位 (Units of Capacity and Frequency)

- **容量单位**（基于 2 的幂）：1 KB = 2^{10} bytes，1 MB = 2^{20} bytes，1 GB = 2^{30} bytes 等。
- **频率与传输速率单位**（基于 10 的幂）：1 kb/s = 10^3 bps，1 Mb/s = 10^6 bps 等。

1.4.4　数据存储的字节序 (Byte Ordering)

对于跨越多个字节的多字节数据（如一个 32 位的整型），在内存中如何排列这些字节是一个关键的体系结构设计选择。

假设我们有一个 4 字节的十六进制变量 `x = 0x01234567`，它的起始存储地址为 `0x100`：

1. **大端序 (Big Endian)**：最高有效字节（MSB, `01`）存放在最低地址 (`0x100`)。后续字节依次存放在递增的地址中（`01 23 45 67`）。常用于 Sun SPARC、PPC Mac 以及互联网网络传输协议中。

2. **小端序 (Little Endian)**：最低有效字节（LSB, `67`）存放在最低地址 (`0x100`)。后续字节依次存放在递增的地址中（`67 45 23 01`）。目前主流的 x86 架构均采用小端序。

字节序类型	0x100	0x101	0x102	0x103
Big Endian (大端序)	01	23	45	67
Little Endian (小端序)	67	45	23	01

1.5　全章总结 (Summary)

本章围绕“信息如何在计算机中表示”展开：无论是数值、字符、逻辑值，还是内存中的多字节数据，本质上都需要被编码为二进制位串。复习时可以抓住以下几条主线：

1. **二进制是共同基础**：数字系统使用离散信号表示信息，二进制因易实现、抗干扰、便于逻辑运算而成为计算机内部表示的核心。

2. **数制转换依赖位权**：任意进制到十进制用按权展开，十进制到其他进制分别处理整数部分和小数部分；二进制与八进制、十六进制之间可按 3 位或 4 位分组快速转换。

3. **整数编码关注符号和范围**：原码、反码、补码和移码解决有符号数表示问题，其中补码最常用，必须掌握其表示范围、符号扩展和截断后的数值变化。

4. **浮点数是近似表示**：IEEE 754 通过符号位、阶码和尾数字段表达 $(-1)^s \times M \times 2^E$，并用规格化值、非规格化值和特殊值覆盖不同数值场景。

5. **浮点运算不等同于实数运算**：

   - 浮点运算**不满足结合律与分配律**。
   - 在进行相等判定（如 `if(a == b)`）时极其危险，通常应使用误差范围（Epsilon）进行判定。

6. **非数值数据也需要编码**：BCD、逻辑值、ASCII、Unicode 与汉字编码体现了不同类型信息到二进制形式的映射。

7. **存储细节影响程序理解**：字节、字长、容量单位、频率单位和大小端字节序都会影响数据在内存中的组织方式，也会影响底层调试和跨平台程序行为。

“计算机中的数据不是原始信息本身，而是经过规则编码后的二进制表示。”

1.6　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. **[Quiz]** Represent decimal 75 as (a) a 7-bit binary number and (b) an 8-bit 8421-BCD code. Then place an even-parity bit at the MSB to form a 9-bit code.

2. **[Extra]** Given the bit pattern `0xAD100002`, interpret the value it represents (show your derivation) as each of: (1) a two's complement integer; (2) an unsigned integer; (3) a biased / excess representation; (4) a single-precision floating-point number.

3. **[Exam]** The BCD representation of decimal 34 is? A. `00110100` B. `0011 0100` C. `100010` D. `0010 0010`

4. **[Exam]** What is the 8-bit two's complement representation of $(-35)_{10}$?

5. **[Exam]** Write the IEEE 754 single-precision binary representation of -17.5625.

6. **[Extra]** Write the IEEE 754 single-precision binary representation of -23.3125_{10}.

7. **[Extra]** For the following nine 32-bit values, sort the labels in **ascending order** (sort only; you need not write out the exact values):

Label	Bit pattern	Interpretation
A	0xF0000000	sign-magnitude
B	0xF0000000	two's complement
C	0xF0000000	biased
D	0xF0000000	IEEE 754 float
E	0xF0000000	one's complement
F	0xF7000000	sign-magnitude
G	0xF7000000	two's complement
H	0xF7000000	biased
I	0xF7000000	IEEE 754 float

8. **[Extra]** Number conversion: $(17.5)_8 = (__)_{10} = (__)_{16}$.

9. **[Extra]** Write the **sign-magnitude / one's complement / two's complement** of the 8-bit machine number of $(-7.7)_{10}$ (1 sign bit + integer + 3 fraction bits).

10. **[Extra]** Let `x7 = 0x10000000`, and the data at that address is `0x1122334455667788`. Execute

```
lb x6, 0(x7)
sd x6, 8(x7)
```

and give the value stored at `0x10000008` on a **big-endian** machine and on a **little-endian** machine.

11. **[Class]** Convert each of the following numbers from its given base into the other three bases in the table (decimal / binary / octal / hexadecimal):(1) 369.3125_{10}; (2) 10111101.101_2; (3) $(326.5)_8$; (4) $(D6.A)_{16}$.

12. **[Class]** Find the radix r in each equation:(a) $(BEE)_r = (2699)_{10}$; (b) $(365)_r = (194)_{10}$. (Hint: expand by place value to form an equation, solve the resulting quadratic, and take the sensible positive integer root.)

13. **[Class]** Write the IEEE 754 representation of 63.25_{10} in (a) single precision (32-bit) and (b) double precision (64-bit). Show the sign, exponent, and fraction fields.

14. **[Class]** Given that `0x0C000000` is the bit pattern of an IEEE 754 **single-precision** float, find the real number it represents (show the derivation of sign, exponent, and mantissa).

15. **[Class]** A custom 16-bit floating-point format: 1 sign bit + 5 exponent bits (bias = 15) + 10 fraction bits. Write the binary representation of -1.5625×10^{-1} (i.e. -0.15625) in this format.

第二章　布尔代数与数字逻辑 (Boolean Algebra)

本章考点

高频任务：逻辑表达式与真值表互换、规范 SOP/POS、代数化简、K 图、PI/EPI、门输入成本、NAND/NOR 变换。**复习路线**：先准确建模，再化简；不要在最小项/最大项索引、原变量/反变量规则上丢分。

2.1　数字逻辑导论 (Introduction to Digital Logic)

2.1.1　模拟信号与数字信号 (Analog vs. Digital Signals)

电子系统可以处理连续变化的模拟信号，也可以处理只落在有限离散区间内的数字信号。数字系统刻意限制设计选择，把连续电压抽象为少数逻辑值，从而使电路设计、组合、验证和扩展都更容易。

- **模拟信号 (Analog signals)**：随时间连续变化，理论上可以取无限多种电压值。
- **数字信号 (Digital signals)**：只关心若干离散电压范围。最常见的是二值逻辑：低电平表示 0，高电平表示 1。
- **数字纪律 (Digital discipline)**：不使用连续值作为语义，而是把电压划分为可识别的逻辑区间。

2.1.2　为什么采用数字系统 (Why Digital?)

- **设计更简单**：用离散电压代替连续电压，电路行为更容易描述和组合。
- **抗干扰更强**：只要噪声没有把信号推过阈值，逻辑含义就不会改变。
- **可构造更复杂系统**：简单、可靠的逻辑门可以层层组合，构成处理器、存储器和通信系统。
- **典型替代场景**：数字相机、数字电视、手机、CD 等数字系统逐步替代了许多模拟前身。

2.2　二值逻辑与逻辑门 (Binary Logic and Gates)

2.2.1　二值变量与基本运算 (Binary Variables & Basic Operations)

二值变量只取两个值。课件中常见的同义表达包括 True/False、On/Off、Yes/No、1/0。本课程统一用 1 和 0 表示两个逻辑值，变量名可写作 A, B, X, Y, Z 等。

三个基本逻辑运算

- **与 (AND)**：记作 $X \cdot Y$ 或直接写作 XY。只有两个输入都为 1 时输出为 1。
- **或 (OR)**：记作 $X + Y$。只要至少一个输入为 1，输出就为 1。
- **非 (NOT)**：记作 $\overline{X}$、X' 或 $\sim X$。输出为输入的反值。

2.2.2　真值表 (Truth Tables)

真值表列出一个逻辑函数在所有输入组合下的输出值。对于 n 个输入变量，真值表共有 2^n 行。

X	Y	$X \cdot Y$	$X + Y$	$\overline{X}$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

注意：上表最后一列只依赖 X，因此在 $X = 0$ 的两行都是 1，在 $X = 1$ 的两行都是 0。

2.2.3　用开关建模逻辑函数 (Logic Functions with Switches)

开关模型有助于理解继电器电路和 CMOS 门电路。

- **输入开关**：逻辑 1 表示开关闭合，逻辑 0 表示开关断开。
- **输出灯泡**：逻辑 1 表示灯亮，逻辑 0 表示灯灭。
- **并联开关**：任一支路闭合即可导通，对应 OR。
- **串联开关**：所有开关都闭合才导通，对应 AND。
- **常闭开关**：输入为 0 时闭合、输入为 1 时断开，可用于表示 NOT。

课件中的开关例子可抽象为：开关 A 串联在总路径上，之后分成两条并联支路，其中一条为 B 与 C 串联，另一条为 D。因此灯亮条件为

$$L(A, B, C, D) = A(BC + D).$$

2.2.4　逻辑门类型 (Logic Gate Types)

逻辑门是逻辑函数的电路实现。常见分类如下：

- **单输入门**：反相器 (NOT gate)、缓冲器 (buffer)。
- **双输入门**：AND、OR、XOR、NAND、NOR、XNOR。
- **多输入门**：三输入或更多输入的 AND、OR、NAND、NOR 等。

- **原始门 (Primitive gates)**：结构简单、速度快，如 AND、OR、NOT、NAND、NOR。
- **复杂门 (Complex gates)**：在一个门级结构中实现更复杂表达式，可降低成本与传播时间。

2.2.5 常见逻辑门真值表 (Common Gate Truth Tables)

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

- **NAND**：$\overline{AB}$，只有 $A = B = 1$ 时输出 0。
- **NOR**：$\overline{A + B}$，只有 $A = B = 0$ 时输出 1。
- **XOR**：$A \oplus B = \overline{A}B + A\overline{B}$，表示“二者不同”。
- **XNOR**：$\overline{A \oplus B} = AB + \overline{A}\,\overline{B}$，表示“二者相同”。

2.3 晶体管与逻辑门实现 (Transistors and Gate Implementation)

2.3.1 从继电器到晶体管 (Relays, Vacuum Tubes, Transistors)

早期计算机通过继电器中的电磁线圈控制开关通断。之后真空管以电子方式开闭电流路径，替代了继电器。现代数字系统主要使用晶体管作为电子开关。

- **继电器 (Relay)**：由磁场控制机械开关，速度慢、体积大。
- **真空管 (Vacuum tube)**：用电子方式控制电流，速度更快但功耗和体积仍较大。
- **晶体管 (Transistor)**：体积小、功耗低、可大规模集成，是现代数字逻辑的基础。

2.3.2 Robert Noyce 与集成电路 (Integrated Circuit)

Robert Noyce (1927–1990) 被称为“Mayor of Silicon Valley”。他于 1957 年共同创办 Fairchild Semiconductor，1968 年共同创办 Intel，并且是集成电路的共同发明者之一。

2.3.3 硅与掺杂 (Silicon and Doping)

- **半导体 (Semiconductor)**：晶体管通常由硅制造。纯硅中自由电荷少，导电性较差。
- **掺杂硅 (Doped silicon)**：通过掺入杂质产生自由载流子，从而提高导电能力。
- **n 型硅**：自由负电荷较多，主要载流子是电子 (electrons)。
- **p 型硅**：自由正电荷较多，主要载流子可理解为空穴 (holes)。

2.3.4　IC 逻辑家族与 CMOS 晶体管 (TTL and CMOS)

- **TTL (Transistor-Transistor Logic)**：基于双极结型晶体管 BJT (Bipolar Junction Transistor)。
- **CMOS (Complementary Metal-Oxide Semiconductor)**：基于场效应晶体管 FET (Field Effect Transistor)。
- **CMOS 管脚概念**：gate、source、drain。
- **NMOS 与 PMOS**：NMOS 常在控制信号为 1 时导通，PMOS 常在控制信号为 0 时导通；二者互补使用可以构成低静态功耗的 CMOS 门。

2.3.5　用晶体管实现门电路 (Gate Implementation)

晶体管或真空管对逻辑函数的实现称为逻辑门。课件中给出 NOR、NAND、NOT 的晶体管实现，并强调这些晶体管门电路可以用开关电路进行抽象建模。

- **NOT**：一个输入控制输出与电源或地之间的连接，输出为输入反相。
- **NAND**：先实现“与”的导通条件，再取反；CMOS 中 NAND 往往比 AND 更自然。
- **NOR**：先实现“或”的导通条件，再取反；CMOS 中 NOR 也是常见原始门。

2.4　IC 参数与数字抽象 (IC Parameters and Digital Abstraction)

2.4.1　电源电压与逻辑电平 (VCC and Logic Levels)

- V_{CC} **或** V_{DD}：逻辑家族的电源电压。课件例子中，许多 HC 器件共用 5V 电源。
- **电源轨 (Power supply rails)**：V_{CC} 与 GND，CMOS 中也常写作 V_{DD} 与 V_{SS}。
- **逻辑 0 与逻辑 1**：例如 0V 可表示 0，5V 可表示 1。但实际电压如 4.99V、3.2V 是否为 1，需要根据输入/输出阈值判断。

2.4.2　噪声与噪声容限 (Noise and Noise Margins)

噪声是任何使信号质量下降的因素，例如导线电阻、电源噪声、相邻导线耦合等。若驱动门输出 5V，经过长导线后接收端只得到 4.5V，只要仍落在合法高电平范围内，接收端仍可判为 1。

四个关键电平参数

- V_{OH}：输出高电平的最低保证电压。
- V_{OL}：输出低电平的最高保证电压。
- V_{IH}：输入被识别为高电平所需的最低电压。
- V_{IL}：输入被识别为低电平所允许的最高电压。

噪声容限为

$$NM_H = V_{OH} - V_{IH}, \qquad NM_L = V_{IL} - V_{OL}.$$

理想缓冲器的直流传输曲线会在中点附近突变，且 $NM_H = NM_L = V_{DD}/2$。真实缓冲器存在过渡区，因此通常 $NM_H, NM_L < V_{DD}/2$。

2.4.3　常见逻辑家族参数 (Logic Family Examples)

Logic Family	V_{DD}	V_{IL}	V_{IH}	V_{OL}	V_{OH}
TTL	5 (4.75–5.25)	0.8	2.0	0.4	2.4
CMOS	5 (4.5–6)	1.35	3.15	0.33	3.84
LVTTL	3.3 (3–3.6)	0.8	2.0	0.4	2.4
LVCMOS	3.3 (3–3.6)	0.9	1.8	0.36	2.7

2.4.4　传播延迟与延迟模型 (Propagation Delay and Delay Models)

输出不会在输入变化的同一瞬间改变。传播延迟 (Propagation Delay) 衡量输入变化到输出响应之间的时间。

- t_{pd}：总称传播延迟。
- t_{PLH}：输出从低到高 (Low-to-High) 的传播延迟。
- t_{PHL}：输出从高到低 (High-to-Low) 的传播延迟。
- **Transport delay**：输入变化后，输出总是在固定延迟后变化。
- **Inertial delay**：若输入造成的输出脉冲窄于拒绝时间 (rejection time)，该窄脉冲会被滤掉，更接近真实电子电路行为。

课件示例中传播延迟为 2.0 ns，拒绝时间为 1.0 ns：宽度不足 1.0 ns 的短脉冲会被惯性延迟模型拒绝。

2.4.5　转换时间、功耗、扇入扇出 (Transition, Power, Fan-in/out)

- **转换时间 (Transition time)**：信号从低电平升到高电平，或从高电平降到低电平所需的时间；也称 slew rate，通常按输出摆幅的 10% 到 90% 测量。

- **静态功耗 (Static power)**：输出不切换时仍消耗的功率，主要由静态供电电流或漏电流造成：

$$P_S = I_{DD} V_{CC}.$$

- **动态功耗 (Dynamic power)**：输出切换时对负载电容充放电造成：

$$P_D \approx C_L V_{CC}^2 f.$$

- **总功耗**：$P_T = P_S + P_D$。CMOS 的静态功耗很低，但动态功耗随负载电容、电源电压平方和切换频率上升。
- **扇入 (Fan-in)**：一个逻辑门最多能接受的输入数量。
- **扇出 (Fan-out)**：一个门的输出最多能驱动多少个标准负载，超过规定负载可能造成逻辑错误或延迟过大。

  扇出会影响传播延迟。课件例子中四输入 NAND 门的延迟模型为

$$t_{pd} = 0.07 + 0.021\,SL \text{ ns},$$

其中 SL 是标准负载数。若 $SL = 4.5$，则 $t_{pd} = 0.1645$ ns。

2.4.6　门成本 (Gate Cost)

在集成电路中，门成本大致与芯片面积成正比；芯片面积又大致与晶体管数量、晶体管尺寸和布线面积有关。若忽略布线面积，门输入数 (gate input count) 可作为门成本的粗略衡量。若已知实际版图面积，则版图面积是更准确的成本指标。

2.5　布尔代数 (Boolean Algebra)

2.5.1　George Boole 与布尔代数

George Boole (1815–1864) 自学数学，后来任教于爱尔兰 Queen's College。他在 1854 年出版 *An Investigation of the Laws of Thought*，引入二值变量以及 AND、OR、NOT 三个基本逻辑运算。布尔代数为数字系统的分析和设计提供数学基础。

2.5.2　公理 (Axioms)

公理	对偶公理
(A1) 若 $X \neq 1$，则 $X = 0$	(A1D) 若 $X \neq 0$，则 $X = 1$
(A2) 若 $X = 0$，则 $\overline{X} = 1$	(A2D) 若 $X = 1$，则 $\overline{X} = 0$
(A3) $0 \cdot 0 = 0$	(A3D) $1 + 1 = 1$
(A4) $1 \cdot 1 = 1$	(A4D) $0 + 0 = 0$
(A5) $0 \cdot 1 = 1 \cdot 0 = 0$	(A5D) $1 + 0 = 0 + 1 = 1$

2.5.3　单变量定理 (Single-Variable Theorems)

定理	对偶定理
(T1) $X+1=1$	(T1D) $X\cdot 0=0$
(T2) $X+0=X$	(T2D) $X\cdot 1=X$
(T3) $X+X=X$	(T3D) $X\cdot X=X$
(T4) $\overline{\overline{X}}=X$	
(T5) $X+\overline{X}=1$	(T5D) $X\cdot\overline{X}=0$

2.5.4　二至三变量定理 (2–3 Variable Theorems)

定理	对偶定理
(T6) $X+Y=Y+X$	(T6D) $XY=YX$
(T7) $(X+Y)+Z=X+(Y+Z)$	(T7D) $(XY)Z=X(YZ)$
(T8) $X(Y+Z)=XY+XZ$	(T8D) $X+YZ=(X+Y)(X+Z)$
(T9) $X+XY=X$	(T9D) $X(X+Y)=X$
(T10) $XY+X\overline{Y}=X$	(T10D) $(X+Y)(X+\overline{Y})=X$
(T11) $XY+\overline{X}Z+YZ=XY+\overline{X}Z$	(T11D) $(X+Y)(\overline{X}+Z)(Y+Z)=(X+Y)(\overline{X}+Z)$

其中 (T11) 是**一致性定理 (Consensus Theorem)**，常用于删除冗余项。

2.5.5　n 变量定理、德摩根定理和香农展开

- **广义幂等律 (Generalized Idempotent Theorem)**：

$$X+X+\cdots+X=X,\qquad X\cdot X\cdot\cdots\cdot X=X.$$

- **德摩根定理 (De Morgan's Theorem)**：

$$\overline{X_1X_2\cdots X_n}=\overline{X_1}+\overline{X_2}+\cdots+\overline{X_n},$$

$$\overline{X_1+X_2+\cdots+X_n}=\overline{X_1}\,\overline{X_2}\cdots\overline{X_n}.$$

- **运算互换形式**：

$$\overline{F(X_1,\ldots,X_n,+,\cdot)}=F(\overline{X_1},\ldots,\overline{X_n},\cdot,+).$$

- **香农展开 (Shannon's Theorem)**：

$$F(X_1,\ldots,X_n)=X_1F(1,X_2,\ldots,X_n)+\overline{X_1}F(0,X_2,\ldots,X_n),$$

$$F(X_1,\ldots,X_n)=[X_1+F(0,X_2,\ldots,X_n)]\,[\overline{X_1}+F(1,X_2,\ldots,X_n)].$$

2.5.6　运算优先级 (Boolean Operator Precedence)

布尔表达式的求值顺序为：

1. 括号；
2. NOT；
3. AND；
4. OR。

因此 $F = A(B + C)(C + D)$ 中，括号内的 OR 先求值，再与 A 相乘。若含义明确，乘号 $\cdot$ 通常省略。

2.5.7　对偶性 (Duality)

一个代数表达式的**对偶 (dual)** 通过交换 $+$ 与 $\cdot$，并交换 0 与 1 得到。若一个恒等式成立，则它的对偶式也成立。

- 例：$x + xy = x$ 成立。
- 两边取反并用德摩根定理：

$$\overline{x + xy} = \overline{x}, \qquad \overline{x}(\overline{x} + \overline{y}) = \overline{x}.$$

- 令 $X = \overline{x}, Y = \overline{y}$，可得

$$X(X + Y) = X,$$

这正是 $x + xy = x$ 的对偶式。

注意：表达式的对偶一般**不等于**原表达式，除非它是自对偶 (self-dual)。例如

$$F = (A + \overline{C})B + 0, \qquad F^D = (A\overline{C} + B) \cdot 1 = A\overline{C} + B.$$

2.5.8　布尔代数证明示例 (Algebraic Proof Examples)

吸收律证明

证明 $A + AB = A$：

$$\begin{aligned} A + AB &= A \cdot 1 + AB \\ &= A(1 + B) \\ &= A \cdot 1 \\ &= A. \end{aligned}$$

一致性定理证明

证明 $AB + \overline{A}C + BC = AB + \overline{A}C$：

$$\begin{aligned} AB + \overline{A}C + BC &= AB + \overline{A}C + BC(A + \overline{A}) \\ &= AB + \overline{A}C + ABC + \overline{A}BC \\ &= AB(1 + C) + \overline{A}C(1 + B) \\ &= AB + \overline{A}C. \end{aligned}$$

德摩根证明例

德摩根律的两个基本二变量形式为：

$$\overline{X + Y} = \overline{X}\,\overline{Y}, \qquad \overline{XY} = \overline{X} + \overline{Y}.$$

课件中的例子：

$$\begin{aligned} G &= \overline{(X + Y)}Z + X\overline{Y} \\ &= \overline{X}\,\overline{Y}Z + X\overline{Y} \\ &= \overline{Y}(\overline{X}Z + X). \end{aligned}$$

再用 $X + \overline{X}Z = X + Z$，可化为

$$G = \overline{Y}(X + Z).$$

2.6　逻辑函数表示与标准形式 (Logic Function Representation)

2.6.1　逻辑函数、真值表与波形 (Functions, Truth Tables, Waveforms)

逻辑函数反映输入变量与输出变量之间的逻辑关系，通常写作

$$F(X_1, X_2, \ldots, X_n).$$

课件强调：逻辑函数可以用真值表、波形图或布尔表达式表示。

- **真值表**：布尔函数的唯一表示；若两个函数真值表完全相同，则函数等价，反之亦然。
- **波形图**：按时间展示输入和输出的变化，适合观察电路时序。
- **布尔表达式**：不唯一，同一个函数可以有许多等价表达式。

真值表规模随变量数指数增长：n 个变量需要 2^n 行。因此变量较多时，直接使用真值表会变得笨重，这正是需要布尔代数和化简方法的原因。

2.6.2　表达式不唯一 (Boolean Expressions Are Not Unique)

同一个函数可以由不同表达式表示。例如下列两个函数真值表相同：

$$F(x, y, z) = \overline{x}\,\overline{y}\,\overline{z} + \overline{x}y\overline{z} + xy\overline{z},$$

$$G(x, y, z) = \overline{x}\,\overline{y}\,\overline{z} + y\overline{z}.$$

二者都在 $xyz = 000, 010, 110$ 时为 1，因此 $F = G$。

2.6.3　函数补 (Complement of a Function)

函数补可以用两种方式得到：

- 在真值表输出列中把 1 改成 0、0 改成 1。
- 对表达式取反，使用德摩根定理交换 $+$ 与 $\cdot$，并对每个变量取反。

函数补**不是**对偶。对偶只交换运算符和常量，不对整个函数取反。

例：求函数补

设

$$F(x,y,z) = x\overline{y}\,\overline{z} + \overline{x}yz.$$

则

$$\begin{aligned} G = F' &= (x\overline{y}\,\overline{z} + \overline{x}yz)' \\ &= \overline{x\overline{y}\,\overline{z}} \cdot \overline{\overline{x}yz} \\ &= (\overline{x} + y + z)(x + \overline{y} + \overline{z}). \end{aligned}$$

2.6.4　文字项、最小项、最大项 (Literals, Minterms, Maxterms)

- **文字 (Literal)**：变量或变量的反，如 X、$\overline{X}$。
- **乘积项 (Product term)**：由若干文字用 AND 相连，如 ABC、$A\overline{C}$。
- **和项 (Sum term)**：由若干文字用 OR 相连，如 $A + B + \overline{C}$。
- **最小项 (Minterm)**：包含全部变量且每个变量恰好出现一次的乘积项。
- **最大项 (Maxterm)**：包含全部变量且每个变量恰好出现一次的和项。

2.6.5　最小项与最大项编号规则

固定变量顺序 (x, y, z) 后，编号 j 的二进制表示 b_j 决定最小项和最大项：

- **最小项** m_j：若某变量在 b_j 中为 0，则该变量取反；若为 1，则不取反。例：$j = 3$，$b_j = 011$，故 $m_3 = \overline{x}yz$。
- **最大项** M_j：若某变量在 b_j 中为 1，则该变量取反；若为 0，则不取反。例：$j = 3$，$b_j = 011$，故 $M_3 = x + \overline{y} + \overline{z}$。

No.	x	y	z	**Minterm**	**Maxterm**
0	0	0	0	$\overline{x}\,\overline{y}\,\overline{z} = m_0$	$x + y + z = M_0$
1	0	0	1	$\overline{x}\,\overline{y}z = m_1$	$x + y + \overline{z} = M_1$
2	0	1	0	$\overline{x}y\overline{z} = m_2$	$x + \overline{y} + z = M_2$
3	0	1	1	$\overline{x}yz = m_3$	$x + \overline{y} + \overline{z} = M_3$
4	1	0	0	$x\overline{y}\,\overline{z} = m_4$	$\overline{x} + y + z = M_4$
5	1	0	1	$x\overline{y}z = m_5$	$\overline{x} + y + \overline{z} = M_5$
6	1	1	0	$xy\overline{z} = m_6$	$\overline{x} + \overline{y} + z = M_6$
7	1	1	1	$xyz = m_7$	$\overline{x} + \overline{y} + \overline{z} = M_7$

2.6.6　规范形式 (Canonical Forms)

在固定变量顺序下，任意布尔函数都有唯一的两种规范形式：

- **规范与或式 (Canonical SOP)**：把真值表中 $F = 1$ 的行对应的最小项相加。

$$F(X_1, \ldots, X_n) = \sum(\text{minterms for 1s}).$$

- **规范或与式 (Canonical POS)**：把真值表中 $F = 0$ 的行对应的最大项相乘。

$$F(X_1, \ldots, X_n) = \prod(\text{maxterms for 0s}).$$

例：若 $f(x, y, z)$ 在行 $1, 2, 4, 6$ 为 1，则

$$f = \sum m(1, 2, 4, 6) = \overline{x}\,\overline{y}z + \overline{x}y\overline{z} + x\overline{y}\,\overline{z} + xy\overline{z}.$$

该函数在行 $0, 3, 5, 7$ 为 0，因此

$$f = \prod M(0, 3, 5, 7).$$

由于 $m_j = M_j'$，所以 $\sum m(1, 2, 4, 6) = \prod M(0, 3, 5, 7)$。

2.6.7　标准形式 (Standard Forms)

标准形式类似规范形式，但每个乘积项或和项不一定包含所有变量，因此**不唯一**。

- **标准 SOP**：乘积项的和，如

$$f_1(x, y, z) = \overline{x}\,\overline{y}z + y\overline{z} + x\overline{z}.$$

- **标准 POS**：和项的积，如

$$f_1(x, y, z) = (x + y + z)(\overline{y} + \overline{z})(\overline{x} + \overline{z}).$$

标准 SOP 转规范 SOP

若乘积项缺少变量 x，乘以 $1 = (x + \overline{x})$ 展开，再删除重复最小项。

标准 POS 转规范 POS

若和项缺少变量 x，加入 $0 = x\overline{x}$ 并用分配律展开，再删除重复最大项。

2.7　逻辑函数化简 (Simplification of Logic Functions)

2.7.1　化简目标与成本准则 (Optimization and Cost Criteria)

逻辑函数化简的目标是得到更简单的电路实现。更简单通常意味着成本更低、面积更小、速度更快。

- **文字成本 L (Literal cost)**：表达式中文字出现的总次数。
- **门输入成本 G (Gate input cost)**：按表达式直接实现时，所有门输入端数量之和；不计反相器输入。
- **含反相器的门输入成本 GN**：在 G 的基础上把反相器输入也计入。

  对于 SOP/POS 表达式，可按下式估算：

$$G = L + \text{非单文字项的项数},$$

$$GN = G + \text{不同取反单文字所需的反相器数}.$$

2.7.2　成本计算示例 (Cost Examples)

表达式	L	G	GN
$F = BD + A\overline{B}C + A\overline{C}\,\overline{D}$	8	11	14
$F = BD + A\overline{B}C + A\overline{B}\,\overline{D} + AB\overline{C}$	11	15	18
$F = (A + B)(A + D)(B + C + \overline{D})(\overline{B} + \overline{C} + D)$	10	14	17

若选择门输入成本作为准则，第一种实现最好。

课件另一个例子：

- $F = ABC + \overline{A}\,\overline{B}\,\overline{C}$：$L = 6, G = 8, GN = 11$。
- $F = (A + \overline{C})(\overline{B} + C)(\overline{A} + B)$：$L = 6, G = 9, GN = 12$。

两个表达式文字成本相同，但第一个表达式的 G 和 GN 更低，因此应选择第一个电路。

2.7.3　代数化简 (Algebraic Manipulation)

布尔代数可直接用于化简电路。

例 1：化简

$$\begin{aligned} F &= \overline{x}yz + \overline{x}y\overline{z} + xz \\ &= \overline{x}y(z + \overline{z}) + xz \\ &= \overline{x}y + xz. \end{aligned}$$

例 2：证明

证明

$$\overline{x}\,\overline{y}\,\overline{z} + \overline{x}y\overline{z} + xy\overline{z} = \overline{x}\,\overline{z} + y\overline{z}.$$

证明：

$$\begin{aligned} \overline{x}\,\overline{y}\,\overline{z} + \overline{x}y\overline{z} + xy\overline{z} &= \overline{x}\,\overline{y}\,\overline{z} + \overline{x}y\overline{z} + \overline{x}y\overline{z} + xy\overline{z} \\ &= \overline{x}\,\overline{z}(\overline{y} + y) + y\overline{z}(\overline{x} + x) \\ &= \overline{x}\,\overline{z} + y\overline{z}. \end{aligned}$$

课件练习补全

若

$$F(X, Y, Z) = \sum m(1, 2, 4, 6),$$

则

$$F = \overline{X}\,\overline{Y}Z + \overline{X}Y\overline{Z} + X\overline{Y}\,\overline{Z} + XY\overline{Z}.$$

可进一步写成

$$F = \overline{Z}(X + Y) + \overline{X}\,\overline{Y}Z.$$

若

$$F(A, B, C) = \sum m(1, 4, 5, 6, 7),$$

则最小项表达式为

$$F = \overline{A}\,\overline{B}C + A\overline{B}\,\overline{C} + A\overline{B}C + AB\overline{C} + ABC,$$

化简为

$$F = A + \overline{B}C.$$

2.7.4　卡诺图基础 (Karnaugh Maps)

卡诺图 (K-map) 是布尔函数的图形化表示。

- n 变量卡诺图有 2^n 个格子。
- 每个格子对应真值表中的一行，也对应一个最小项或最大项。
- 相邻格子只允许一个变量取值不同，因此可以合并并消去该变量。
- 多个相邻格子组成的区域对应一个标准项。

2.7.5　卡诺图排列 (K-map Layouts)

卡诺图边缘采用格雷码顺序，使相邻行/列只变一个变量。

二变量卡诺图

	$X=0$	$X=1$
$Y=0$	m_0	m_2
$Y=1$	m_1	m_3

三变量卡诺图

$Z\backslash XY$	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5

四变量卡诺图

$CD\backslash AB$	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

2.7.6　卡诺图优化原则 (Principles and Rules)

- 相邻两个最小项可合并，如 $AB + A\overline{B} = A$。
- 卡诺图中的圈称为 Karnaugh circles，圈内格子数量必须是 2 的幂：1、2、4、8、...。
- 每个 1 至少被圈一次。
- 每个圈应尽量大。
- 圈可以跨越边界，左右边相邻，上下边也相邻。
- 不关心项 (don't care, X) 只有在有助于化简时才圈入。

2.7.7　蕴含项、主蕴含项、必要主蕴含项 (Implicants)

- **蕴含项 (Implicant)**：一个能覆盖若干函数值为 1 的最小项的乘积项，如 ABC、AC、BC。
- **主蕴含项 (Prime implicant)**：在卡诺图中无法再扩大的圈所对应的乘积项。
- **必要主蕴含项 (Essential prime implicant)**：至少覆盖一个没有被其他主蕴含项覆盖的最小项的主蕴含项。

- **选择规则**：尽量减少主蕴含项之间的重叠；最终选择的每个主蕴含项最好至少覆盖一个其他已选项没有覆盖的最小项。

2.7.8　三变量卡诺图例题

课件例题：

$$F(X,Y,Z)=\sum m(3,5,6,7).$$

最小项为 011、101、110、111，即至少两个变量为 1 时输出为 1。卡诺图可圈出三组相邻对：

$$m_6,m_7\Rightarrow XY,\qquad m_5,m_7\Rightarrow XZ,\qquad m_3,m_7\Rightarrow YZ.$$

因此

$$F=XY+XZ+YZ.$$

2.7.9　四变量卡诺图例题

课件给出的四变量函数真值表对应卡诺图输出为

$CD\backslash AB$	00	01	11	10
00	1	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	1

化简结果为

$$Y=\overline{A}C+\overline{A}BD+A\overline{B}\,\overline{C}+\overline{B}\,\overline{D}.$$

2.7.10　带不关心项的例题 (Don't Cares)

若某些输入组合不会出现，或其输出值无关紧要，可记作 don't care。卡诺图中可把它当作 1 使用，也可当作 0 使用，取决于哪种选择能降低成本。

课件例题中，不关心项帮助扩大圈，得到更简单的表达式：

$$Y=A+\overline{B}\,\overline{D}+C.$$

2.7.11　系统化化简示例 (Systematic Simplification)

例 1

$$F(A,B,C,D)=\sum m(1,2,5,7,8,10,12,13,15).$$

必要分组可得到：

- $m_1,m_5\Rightarrow \overline{A}\,\overline{C}D$；

- $m_2, m_{10} \Rightarrow \overline{B}C\overline{D}$;
- $m_8, m_{12} \Rightarrow A\overline{C}\,\overline{D}$;
- $m_5, m_7, m_{13}, m_{15} \Rightarrow BD$。

因此

$$F = \overline{A}\,\overline{C}D + \overline{B}C\overline{D} + A\overline{C}\,\overline{D} + BD.$$

例 2

$$F(W, X, Y, Z) = \sum m(3, 4, 5, 7, 9, 13, 14, 15).$$

其中 m_3 只能与 m_7 合并，m_4 只能与 m_5 合并，m_9 只能与 m_{13} 合并，m_{14} 只能与 m_{15} 合并。因此一个最小 SOP 为

$$F = \overline{W}YZ + \overline{W}X\overline{Y} + W\overline{Y}Z + WXY.$$

2.7.12　结果可能不唯一 (Results May Not Be Unique)

卡诺图化简可能存在多个同等成本的最小表达式。课件例子：

$$F(X, Y, Z) = XZ' + X'Z + YZ' + Y'Z.$$

对应最小项集合为

$$F = \sum m(1, 2, 3, 4, 5, 6).$$

由于不同主蕴含项选择可能得到相同成本，最终表达式不一定唯一；复习时应以覆盖所有 1、成本最小、无不必要重叠为判断标准。

2.7.13　德摩根与气泡推动 (DeMorgan and Bubble Pushing)

德摩根定理可用于在门图中移动反相气泡：

$$\overline{AB} = \overline{A} + \overline{B}, \qquad \overline{A + B} = \overline{A}\,\overline{B}.$$

- **向后推动 (Backward)**：门体改变类型，并在输入端添加气泡。
- **向前推动 (Forward)**：门体改变类型，并在输出端添加气泡。
- **规则**：从输出开始向输入工作；把最终输出端的气泡往回推；画图时让相邻气泡彼此抵消。

例如电路表达式 $Y = AB + CD$ 可通过气泡推动变成等价的 NAND/NOR 形式，常用于只用 NAND 门或只用 NOR 门实现电路。

2.7.14　多级优化 (Multiple-Level Optimization)

两级电路是 SOP 或 POS 加上可选输入/输出反相器。多级电路不是简单两级结构，通过因式分解可减少门输入成本。

课件优化示例：

$$\begin{aligned} G &= ABC + ABD + E + ACF + ADF && (a), \quad G = 17, \\ G &= AB(C + D) + E + AF(C + D) && (b), \quad G = 13, \\ G &= (AB + AF)(C + D) + E && (c), \quad G = 12, \\ G &= A(B + F)(C + D) + E && (d), \quad G = 9. \end{aligned}$$

核心思想是不断提取公共因子：

$$ABC + ABD + ACF + ADF = A(B + F)(C + D).$$

2.8　附加门与电路 (Additional Gates and Circuits)

2.8.1　异或与同或 (XOR and XNOR)

XOR 是数字逻辑中非常重要的函数，可直接作为电子门实现，也可由其他门互连得到。XNOR 是 XOR 的补函数。

X	Y	$X \oplus Y$	$\overline{X \oplus Y}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

表达式：

$$X \oplus Y = \overline{X}Y + X\overline{Y}, \qquad \overline{X \oplus Y} = XY + \overline{X}\,\overline{Y}.$$

XOR 表示“X or Y, but not both”。

2.8.2　XOR 的实现与恒等式

- **SOP 实现**：$X \oplus Y = \overline{X}Y + X\overline{Y}$。
- **仅用 NAND 实现**：

$$X \oplus Y = (X \text{ NAND } (X \text{ NAND } Y)) \text{ NAND } (Y \text{ NAND } (X \text{ NAND } Y)).$$

常用恒等式：

$X \oplus 0 = X$	$X \oplus 1 = \overline{X}$
$X \oplus X = 0$	$X \oplus \overline{X} = 1$
$X \oplus Y = Y \oplus X$	$(X \oplus Y) \oplus Z = X \oplus (Y \oplus Z)$

三变量 XOR 展开为

$$\begin{aligned} X \oplus Y \oplus Z &= (X\overline{Y} + \overline{X}Y) \oplus Z \\ &= (X\overline{Y} + \overline{X}Y)\overline{Z} + (XY + \overline{X}\,\overline{Y})Z \\ &= X\overline{Y}\,\overline{Z} + \overline{X}Y\overline{Z} + \overline{X}\,\overline{Y}Z + XYZ. \end{aligned}$$

它在输入中 1 的个数为奇数时输出 1，因此称为奇函数 (odd function)。其补函数为偶函数 (even function)。

2.8.3 XOR/XNOR 的应用

- 加法器、减法器、乘法器；
- 计数器、递增器、递减器；
- 奇偶校验生成器与检查器。

2.8.4 奇偶校验生成器与检查器 (Parity Generators and Checkers)

奇偶校验位是在 n 位数据后增加 1 位，形成 $n+1$ 位码字，用于检测传输过程中是否出现单比特错误。

- **偶校验 (Even parity)**：码字中 1 的总数为偶数。
- **奇校验 (Odd parity)**：码字中 1 的总数为奇数。
- **生成器 (Generator)**：根据数据位生成校验位 P。
- **检查器 (Checker)**：接收数据和校验位后生成错误指示 E。

课件例子：$n = 3$，数据 $(X, Y, Z) = (0, 0, 1)$。若生成偶校验码字，则 1 的个数目前为奇数，需要 $P = 1$，所以

$$(X, Y, Z, P) = (0, 0, 1, 1).$$

若传输过程中 Y 从 0 变为 1，则接收端 1 的个数改变，检查器输出 $E = 1$ 表示错误；$E = 0$ 表示 OK。

2.8.5 高阻输出与三态缓冲器 (Hi-Z and 3-State Buffer)

普通逻辑门输出只能为 0 或 1，且输出端不能直接连在一起，否则不同门可能同时驱动相反电平，造成冲突。

- **Hi-Z**：高阻态，也写作 Hi-z、Hiz、HiZ。它不是逻辑 0 或 1，而是近似开路、断开连接。

- **用途**：允许多个器件共享同一条总线，但同一时刻只能有一个器件被使能驱动总线。
- **三态缓冲器**：有数据输入 IN、控制输入 EN、数据输出 OUT。

EN	IN	OUT
0	X	Hi-Z
1	0	0
1	1	1

当 $EN = 0$ 时，无论 IN 是什么，输出都是 Hi-Z；当 $EN = 1$ 时，$OUT = IN$。输入或使能端也可设计为反相有效。

2.8.6　多路复用线与传输门 (Multiplexed Line and Transmission Gate)

多个三态缓冲器可组成一条多路复用线：只要保证同一时刻仅一个 EN 有效，就可以让多个数据源共享同一输出线。

传输门 (Transmission Gate) 是一种用于连接或断开电路中两点的电子开关。CMOS 传输门通常由 NMOS 与 PMOS 并联构成，互补控制信号使其在导通时能较好传递 0 和 1。课件还展示了用传输门实现 XOR 的电路结构。

2.9　全章总结 (Summary)

本讲围绕“数字逻辑如何从物理电压抽象为布尔函数，再映射成门级电路”展开。复习时可抓住以下主线：

1. **数字抽象**：连续电压通过阈值和噪声容限被解释为离散逻辑 0/1。
2. **逻辑门基础**：AND、OR、NOT 是基本运算；NAND、NOR、XOR、XNOR 是常用扩展门。
3. **器件实现**：继电器、真空管到晶体管的发展说明了逻辑开关的物理实现；CMOS 是现代数字逻辑的基础。
4. **IC 参数**：V_{CC}、逻辑电平、噪声容限、传播延迟、转换时间、功耗、扇入扇出和门成本共同影响电路可靠性与性能。
5. **布尔代数**：公理、定理、对偶、德摩根和香农展开是表达式变形的数学工具。
6. **逻辑函数表示**：真值表唯一，表达式不唯一；规范 SOP/POS 唯一，标准 SOP/POS 不唯一。
7. **逻辑化简**：代数法和卡诺图都服务于降低文字成本或门输入成本；卡诺图适合变量数较少的函数。
8. **附加电路**：XOR 支撑加法与奇偶校验；Hi-Z、三态缓冲器和传输门支撑共享总线和可控连接。

“数字系统的核心，是把物理世界中不完美的连续信号，稳定地抽象为可组合、可证明、可优化的布尔逻辑。”

2.10　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. **[Quiz]** For the original expression

$$F = A(B + \overline{C}) + \overline{A}(\overline{B} + C) + BCD + \overline{B}\,\overline{C}D,$$

calculate the gate-input cost, simplify F, and calculate the gate-input cost of the simplified expression. State clearly whether inverter inputs are included in your cost model.

2. **[Quiz]** For

$$F(A, B, C, D) = \sum m(1, 3, 7, 11, 15) + \sum d(0, 2, 5),$$

(a) draw the K-map; (b) list every prime implicant and every essential prime implicant; (c) obtain minimum SOP and POS expressions separately.

3. **[Extra]** Simplify the following using algebraic manipulation, and compute the gate-input cost before and after simplification:

$$F = D\overline{C} + C + ABC + AB\overline{C} + B\overline{C} + CD$$

4. **[Exam]** Simplify $F = A\overline{C} + ABC + AC\overline{D} + CD$. The result equals:
A. $A + CD$　B. $A\overline{C} + AC + CD$　C. $A + AC\overline{D} + CD$　D. $A\overline{D} + CD$

5. **[Exam]** Gate-cost model: each NOT counts as 1, each n-input gate counts as n. Find the total gate cost of the direct implementation of$F = AB\overline{D} + B + C(B + \overline{A}D)$.
A. 10　B. 12　C. 14　D. 16

6. **[Extra]** Find the gate-input cost G of $F = A + B\overline{C}(A\overline{B} + \overline{A}B)$.
A. 9　B. 10　C. 11　D. 12

7. **[Extra]** How many 1-cells does $AB\overline{C} + A\overline{D}$ occupy in a 4-variable Karnaugh map?
A. 13　B. 12　C. 6　D. 5

8. **[Extra]** Find the simplified complement $\overline{F}$ of:

$$F = \overline{AC + \overline{A}BC + \overline{B}\overline{C}} + AB\overline{C}$$

9. **[Extra]** Simplify using Boolean algebra (state the law used at each step):

$$F = (\overline{A}\,\overline{B} + \overline{A}B + A\overline{B})(\overline{A}C + \overline{B}C + AB)$$

10. **[Exam]** A 4-variable truth table is given below (X = don't-care).

A	B	C	D	Y	A	B	C	D	Y
0	0	0	0	0	1	0	0	0	X
0	0	0	1	1	1	0	0	1	0
0	0	1	0	X	1	0	1	0	0
0	0	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	0	0	1
0	1	0	1	X	1	1	0	1	1
0	1	1	0	0	1	1	1	0	X
0	1	1	1	0	1	1	1	1	1

(a) Write $Y = \sum m(\cdots) + \sum d(\cdots)$; (b) draw the K-map, find all PIs and EPIs, and write the minimal SOP; (c) use the K-map to find the minimal POS.

11. **[Extra]** For $F(A,B,C,D) = \sum m(0,1,2,3,6,8) + \sum d(10,11,12,13,14,15)$:(a) draw the K-map and write the product-of-maxterms form $\prod M(\cdots)$; (b) circle the essential prime implicants and list the corresponding product terms; (c) give the minimal SOP and POS.

12. **[Extra]** Use a K-map to simplify $F = \sum m(0,1,5,7,8,11,13) + \sum d(3,9,12,15)$ into the minimal **NAND-NAND** form (mark the largest groups and their product terms).

13. **[NumLogic]** How many discrete values can a binary logic variable take? A. 1 B. 2 C. 3 D. 4.

How many output states does a three-state (tri-state) buffer have? When several tri-state buffer outputs are wired together, at most how many may be enabled at once?

14. **[Class]** The truth tables of functions E, F are given below:

X	Y	Z	E	F
0	0	0	0	1
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	1	0
1	1	1	0	1

(a) list the minterms and maxterms of E and F; (b) list the minterms of $\overline{E}, \overline{F}$; (c) list the minterms of $E + F$ and $E \cdot F$; (d) write E, F in algebraic sum-of-minterms form; (e) simplify E, F to expressions with the fewest literals.

15. **[Class]** Prove the following Boolean identities algebraically (state the theorem used at each step):(a) $\overline{A}C + A\overline{B} + B\overline{C} = \overline{A}C + A\overline{B}$ (consensus);(b) $(A+B)(\overline{A}+C)(B+C) = (A+B)(\overline{A}+C)$.

16. **[Class]** For the following functions with don't-cares, find all PIs and EPIs, and write the minimal SOP using the selection rules:
(a) $F(A,B,C) = \sum m(3,5,6) + \sum d(0,7)$;
(b) $F(W,X,Y,Z) = \sum m(0,2,4,5,8,14,15) + \sum d(7,10,13)$;
(c) $F(A,B,C,D) = \sum m(4,6,7,8,12,15) + \sum d(2,3,5,10,11,14)$.

第三章　组合逻辑设计 (Combinational Logic)

本章考点

高频任务：规格到真值表/方程、译码器与 MUX 实现函数、优先编码器、多位比较器、关键路径与冒险、Verilog 建模。**复习路线**：设计题使用 specification → formulation → optimization → mapping → verification 的固定流程。

3.1　Verilog HDL 简介 (Introduction to Verilog HDL)

3.1.1　背景与 HDL 概念 (Background & What is HDL)

- **异构计算 (Heterogeneous Computing)**：现代系统常同时使用 CPU、GPU、FPGA、ASIC 等不同计算单元。CPU 灵活，GPU 擅长大规模并行，FPGA 可重构，ASIC 面向固定功能且性能/功耗最优。
- **数字电路设计**：本课程重点从逻辑功能出发，理解组合逻辑、时序逻辑以及它们如何被映射到 FPGA 或 ASIC。
- **硬件描述语言 (Hardware Description Language, HDL)**：用形式化方法描述数字电路和数字逻辑系统的语言。HDL 描述的是硬件结构与行为，不只是软件执行流程。
- **FPGA vs. ASIC**：FPGA 通过下载 bitstream 实现逻辑，开发周期短、可重配置；ASIC 最终 tape-out 生成芯片版图，前期成本高，但量产后面积、功耗和速度更优。

3.1.2　基于 HDL 的设计流程 (HDL-based Design Flow)

典型流程如下：

1. **逻辑设计 (Logic Design with HDL)**：定义模块、端口、数据类型和逻辑功能。
2. **仿真 (Simulation)**：编写 testbench，施加输入激励，观察输出是否符合预期。
3. **综合 (Synthesis)**：解析 HDL，进行多级逻辑综合，并映射到目标工艺库或 FPGA 资源。不可综合代码只能用于仿真，不能变成真实硬件。
4. **物理设计 (Physical Design)**：包括布局 (Placement)、布线 (Routing)、静态时序分析 (Static Timing Analysis)、LVS、DRC 等。
5. **最终步骤 (Final Step)**：ASIC 进入 tape-out；FPGA 下载 bitstream 文件到器件。

最小模块示例

```
module top(
    input  a,
    input  b,
    output c
);
assign c = a & b;
endmodule
```

其中 `module` 开始模块定义，端口列表给出输入输出，`assign` 是连续赋值语句，`endmodule` 结束模块。

Testbench 示例

```
module sim_top();
reg  a_in, b_in;
wire c_out;

top dut(
    .a(a_in),
    .b(b_in),
    .c(c_out)
);

initial begin
    a_in = 1'b0;
    b_in = 1'b0;
    #2 b_in = 1'b1;
    #2 a_in = 1'b1;
    #2 $finish;
end

always @(*) begin
    $display("a=%d,b=%d,c=%d", a_in, b_in, c_out);
end
endmodule
```

复习提示：testbench 中的延时、显示和激励通常用于仿真，不代表一定可综合。

3.1.3 Verilog 词法规则 (Lexical Conventions)

- Verilog 与 C 语言相似，**大小写敏感**，所有关键字均为小写。
- Verilog 程序由 token 组成，包括空白符、注释、分隔符、数字、字符串、标识符和关键字。
- **空白符**包括空格、制表符和换行。除字符串内部和分隔 token 外，空白一般被忽略。
- **注释**与 C 相同：`//` 为单行注释，`/* ... */` 为多行注释。嵌套注释是否可用取决于编译器，通常不推荐。

数字表示 (Number Specification)

Verilog 数字常量

- **有位宽数字 (Sized Numbers)**：格式为 `<size>'<base><number>`。例如 `4'b1111`、`12'habc`、`16'd255`。
- **基数 (Base)**：`d/D` 表示十进制，`b/B` 表示二进制，`o/O` 表示八进制，`h/H` 表示十六进制。
- **无位宽数字 (Unsized Numbers)**：默认十进制，默认位宽至少 32 位，具体由编译器决定。例如 `23232`、`'habc`、`'o234`。
- **x 与 z**：`x` 表示未知值 (Unknown)，`z` 表示高阻态 (High-impedance)。在十六进制中一个 `x/z` 表示 4 位，八进制表示 3 位，二进制表示 1 位。
- **位宽扩展**：若指定大小大于给出的数字，最高有效位为 `x` 则用 `x` 扩展，为 `z` 则用 `z` 扩展，否则零扩展。
- **负数**：负号放在位宽前，如 `-6'd3`；`4'd-2` 非法。负值内部通常以补码存储。
- **可读性符号**：下划线可插入数字中提高可读性，如 `12'b1111_0000_1010`，但不能作为第一个字符。数字中的 `?` 与 `z` 等价。

字符串、标识符与转义标识符

- **字符串 (Strings)**：用双引号包围，如 `"Hello world!"`、`"text\tcolumn1\n"`。
- **普通标识符 (Identifiers)**：可包含字母、数字、下划线和 `$`，应以字母或下划线开头；只有系统任务能以 `$` 开头。
- **关键字 (Keywords)**：Verilog 保留标识符，如 `reg`、`input`、`module`。
- **转义标识符 (Escaped Identifiers)**：以反斜杠开头，以空白符结束，中间可以包含任意可打印字符。例如 `\a+b-c` 的实际标识符是 `a+b-c`。

3.1.4　基本语法 (Basic Syntax)

模块、端口和实例化

```
module model_name(port_list);
    // optional port definition
    // data type definition
    // logic function description
endmodule

model_name instance_identifier(port_related_list);
```

实例化时常用命名端口连接，如 `.a(a_in)`，有利于避免端口顺序错误。

数据类型 (Data Types)

- **Net**：表示物理连接，典型类型是 `wire`。
- **Register**：表示过程赋值目标或存储类变量，典型类型是 `reg`。在组合逻辑 `always` 块中也常用 `reg` 作为过程赋值变量。
- **向量 (Vector)**：多位信号，如 `wire [7:0] a;`、`reg [31:0] rdata, wdata;`。
- **数组 (Array)**：如 `wire b[2:0]` 或 `reg [15:0] mem[1023:0]`。
- **参数 (Parameter)**：模块局部常量，常用于位宽和固定配置。

端口类型规则

- **input**：模块定义中默认为 `wire`；实例化连接端可以是 `wire` 或 `reg`。
- **output**：模块定义中可以是 `wire` 或 `reg`；实例化连接端必须是 `wire`。
- **错误示例**：`input reg in1` 不符合普通输入端口定义习惯；数组形式端口需注意维度声明位置。

运算符与优先级

课件列出的常用运算符包括：

- 一元运算符：`+`、`-`。
- 按位运算符：`~`、`&`、`|`、`^`、`^~`。
- 算术运算符：`+`、`-`、`*`、`/`、`%`。
- 归约运算符：`&`、`~&`、`|`、`| ~`、`^`、`^~`。
- 逻辑运算符：`&&`、`||`、`!`。
- 相等和关系运算符：`==`、`!=`、`<`、`<=`、`>`、`>=`。
- 拼接、移位和条件运算符：`{ }`、`>>`、`<<`、`?:`。

3.1.5　建模方法 (Modeling Methods)

- **结构化建模 (Structured Modeling)**：从模块级、门级或开关级描述硬件连接。
- **数据流建模 (Dataflow Modeling)**：适合组合逻辑，使用连续赋值语句 `assign`。
- **行为建模 (Behavioral Modeling)**：核心是 `always` 块，形式为 `always @(event signal list) procedure statement`。

两位大于比较器示例

设输入为 $A = A_1A_0$、$B = B_1B_0$，输出 `A_greater_than_B`。门级图对应的逻辑为：

$$A > B = A_1\overline{B_1} + A_1A_0\overline{B_0} + A_0\overline{B_1B_0}$$

```
// Structural model
module comparator_greater_than_structural(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    wire B0_n, B1_n, and0_out, and1_out, and2_out;

    not inv0(B0_n, B[0]), inv1(B1_n, B[1]);
    and and0(and0_out, A[1], B1_n),
        and1(and1_out, A[1], A[0], B0_n),
        and2(and2_out, A[0], B1_n, B0_n);
    or  or0(A_greater_than_B, and0_out, and1_out, and2_out);
endmodule

// Dataflow model
module comparator_greater_than_dataflow(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    wire B1_n, B0_n, and0_out, and1_out, and2_out;

    assign B1_n = ~B[1];
    assign B0_n = ~B[0];
    assign and0_out = A[1] & B1_n;
    assign and1_out = A[1] & A[0] & B0_n;
    assign and2_out = A[0] & B1_n & B0_n;
    assign A_greater_than_B = and0_out | and1_out | and2_out;
endmodule

// Conditional dataflow model
module comparator_greater_than_conditional2(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    assign A_greater_than_B = (A > B) ? 1'b1 : 1'b0;
endmodule

// Conditional model using combinations
module comparator_greater_than_conditional(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    assign A_greater_than_B =
        (A == 2'b00) ? 1'b0 :
        (A == 2'b01) ? ~(B[1] | B[0]) :
        (A == 2'b10) ? ~B[1] :
        (A == 2'b11) ? ~(B[1] & B[0]) :
        1'bx;
endmodule

// Behavioral-style concise model
module comparator_greater_than_behavioral(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
```

```
    assign A_greater_than_B = A > B;
endmodule

// Testbench for structural model
module comparator_testbench_verilog();
reg  [1:0] A, B;
wire struct_out;

comparator_greater_than_structural U1(A, B, struct_out);

initial begin
    A = 2'b10; B = 2'b00;
    #10 B = 2'b01;
    #10 B = 2'b10;
    #10 B = 2'b11;
end
endmodule
```

3.2　组合逻辑电路基础 (About Combinational Logic Circuits)

3.2.1　数字逻辑电路 (Digital Logic Circuit)

- 数字逻辑电路可以看作一个黑盒：输入是 0/1，输出也是 0/1。
- 它由互相连接的数字部件组成，包括电路元件和节点。节点可分为输入节点、内部节点和输出节点。
- 数字逻辑电路主要分为两类：组合逻辑电路 (Combinational Logic Circuit) 和时序逻辑电路 (Sequential Logic Circuit)。

3.2.2　组合逻辑电路定义

组合逻辑电路具有：

- m 个布尔输入；
- n 个布尔输出；
- n 个开关函数，每个函数把 2^m 种输入组合映射到对应输出。

组合逻辑的核心特征

当前输出只依赖当前输入值，不依赖过去输入，也不保存状态。因此组合逻辑是 **memory-less** 的。

3.2.3　组合逻辑与时序逻辑的区别

- **组合电路**：输出只取决于当前输入。
- **时序电路**：由组合逻辑和存储元件组成，输出取决于当前输入以及存储元件中保存的历史状态。

- 时序电路可理解为：组合逻辑根据当前输入和当前状态生成输出与下一状态，下一状态写入存储元件。

3.2.4　组合逻辑电路的结构约束

- 每个元件本身必须是组合逻辑电路。
- 一个节点只能作为一个元件的输出，避免多个驱动源冲突。
- 不允许出现反馈环路 (Loop)，否则可能形成状态或不稳定振荡。

3.2.5　组合电路的设计选择

- **两级逻辑 (2-level)**：通常形如与或式或或与式，结构直接，便于从最小项实现；但可能需要较大的扇入。
- **多级逻辑 (multi-level)**：通过公共因子提取降低门输入成本和面积，但可能增加路径层数。
- **权衡目标**：门延迟、扇入/扇出、面积成本和速度之间需要折中。
- **功能块 (Functional Blocks)**：常用组合功能可封装为译码器、编码器、多路选择器、加法器等模块。

3.2.6　组合逻辑设计流程 (Design Procedure)

1. **Specification**：若没有现成规格，先用文字或 HDL 写清输入、输出和功能。
2. **Formulation**：由规格导出真值表或初始布尔方程。
3. **Optimization**：进行两级或多级优化，画出逻辑图或给出 AND/OR/NOT 网表。
4. **Technology Mapping**：把逻辑图或网表映射到选定实现技术，如 NAND、NOR、AOI 门或 FPGA LUT。
5. **Verification**：验证最终设计是否满足规格。

3.3　组合逻辑设计示例 (Design Examples)

3.3.1　示例 1：三开关控制一盏灯

规格与真值表

房间中只有一盏灯，由三个开关 S_1, S_2, S_3 控制，每个开关都能独立改变灯的状态。设开关闭合为 1，打开为 0；灯亮为 1，灯灭为 0。

S_3	S_2	S_1	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

由真值表得到标准与或式：

$$F = \overline{S_3 S_2} S_1 + \overline{S_3} S_2 \overline{S_1} + S_3 \overline{S_2 S_1} + S_3 S_2 S_1$$

K 图中四个 1 彼此不能合并，因此两级与或形式已经无法进一步通过 K 图合并。观察函数可知它是三输入奇偶校验：

$$F = S_1 \oplus S_2 \oplus S_3$$

若工艺库中有 XOR 门，使用两个 XOR 门通常比标准与或式门数更少。

Verilog 描述

```
module lamp_control(s1, s2, s3, F);
    input  s1, s2, s3;
    output F;
    wire s1, s2, s3, f;

    assign F = (~s3 & ~s2 & s1) |
               (~s3 &  s2 & ~s1) |
               ( s3 & ~s2 & ~s1) |
               ( s3 &  s2 & s1);
endmodule
```

3.3.2　示例 2：BCD 到余 3 码转换器

规格

- 输入为 BCD 码 A, B, C, D，表示十进制数字 0 到 9，即 0000 到 1001。
- 输出为余 3 码 W, X, Y, Z，等于对应 BCD 数值加上二进制 0011。
- BCD 输入 1010 到 1111 不是合法十进制数字，可作为无关项 (Don't Cares)。
- 实现目标为多级电路，并映射到 NAND 等门库。

Input BCD $ABCD$	Output Excess-3 $WXYZ$
0000	0011
0001	0100
0010	0101
0011	0110
0100	0111
0101	1000
0110	1001
0111	1010
1000	1011
1001	1100

两级 K 图优化结果

$$W = A + BC + BD$$

$$X = \overline{B}C + \overline{B}D + B\overline{CD}$$

$$Y = CD + \overline{CD}$$

$$Z = \overline{D}$$

课件中对应的门输入成本为:

$$G = 7 + 10 + 6 + 0 = 23$$

多级提取优化

提取公共因子:

$$T_1 = C + D$$

得到:

$$W = A + BT_1$$

$$X = \overline{B}T_1 + B\overline{CD}$$

$$Y = CD + \overline{CD}$$

$$Z = \overline{D}$$

对应成本降低为:

$$G = 2 + 4 + 7 + 6 + 0 = 19$$

继续利用布尔变换:

$$\overline{T_1} = \overline{C + D} = \overline{C}\,\overline{D}$$

进一步写成：

$$W = A + BT_1$$

$$X = \overline{B}T_1 + B\overline{T_1}$$

$$Y = CD + \overline{T_1}$$

$$Z = \overline{D}$$

此时课件给出的成本为：

$$G = 2 + 1 + 4 + 6 + 4 + 0 = 17$$

技术映射与 Verilog

技术映射可使用包含反相器、2 输入 NAND、3 输入 NAND、2 输入 NOR、2-2 AOI 的门库。映射时核心思想是把优化后的多级表达式转换为目标门库可直接实现的结构。

```
module BCD_Excess_3(A, B, C, D, W, X, Y, Z);
    input  A, B, C, D;
    output W, X, Y, Z;
    wire A, B, C, D, W, X, Y, Z, T1;

    assign T1 = C | D;
    assign W  = A | B & C | B & D;
    assign X  = ~B & T1 | B & ~T1;
    assign Y  = C & D | ~T1;
    assign Z  = ~D;
endmodule
```

3.4　经典组合逻辑功能块 (Classic/Basic Designs)

3.4.1　功能与功能块 (Functions and Functional Blocks)

- 本节讨论的是设计中反复出现、很有用的组合逻辑功能。
- 每个常用功能通常有对应的组合电路实现，称为功能块 (Functional Block)。
- 过去这些功能块常做成 SSI、MSI、LSI 芯片；今天它们更多作为 VLSI 内部模块或 HDL 中的可综合子模块使用。

3.4.2　基本逻辑功能 (Rudimentary Logic Functions)

四类基本组合逻辑功能如下：

- **Value-Fixing**：固定输出 $F = 0$ 或 $F = 1$，不需要布尔运算符。
- **Transferring**：传输输入，$F = X$。
- **Inverting**：反相，$F = \overline{X}$，需要一个反相器。
- **Enabling**：使能传输，常见形式为 $F = X \cdot EN$ 或 $F = X + EN$。

X	$F=0$	$F=X$	$F=\overline{X}$	$F=1$
0	0	0	1	1
1	0	1	0	1

多位基本功能与总线

- 宽线表示总线 (Bus)，即一个向量信号。
- 若 $F=(F_3,F_2,F_1,F_0)$，则既可以拆成单独位，也可以取连续位段，如 $F(2:1)$，还可以取不连续位集合，如 $F(3),F(1:0)$。
- 多位传输、反相、固定值和使能可看作对每一位并行执行同一种基本功能。

3.4.3 使能功能 (Enabling Function)

- **Enabling** 允许输入信号通过到输出。
- **Disabling** 阻止输入通过，并把输出替换为固定值。
- 禁用时输出可为高阻态 Hi-Z、0 或 1。三态缓冲器和传输门常用于产生 Hi-Z。

3.4.4 译码器 (Decoder)

定义

译码 (Decoding) 是把 n 位输入码转换为 m 位输出码的过程，通常满足：

$$n \le m \le 2^n$$

每个合法输入码产生唯一输出码。执行译码的电路称为译码器。常见的 n-to-m line decoder 会产生 n 个输入变量的 2^n 个或更少最小项。

1-to-2 与 2-to-4 译码器

1-to-2 译码器：

$$D_0=\overline{A},\qquad D_1=A$$

2-to-4 译码器：

$$D_0=\overline{A_1A_0},\quad D_1=\overline{A_1}A_0,\quad D_2=A_1\overline{A_0},\quad D_3=A_1A_0$$

课件指出，2-to-4 译码器可由两个 1-to-2 译码器和四个 2 输入 AND 门组成。

3-to-8 译码器

A_2	A_1	A_0	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

3-to-8 译码器可由三个 1-to-2 译码器和八个 3 输入 AND 门组成。

译码器扩展 (Decoder Expansion)

1. 令 $k = n$。
2. 使用 2^k 个输出 AND 门，它们由两个较小译码器驱动。若 k 为偶数，两个译码器输出数均为 $2^{k/2}$；若 k 为奇数，一个输出 $2^{(k+1)/2}$，另一个输出 $2^{(k-1)/2}$。
3. 对每个得到的译码器重复上述过程，直到到达 1-to-2 译码器。

3-to-8 的例子中，最接近均分的拆法是一个 2-to-4 译码器和一个 1-to-2 译码器；2-to-4 又可拆为两个 1-to-2。

带使能译码器与解复用器

带使能信号 EN 的 2-to-4 译码器真值表如下：

EN	A_1	A_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

这里的 X 表示 0 或 1。若把 EN 的值分配到四个输出之一，该电路也可视为 **解复用器 (Demultiplexer)**。

3.4.5　用译码器和 OR 门实现组合逻辑

若要实现 m 个 n 变量函数，可以使用：

- 各输出函数的最小项和表达式；
- 一个 n-to-2^n 译码器；
- m 个 OR 门，每个输出函数一个。

两种设计方法：

1. 根据真值表，在输出为 1 的行，把对应译码器输出连接到该函数的 OR 门。
2. 直接写出每个输出函数的最小项和，然后把这些最小项 OR 起来。

二进制一位全加器示例

输入 X, Y, Z，输出和位 S 与进位 C：

$$S(X,Y,Z) = \sum m(1,2,4,7)$$

$$C(X,Y,Z) = \sum m(3,5,6,7)$$

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

用 3-to-8 译码器产生最小项 0 到 7，再把 $1, 2, 4, 7$ 接到 S 的 OR 门，把 $3, 5, 6, 7$ 接到 C 的 OR 门。

3.4.6　七段显示译码器 (Seven-Segment Display Decoder)

- 七段数码管的段通常命名为 a, b, c, d, e, f, g，可显示 0 到 9。
- **共阳极 (Common Anode Display, CAD)**：所有 LED 阳极接逻辑 1，各段阴极接低电平点亮，因此 $a \sim g = 0$ 表示亮，1 表示灭，即低有效 (Active Low)。
- **共阴极 (Common Cathode Display, CCD)**：所有 LED 阴极接地，各段阳极接高电平点亮，因此 $a \sim g = 1$ 表示亮，0 表示灭，即高有效 (Active High)。

以下表格按高有效段输出给出 BCD-to-seven-segment decoder：

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
其他输入				0	0	0	0	0	0	0

3.4.7　编码器 (Encoder)

8-to-3 编码器

8-to-3 编码器把 one-hot 输入 I_0 到 I_7 编成 3 位输出 $O_0O_1O_2$：

$$O_0 = I_4 + I_5 + I_6 + I_7$$

$$O_1 = I_2 + I_3 + I_6 + I_7$$

$$O_2 = I_1 + I_3 + I_5 + I_7$$

I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	O_0	O_1	O_2
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

普通编码器有两个问题：

1. 若多个输入同时为 1，例如 I_3 和 I_6 同时为 1，输出不再唯一。
2. 全部输入为 0 与 $I_0 = 1$ 都可能输出 000，无法区分。

优先编码器 (Priority Encoder)

4 输入优先编码器设 D_3 优先级最高，输出 A_1, A_0 表示最高优先级的 1，V 表示是否至少存在一个 1。

行覆盖最小项数	D_3	D_2	D_1	D_0	A_1	A_0	V
1	0	0	0	0	X	X	0
1	0	0	0	1	0	0	1
2	0	0	1	X	0	1	1
4	0	1	X	X	1	0	1
8	1	X	X	X	1	1	1

从表可直接读出并优化：

$$A_1 = D_3 + D_2$$

$$A_0 = D_3 + \overline{D_2} D_1$$

$$V = D_3 + D_2 + D_1 + D_0$$

3.4.8　多路选择器 (Multiplexer)

定义

多路选择器执行数据选择：从一组信息输入中，根据选择线选择一路送到输出。典型 MUX 有：

- n 个选择输入 $S_{n-1}, \ldots, S_0$；
- 2^n 个信息输入 $I_{2^n-1}, \ldots, I_0$；
- 一个输出 Y。

也可设计 $m < 2^n$ 个信息输入、n 个选择输入的 MUX。

2-to-1 MUX

当 $S = 0$ 时选择 I_0，当 $S = 1$ 时选择 I_1：

$$Y = \overline{S} I_0 + S I_1$$

其结构可分为一个 1-to-2 译码器、两个使能电路和一个 2 输入 OR 门。推广到 2^n-to-1 MUX 时，可看作 n-to-2^n 译码器加 $2^n \times 2$ AND-OR 结构。

4-to-1 MUX 与扩展

4-to-1 MUX 的选择关系为：

S_0	S_1	F
0	0	A
0	1	B
1	0	C
1	1	D

结构上可使用 2-to-4 译码器和 4×2 AND-OR。64-to-1 MUX 对应 6-to-2^6 译码器和 $2^6\times2$ AND-OR。

位宽扩展 (Multiplexer Width Expansion)

MUX 不只能选择单个 bit，也可以选择 bit 向量。方法是把多个相同选择控制的 MUX 并联，例如 4-to-1 quad multiplexer 同时选择 4 位宽数据。

其他选择实现

- 使用三态逻辑替代 AND-OR，课件给出的门输入成本为 18。
- 把译码分布到三态驱动器中，门输入成本可降至 14；课件同时比较了 AND-OR 实现的 GN 为 22、AND-OR 替换为三态驱动器后的 GN 为 18。
- 使用传输门 (Transmission Gate) 实现 MUX，课件给出的门输入成本为 8，低于三态逻辑的 14 和门级逻辑的 18 或 22。

3.4.9　用 MUX 实现组合逻辑

若要实现 m 个 n 变量函数，可使用一个 m 位宽的 2^n-to-1 MUX：

1. 写出函数真值表。
2. 按真值表输入变量顺序，把函数输入接到 MUX 选择线 $S_{n-1},\ldots,S_0$。
3. 将 MUX 输出标记为对应函数输出。
4. 按真值表把每个信息输入固定为 0 或 1；无关项可任选 0 或 1。

用 Dual 8-to-1 MUX 实现一位全加器

选择线为 X,Y,Z，两个输出分别为 S 和 C。按照全加器真值表设置两组信息输入：

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

对应设置：

k	0	1	2	3	4	5	6	7
$I_{k,0}$ (S)	0	1	1	0	1	0	0	1
$I_{k,1}$ (C)	0	0	0	1	0	1	1	1

3.4.10　半加器与全加器补充 (Half Adder & Full Adder)

- **半加器 (Half Adder)**：输入 X, Y，输出和位 $S = X \oplus Y$，进位 $C = XY$。
- **全加器 (Full Adder)**：输入 X, Y, Z，其中 Z 可理解为低位进位。输出：

$$S = X \oplus Y \oplus Z$$

$$C = XY + XZ + YZ$$

- 全加器也可由两个半加器和一个 OR 门组成。

3.5　时序分析 (Timing Analysis)

3.5.1　电路延迟 (Circuit Delay)

理想情况下，逻辑门输出会在输入变化时立即变化；实际中，反相器、传输门等器件都有延迟和上升/下降时间。因此组合逻辑虽然没有状态，但输出不会瞬时稳定。

3.5.2　传播延迟与污染延迟

- **传播延迟 (Propagation Delay, T_{pd})**：输入变化到输出变化的最大延迟。
- **污染延迟 (Contamination Delay, T_{cd})**：输入变化到输出开始可能变化的最小延迟。

T_{pd} 和 T_{cd} 不同的原因包括：

- 上升沿和下降沿延迟不同；
- 多输入、多输出路径速度不同；
- 温度影响：电路热时变慢，冷时变快。

3.5.3　关键路径与最短路径

- **关键路径 (Critical/Longest Path)**：电路中传播延迟最大的路径。

$$T_{pd,\ circuit} = \sum_{\text{critical path}} T_{pd,\ element}$$

- **最短路径 (Shortest Path)**：电路中污染延迟最小的路径。

$$T_{cd,\ circuit} = \sum_{\text{shortest path}} T_{cd,\ element}$$

课件示例中，关键路径经过两个 AND 门和一个 OR 门：

$$T_{pd} = 2T_{pd_AND} + T_{pd_OR}$$

最短路径只经过最后一级 AND：

$$T_{cd} = T_{cd_AND}$$

3.5.4　竞争冒险与毛刺 (Race Hazard & Glitch)

- **Race Hazard**：同一输入变化经不同路径到达输出，路径延迟不同，可能导致短暂错误。
- **Glitch**：单个输入变化导致输出发生多次变化，即产生短暂脉冲。

课件示例可理解为 $F = A + \overline{A}$。理想逻辑恒为 1，但由于反相路径有延迟，当 A 从 1 变 0 时，直接路径先变 0，反相信号还没变 1，输出会短暂跌为 0，形成毛刺。

3.6　逻辑电路分析 (Analysis of Logic Circuits)

3.6.1　分析步骤

1. **写出电路布尔函数**：从输入信号开始，逐个门定义中间信号关系，并做必要优化。
2. **导出真值表**：明确输入和输出之间的关系。
3. **功能分析**：定义每个信号和整个电路的功能，必要时画时序图。
4. **验证**：检查最终分析或设计是否正确。

3.6.2　示例：分析一个组合逻辑电路

课件中电路的中间信号为：

$$P_1 = A \oplus B$$

$$P_2 = B \oplus C$$

$$P_3 = \overline{\overline{A} + \overline{B}} = AB$$

$$P_4 = \overline{A + C}$$

$$P_5 = \overline{P_1 P_2} = \overline{(A \oplus B)(B \oplus C)}$$

$$P_6 = P_3 + P_4 = AB + \overline{A + C}$$

最终输出为：

$$F = \overline{P_5 P_6} = \overline{\overline{(A \oplus B)(B \oplus C)} \cdot (AB + \overline{A + C})}$$

布尔函数化简

由德摩根律：

$$F = (A \oplus B)(B \oplus C) + \overline{AB + \overline{A + C}}$$

继续展开并化简：

$$\begin{aligned} F &= (A\overline{B} + \overline{A}B)(B\overline{C} + \overline{B}C) + (\overline{A} + \overline{B})(A + C) \\ &= A\overline{B}C + \overline{A}B\overline{C} + \overline{A}C + A\overline{B} + \overline{B}C \\ &= \overline{A}B\overline{C} + \overline{A}C + A\overline{B} + \overline{B}C \\ &= \overline{A}C + A\overline{B} + \overline{A}B\overline{C} \\ &= \overline{A}(C + B\overline{C}) + A\overline{B} \\ &= \overline{A}C + \overline{A}B + A\overline{B} \\ &= \overline{A}C + (A \oplus B) \end{aligned}$$

A	B	C	$A \oplus B$	$\overline{A}C$	F
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	1	0	1
1	0	1	1	0	1
1	1	0	0	0	0
1	1	1	0	0	0

从真值表可读出：

$$F = 1 \Longleftrightarrow A \neq B \text{ 或 } A < C$$

因此原电路不是最优实现。更简洁的实现是一个 XOR 门产生 $A \oplus B$，一个反相器和 AND 门产生 $\overline{A}C$，最后用 OR 门合并。

3.7　复习小结 (Summary)

- Verilog HDL 用模块、端口、数据类型和赋值/过程语句描述硬件；要区分可综合逻辑和仅用于仿真的 testbench 代码。

- 组合逻辑是无记忆电路，输出只依赖当前输入；设计流程包括规格、公式化、优化、技术映射和验证。
- 常用组合功能块包括基本逻辑功能、使能、译码器、编码器、优先编码器、多路选择器、解复用器、七段显示译码器、半加器和全加器。
- 译码器适合从最小项实现函数；MUX 适合直接按真值表固定信息输入实现函数。
- 实际电路存在传播延迟和污染延迟，关键路径决定最大延迟，最短路径影响污染延迟；路径延迟差会导致竞争冒险和毛刺。
- 分析已有电路时，应先逐级列中间信号，再化简、列真值表、解释功能并验证是否可优化。

3.8　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. **[Exam]** Express the following using the Verilog conditional operator (`?:`); choose the equivalent form:

$$O = \overline{S_0}\,\overline{S_1} I_3 + \overline{S_0} S_1 I_2 + S_0 \overline{S_1} I_1 + S_0 S_1 I_0$$

A. `O = S0 ? (S1 ? I0 : I1) : (S1 ? I2 : I3);`
B. `O = S1 ? (S0 ? I0 : I2) : (S0 ? I1 : I3);`
C. `O = S0 & S1 ? I0 : I3;`
D. `O = S0 ? I0 : I1;`

2. **[Exam/Extra]** Using one 4-to-1 MUX (A, B on the select lines) plus a few external gates, implement

$$F(A,B,C,D) = \sum m(1,3,4,11,12,13,14,15)$$

(Hint: for each case $AB = 00, 01, 10, 11$, find F as a function of C, D.)

3. **[Exam]** Using only **two 3-to-8 decoders with enable**, **one inverter**, and **OR gates with at most 4 inputs**, implement$F(A,B,C,D) = \sum m(1,3,4,11,12,13,14,15)$. Draw the complete logic diagram.

4. **[Extra]** (a) Design a 4-to-16 decoder using two 3-to-8 decoders and sixteen 2-input AND gates; (b) design a 4-to-16 decoder with enable using five 2-to-4 decoders with enable.

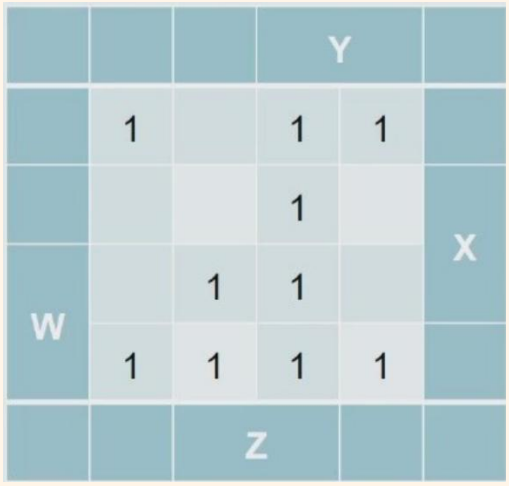
5. **[Extra]** Design a combinational circuit that compares two 4-bit unsigned numbers A, B

and outputs X: $X = 1$ when $A < B$, otherwise $X = 0$.

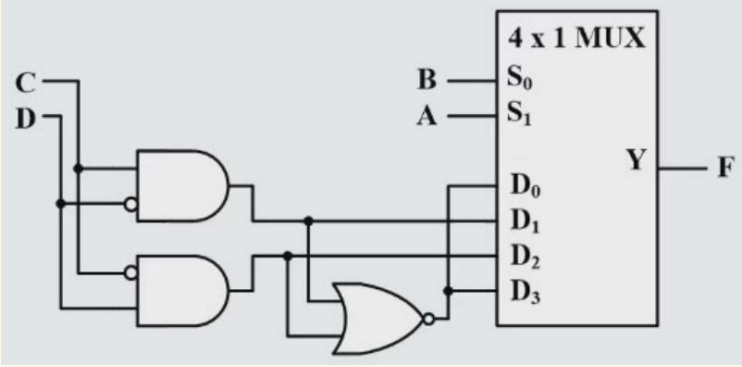
6. **[Extra]** 1-bit unsigned comparator: inputs A, B, outputs $X(A{>}B), Y(A{<}B), Z(A{=}B)$. (1) Build the truth table; (2) implement it using **one 2-to-4 decoder** plus gates; (3) using this 1-bit comparator as a module, design a 2-bit unsigned comparator from **two** such modules plus gates ($A[1{:}0], B[1{:}0]$, outputs $A{>}B, A{<}B, A{=}B$).

7. **[Extra]** Implement a 1-bit full adder $S = \sum m(1, 2, 4, 7)$, $C = \sum m(3, 5, 6, 7)$ using a 3-to-8 decoder plus OR gates.

8. **[Recall]** In the Karnaugh map shown below, how many prime implicants does the function have? A. 4　B. 5　C. 6　D. 7



9. **[Extra]** Analyze the logic circuit below (A, B on select lines S_1, S_0; C, D feed the data inputs $D_0 \sim D_3$ through external gates; output F): build the truth table of F, write the logic expression of F, and state what function this circuit can at most realize.



10. **[NumLogic]** Design a three-input majority voter with one 3-to-8 decoder and necessary gates. For inputs A, B, C, the output $F = 1$ exactly when at least two inputs are 1. Give the truth table, canonical Boolean function, and complete logic diagram.

第四章　算术运算单元 (Arithmetic Unit)

本章考点

高频任务：半/全加器、RCA/CLA 进位与延迟、统一加减法、进位与溢出、ALU/移位器、Booth 乘法、恢复/不恢复除法、浮点运算步骤。**复习路线**：每道题先区分 signed 与 unsigned；进位 CF 不等于有符号溢出 OF。

4.1　运算单元总览与迭代组合电路 (Overview & Iterative Combinational Circuits)

4.1.1　本讲主线 (Lecture Roadmap)

本讲围绕处理器中负责计算的硬件展开，课件内容覆盖：

- **加法 (Addition)**：半加器、全加器、多位加法器、进位传播与快速进位。
- **减法 (Subtraction)**：无符号减法、补码减法、减法器与借位。
- **算术逻辑单元 (Arithmetic Logic Unit, ALU)**：将算术、逻辑、移位、比较等操作统一到一个可控数据通路中。
- **乘法 (Multiplication)**：移位加法、逐次乘法器、Booth 编码、阵列乘法器。
- **除法 (Division)**：恢复除法、非恢复除法、有符号除法。
- **浮点运算 (Floating-Point Operations)**：浮点加法、乘法、除法与硬件复杂度。

4.1.2　迭代组合电路 (Iterative Combinational Circuits)

算术函数通常作用在二进制向量上，并且每一位位置经常重复相同的子功能。因此可以先设计一个处理单个位或少数位的**单元 (Cell)**，再将多个单元互连形成完整的**迭代阵列 (Iterative Array)**。

- **Cell**：完成每一位的局部逻辑，如 1 位全加器。
- **Iterative Array**：由多个 cell 重复连接而成，可以是一维阵列，也可以是二维阵列。
- **典型例子**：行波进位加法器是一维阵列；阵列乘法器是二维阵列。

复习抓手

本讲的核心问题不是“如何手算加减乘除”，而是“硬件如何用有限的门、寄存器、移位器和控制信号完成这些运算”。复习时要同时关注：功能正确性、标志位含义、延迟、硬件面积和可流水化程度。

4.2　加法器 (Adders)

4.2.1　半加器 (Half-Adder)

半加器是 2 输入、1 位宽的二进制加法器，用于计算 $A+B$，输出一个和位 F 与一个进位位 C。

A	B	和位 F	进位 C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

逻辑表达式为：

$$F = A \oplus B, \qquad C = AB$$

4.2.2　全加器 (Full Adder)

全加器在半加器基础上加入来自低位的进位输入 C_{in}，输出和位 F 与进位输出 C_{out}。

A	B	C_{in}	F	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

常用表达式：

$$F = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

4.2.3　多位进位传播加法器 (Carry Propagate Adders, CPA)

多位加法器由多个 1 位加法单元连接构成。不同结构的核心差异在于如何处理进位。

类型	基本思想	特点
Ripple-Carry Adder (RCA)	将全加器串联，低位进位逐级传到高位。	硬件简单，但慢。
Carry Skip Adder	当一个块整体能传播进位时，让进位跳过该块内部。	比 RCA 快，块长可不均匀优化。
Carry Select Adder (CSA)	对每个块同时假设输入进位为 0 和 1，最后用选择器选结果。	速度快，硬件冗余。
Carry-Lookahead Adder (CLA)	用 generate/propagate 方程提前算出进位。	快，门扇入和硬件复杂度上升。
Prefix Adder	通过前缀网络并行计算每一列的进位。	很快，约 $\log_2 N$ 级。

4.2.4　行波进位加法器 (Ripple-Carry Adder, RCA)

RCA 将 N 个 1 位全加器串起来，每一级必须等待低位进位到达后才能确定结果。

- **优点**：结构最简单，面积小。
- **缺点**：最坏路径是进位穿过整个链，因此速度慢。
- **延迟公式**：

$$t_{ripple} = Nt_{FA}$$

其中 t_{FA} 是 1 位全加器的延迟。

4.2.5　超前进位加法器 (Carry Lookahead Adder, CLA)

CLA 的关键是将每一位的进位行为拆成**生成 (Generate)** 与**传递 (Propagate)**。

$$P_i = A_i \oplus B_i, \qquad G_i = A_i B_i$$

$$S_i = P_i \oplus C_i, \qquad C_{i+1} = G_i + P_i C_i$$

- 若 $A_i = B_i = 1$，则无论 C_i 是多少，高位进位都会被生成，即 $G_i = 1$。
- 若 $A_i \oplus B_i = 1$ 且 $C_i = 1$，则输入进位会被传播到高位，即 $P_i C_i = 1$。

从 C_0 开始展开：

$$\begin{aligned}
C_1 &= G_0 + P_0 C_0 \\
C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0 \\
C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\
C_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0
\end{aligned}$$

4.2.6　组超前进位 (Group Carry Lookahead)

直接展开到很多位会遇到门扇入限制，因此课件引入组生成与组传递。对 4 位块 $0 \sim 3$：

$$G_{0\sim3} = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

$$P_{0\sim3} = P_3P_2P_1P_0$$

于是：

$$C_4 = G_{0\sim3} + P_{0\sim3}C_0$$

16 位 CLA 可由四个 4 位 CLA 块组成：

$$C_4 = G_{0\sim3} + P_{0\sim3}C_0$$
$$C_8 = G_{4\sim7} + P_{4\sim7}C_4$$
$$C_{12} = G_{8\sim11} + P_{8\sim11}C_8$$
$$C_{16} = G_{12\sim15} + P_{12\sim15}C_{12}$$

课件给出的 16 位 CLA 延迟例子中，假设 `NOT`=1，`XOR`=3，孤立 `AND`=3，`AND-OR`=2：

- RCA 最长延迟：$3 + 15 \times 2 + 3 = 36$。
- CLA 最长延迟：$3 + 3 \times 2 + 3 = 12$。

4.2.7　前缀加法器 (Prefix Adder)

前缀加法器先计算每一列的进位，再计算和位：

$$S_i = (A_i \oplus B_i) \oplus C_{i-1}$$

课件将第 -1 列定义为输入进位：

$$G_{-1} = C_{in}, \qquad P_{-1} = 0$$

列 i 的输入进位等于列 $i-1$ 到 -1 的组生成：

$$C_{i-1} = G_{i-1:-1}$$

对跨越 $i:j$ 的块，若以 k 为分界：

$$G_{i:j} = G_{i:k} + P_{i:k}G_{k-1:j}$$

$$P_{i:j} = P_{i:k}P_{k-1:j}$$

前缀加法器延迟近似为：

$$t_{PA} = t_{pg} + \log_2 N \cdot t_{pg_prefix} + t_{XOR}$$

4.2.8　Carry Skip 与 Carry Select

- **Carry Skip Adder**：若某一块所有位都能传播进位，则进位无需在块内部逐级传播，可直接跳到下一块。为了最优速度，块长通常不必相等。
- **Carry Select Adder**：将每一块在 $C_{in}=0$ 和 $C_{in}=1$ 两种情况下都预先算好，再根据真实进位选择结果。课件中特别提到可以按 nibble（4 位块）做选择。

4.2.9　32 位 CLA 与延迟比较

补充页中给出 32 位 CLA 可由多个 4 位块构成。其延迟模型为：

$$t_{CLA}=t_{pg}+t_{pg_block}+\left(\frac{N}{k}-1\right)t_{AND_OR}+kt_{FA}$$

- t_{pg}：生成所有 P_i,G_i 的延迟。
- t_{pg_block}：生成块级 $P_{i:j},G_{i:j}$ 的延迟。
- t_{AND_OR}：最终 AND/OR 门中从 C_{in} 到 C_{out} 的延迟。
- k：每个 CLA 块的位数。

课件总结：若只按门级数估计，RCA 与 CLA 的 C_i、S_i 门级可能分别是 RCA 的 $(2,1)$ 与 CLA 的 $(3,4)$；但真实延迟还取决于门类型与负载。一般当 $N>16$ 时，CLA 明显快于 RCA。

4.3　减法、溢出与标志位 (Subtraction, Overflow & Flags)

4.3.1　无符号补码减法 (Unsigned 2's Complement Subtraction)

对 n 位无符号数 $M-N$，可将减法转化为加法：

$$M-N=M+(2^n-N)-2^n$$

也就是给被减数 M 加上减数 N 的二进制补码。

- 若 $M>N$，加法产生最高位进位 $r_n=1$，丢弃该进位后得到正结果 $M-N$。
- 若 $M<N$，没有最高位进位，得到的是 $N-M$ 的补码；若要写成普通带负号形式，需要对结果再取补码并在前面加负号。

例 1：$01010100_2-01000011_2$

$$01010100+10111101=1\,00010001$$

最高位进位为 1，丢弃进位，结果为：

$$00010001_2=17_{10}$$

例 2：$01000011_2 - 01010100_2$

$$01000011 + 10101100 = 0\,11101111$$

最高位进位为 0，需要修正。对 11101111 取补码得 00010001，因此结果为：

$$-00010001_2 = -17_{10}$$

4.3.2　统一加减法硬件 (Adder-Subtractor)

补码表示使加法和减法可以使用同一套加法器：

- 当控制位 $S = 0$：执行 $A + B$。
- 当控制位 $S = 1$：将 B 各位取反，并令最低位进位 $C_0 = 1$，执行

$$A + \bar{B} + 1 = A - B$$

- 该电路既可用于无符号数加减，也可用于带符号补码数加减；区别在于如何解释结果与标志位。

4.3.3　进位与溢出 (Carry & Overflow)

- **Carry** 主要用于无符号整数，表示无符号结果是否超出 n 位范围。
- **Overflow** 主要用于带符号补码整数，表示带符号结果是否超出可表示范围。

以 4 位补码为例，范围为 $[-8, +7]$。计算 $4 + 5$ 得到位模式 1001，若按补码解释为 -7，发生正溢出；计算 $(-4) + (-5)$ 得到位模式 0111，若按补码解释为 $+7$，发生负溢出。

4.3.4　带符号溢出检测 (Detecting Signed Overflow)

补码加法中，溢出只会发生在以下两种情况：

- 两个正数相加，结果为负。
- 两个负数相加，结果为正。

等价判定：

- 若两个操作数符号位相同，而结果符号位不同，则发生溢出。
- 或者比较符号位的进位输入与进位输出：

$$OF = V = C_n \oplus C_{n-1}$$

Carry 和 Overflow 不可混用

无符号加法看 **Carry Flag**，带符号补码加法看 **Overflow Flag**。课件中特别强调：带符号运算不能只用最高位进位 C_n 判断溢出。

4.3.5　重要标志位 (Important Flags)

标志位	含义与适用场景
ZF (Zero Flag)	当结果为 0 时置 1；无符号与带符号运算都适用。
SF/NF (Sign Flag)	结果的最高位 S_{n-1}；主要用于带符号运算。
CF (Carry/Borrow Flag)	加法中通常表示 C_{out}；减法中常表示借位，课件写作 $\sim C_{out}$；主要用于无符号运算。
OF (Overflow Flag)	表示带符号补码运算溢出；主要用于带符号运算。

4.3.6　半减器与全减器 (Half/Full Subtractor)

补充页给出直接构造减法器的方法。半减器计算 $X - Y$：

X	Y	差 D	借位 B_{out}
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

半减器公式：

$$D = X \oplus Y, \qquad B_{out} = \bar{X}Y$$

全减器计算 $X - Y - B_{in}$：

$$D = X \oplus Y \oplus B_{in}$$

$$B_{out} = \bar{X}Y + \bar{X}B_{in} + YB_{in}$$

n 位减法器可以用两种方式实现：

1. 串接 n 个全减器，形成 borrow ripple subtractor。
2. 使用 n 位加法器和 n 个反相器，将 Y 取反并令最低位进位为 1，从而计算 $X + \bar{Y} + 1$。

4.4　算术逻辑单元与移位器 (ALU & Shifters)

4.4.1　什么是 ALU?

ALU (Arithmetic-Logic Unit) 执行多种算术与逻辑操作，是处理器的核心计算部件。课件中称它为处理器的“heart”：CPU 的其他部分很大程度上是在为 ALU 提供操作数、控制信号和结果存储。

构造 ALU 的两类思路：

- **扩展加法器 (extend the adder)**：在加法器周围增加反相、逻辑门、选择器和标志位逻辑。
- **并行冗余选择 (parallel redundant select)**：多个功能并行计算，最终由多路选择器选择输出。

4.4.2　简单 1 位 ALU

课件给出一个支持 3 个操作的简单 ALU：

功能码 $F_{1:0}$	功能
00	$A\&B$
01	$A\|B$
10	$A+B$
11	未使用

在多位版本中，可同时输出 **OF**、**SF**、**ZF**、**CF** 等标志位。

4.4.3　N 位 8 操作 ALU

课件给出的 N 位 ALU 使用 3 位控制线选择 8 类操作：

ALU Control Lines	Function
000	And
001	Or
010	Add
110	Sub
111	SLT (Set Less Than)
100	not
101	srl (shift right logical)
011	xor

硬件结构上，逻辑门、加法/减法器、移位器、SLT 结果等输出会进入 MUX，由控制线选择最终结果，同时生成 Zero 与 Overflow 等信号。

4.4.4　SLT 指令示例 (Set Less Than)

课件示例：配置 32 位 ALU 执行 SLT，$A = 25$，$B = 32$。

- 因为 $A < B$，所以输出应为 32 位的 1，即 `0x00000001`。
- 对应 MIPS 风格语义：`slt rd, rs, rt`，若 $rs < rt$ 则 $rd = 1$，否则 $rd = 0$。
- ALU 内部可通过执行 $A - B$ 判断：$25 - 32 = -7$，结果符号位为 1，说明 $A < B$。
- 输出寄存器只有最低位为 1，其余位为 0。

补充注意：对带符号 SLT，若减法发生溢出，不能只看符号位，稳妥的比较位可写为：

$$set = S_{31} \oplus OF$$

4.4.5　移位器的用途 (Various Use of Shifters)

课件区分三类移位/旋转操作：

- **逻辑移位 (Logical Shifter)**：左移或右移后空位补 0。
  - `11001 >> 2 = 00110`
  - `11001 << 2 = 00100`
- **算术移位 (Arithmetic Shifter)**：左移与逻辑移位相同；右移时空位补旧最高有效位 MSB，用于保持补码符号。
  - `11001 >>> 2 = 11110`
  - `11001 <<< 2 = 00100`
- **循环移位 (Rotator)**：从一端移出的位从另一端补入。
  - `11001 ROR 2 = 01110`
  - `11001 ROL 2 = 00111`

4.4.6　移位作为乘除法 (Shifters as Multipliers and Dividers)

对二进制整数：

$$A << N = A \times 2^N$$

$$A >>> N \approx A \div 2^N$$

课件示例：

- `00001 << 2 = 00100`，即 $1 \times 2^2 = 4$。
- `11101 << 2 = 10100`，即 $-3 \times 2^2 = -12$。
- `01000 >>> 2 = 00010`，即 $8 \div 2^2 = 2$。
- `10000 >>> 2 = 11100`，即 $-16 \div 2^2 = -4$。

4.4.7　组合移位器与桶形移位器 (Combinational and Barrel Shifters)

传统双向移位寄存器带并行装载时，一个寄存器传输和移位操作可能需要多个时钟：

1. 从总线并行装载到移位寄存器。
2. 执行左移、右移或保持。
3. 将结果传送到目标寄存器。

若将移位器实现为组合电路，则信号直接通过门和 MUX 传播，不需要额外移位时钟，数据通路中的一次寄存器传输可更快完成。

4 位循环左移 (ROL) 桶形移位器的功能表：

S_1	S_0	$Y_3Y_2Y_1Y_0$	**操作**
0	0	$D_3D_2D_1D_0$	不旋转
0	1	$D_2D_1D_0D_3$	左旋 1 位
1	0	$D_1D_0D_3D_2$	左旋 2 位
1	1	$D_0D_3D_2D_1$	左旋 3 位

4.4.8　数据通路中的 ALU (Datapath and ALU)

CPU 可以分成两部分：

- **数据通路 (Datapath)**：寄存器、共享 ALU、移位器和互连总线。
- **控制单元 (Control Unit)**：产生选择信号，控制数据在总线、ALU、移位器和寄存器之间流动。

当 ALU 是组合电路时，从源寄存器经 ALU 到目标寄存器的寄存器传送操作可在一个时钟周期内完成。移位操作可放在独立单元中，也可整合到 ALU 内。

4.5　乘法器 (Multipliers)

4.5.1　二进制乘法基础

乘法比加法复杂，因为直接实现通常包含多次移位和加法。复杂运算会带来两类代价：

- 更大的硅片面积 (area)。
- 更长的运行时间，例如多周期完成，或拉长单个时钟周期。

二进制乘法与十进制竖式乘法类似：

- 用乘数的每一位与被乘数相乘，形成部分积。
- 将部分积按位权左移后求和，得到最终乘积。

若

$$A = \sum_{i=0}^{m-1} a_i 2^i, \qquad B = \sum_{j=0}^{n-1} b_j 2^j$$

则

$$A \cdot B = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} a_i b_j 2^{i+j}$$

无符号 m 位乘 n 位的结果最多需要 $m+n$ 位。

4.5.2　直接竖式算法 (Straightforward Algorithm)

课件例子：

$$
\begin{array}{rl}
\texttt{01010010} & \text{multiplicand} \\
\times\ \texttt{01101101} & \text{multiplier} \\
\hline
\texttt{0100010111101010} & \text{full sum}
\end{array}
$$

含义是：从乘数最低位开始扫描，若当前位为 1，则把被乘数按当前位置左移后加入部分和；若当前位为 0，则加入 0。课件后面的逐页动画展示了 partial sum 如何随着每个乘数位逐步增加。

4.5.3　逐次乘法硬件实现 1 (Implementation 1)

第一种实现保存三个寄存器：

- **Multiplicand**：被乘数寄存器，逐轮左移。
- **Multiplier**：乘数寄存器，逐轮右移。
- **Product**：部分积寄存器。

每一轮：

1. 测试乘数最低位 `Multiplier0`。
2. 若为 1，将 multiplicand 加到 product。
3. multiplicand 左移 1 位。
4. multiplier 右移 1 位。
5. 重复到指定位数结束。

例：$0010_2 \times 0110_2 = 2 \times 6 = 12$，最终乘积为 `00001100`。
缺点：

- ALU 宽度是必要宽度的两倍。
- multiplicand 寄存器也需要两倍位宽。
- product 寄存器在最后一步前并不需要完整 $2n$ 位。
- multiplier 寄存器在过程中逐渐被清空。

4.5.4　逐次乘法硬件实现 2 与 3

Implementation 2 的思想是：

- 被乘数固定 (Multiplicand stationary)。
- 乘数右移 (Multiplier right)。
- 部分积右移 (Partial Product right)。
- 只需将 multiplicand 加到 product 的左半部分。

Implementation 3 进一步把乘数放在部分积寄存器的右半边：

- 初始 `Product|Multiplier` 为“高半部分 0 + 低半部分乘数”。

- 每轮测试最低位，也就是当前 `Product0`。
- 若为 1，将 multiplicand 加到 product 左半部分。
- 整个 `Product|Multiplier` 右移 1 位。

  对 $0010_2 \times 0110_2$，Implementation 3 最终也得到 `00001100`，但寄存器利用率更高。

4.5.5　有符号乘法与 Booth 算法

基本有符号乘法可以这样做：

1. 保存两个操作数的符号。
2. 将它们转成无符号绝对值。
3. 执行无符号乘法。
4. 若两个符号相同，结果为正；若符号不同，结果为负。

更好的方法是 **Booth's Algorithm**，前提是硬件支持加法和减法。它的思想是：当乘数中出现连续的 1 时，不必对每个 1 都加一次，而是在连续 1 的开始做一次减法，在连续 1 的结束做一次加法，中间只移位。

当前位	右侧上一位	含义	操作
1	0	一串 1 的开始	Subtract
1	1	一串 1 的中间	Nothing
0	1	一串 1 的结束	Add
0	0	一串 0 的中间	Nothing

1 位 Booth 编码：

Y_j	Y_{j-1}	**1-bit Encoding**
0	0	$+0$
0	1	$+1$
1	0	-1
1	1	-0

2 位 Booth 编码：

Y_{j+1}	Y_j	Y_{j-1}	**2-bit Encoding**
0	0	0	$+0$
0	0	1	$+1$
0	1	0	$+1$
0	1	1	$+2$
1	0	0	-2
1	0	1	-1
1	1	0	-1
1	1	1	-0

2 bits/cycle Booth 乘法器每轮处理两个乘数位：

- 编码为 -2：被乘数左移 1 位后相减。
- 编码为 -1：相减。
- 编码为 0：不操作。
- 编码为 $+1$：相加。
- 编码为 $+2$：被乘数左移 1 位后相加。

4.5.6　Booth 算法为什么成立?

若乘数为 $a_{31}a_{30}\dots a_0$，普通表示为：

$$b(a_{31}a_{30}\dots a_0) = a_0 b2^0 + a_1 b2^1 + \cdots + a_{31} b2^{31}$$

Booth 重写为相邻位差：

$$= (0 - a_0)b2^0 + (a_0 - a_1)b2^1 + \cdots + (a_{30} - a_{31})b2^{31} + a_{31}b2^{32}$$

当乘数中有长串连续的 1 时，这种表示会把很多个加法压缩成“开头一次减法 + 结尾一次加法”，从而减少部分积数量。对负乘数也成立，因为补码的最高位本来就带有负权重。

4.5.7　Booth 示例与易错点

课件示例：

- $0010_2 \times 1101_2 = 2 \times (-3)$，Booth 逐轮得到结果 `11111010`，即 -6。
- 负被乘数例：$1010_2 \times 0110_2 = (-6) \times 6 = -36$，结果 `11011100`。
- 负乘数例：$1010_2 \times 1110_2 = (-6) \times (-2) = 12$，结果 `00001100`。

课件还提醒有符号部分积必须做**符号扩展**；如果乘数最高位为 1，最高位对应的部分积需要做减法而不是普通加法。

4.5.8　阵列乘法器与更快乘法 (Array Multiplier & Faster Multiplication)

阵列乘法器用基本 cell 同时产生并累加部分积。每个 cell 可看作由 A_iB_j、输入和 S_{in}、输入进位 C_{in} 进入全加器，输出 S_{out} 与 C_{out}。

- **优点**：概念直观，所有部分积并行推进。
- **缺点**：硬件成本较高。
- **加速方式**：使用多级加法器树，或流水线化，使多个乘法可并行处在不同阶段。
- **工程注意**：整数乘法相对昂贵，在数组地址计算中常可用移位替代乘以 2^k。

4.6　除法器 (Dividers)

4.6.1　无符号除法与恢复除法 (Unsigned and Restoring Division)

二进制除法类似竖式除法。硬件需要判断“当前余数是否足够减去除数”:

$$\text{condition: remainder} \geq \text{divisor}$$

硬件通常通过减法判断:

$$\text{remainder} - \text{divisor} \geq 0$$

若减得动，则保留新余数并写入商位 1；若减不动，则必须把余数恢复到减法前的值，并写入商位 0。这就是**恢复除法 (Restoring Division)**。

4.6.2　恢复除法实现 1

Implementation 1 的流程:

1. 将 dividend 放入 remainder 寄存器低半部分。
2. 用 divisor 寄存器与 remainder 寄存器做减法。
3. 若 remainder ≥ 0，商寄存器左移并写入 1。
4. 若 remainder < 0，加回 divisor 恢复余数，商寄存器左移并写入 0。
5. divisor 右移 1 位。
6. 对 32 位除法重复到结束。

课件例子 $7 \div 2$，最终:

$$Q = 3, \qquad R = 1$$

4.6.3　恢复除法实现 2 与改进

Implementation 2 的改进:

- 跳过第一次无意义的 subtract，因为一开始不可能向商中移入 1。
- 先将 remainder 左移，再做减法。

- quotient 可存放在 remainder 寄存器低半部分。
- 只需 32 位 ALU。
- 结束时将 remainder 左半部分右移 1 位，撤销多余移位。

这种结构与逐次乘法器更相似，但恢复除法仍可能每一位需要两次 ALU 操作：一次试减，一次在失败时恢复。

4.6.4　非恢复除法 (Non-Restoring Division)

恢复除法的问题是失败时需要“加回去”。非恢复除法避免了这一步。

若当前余数试减后为负：

$$R_i = 2R_{i-1} - d < 0$$

下一步若不恢复，而是改做加法，可得到同样效果：

$$R_{i+1} = 2R_i + d$$

直观理解：如果上一步多减了一个 d，下一步把余数左移后会放大这个误差，因此通过“加 d”修正，而不是先恢复再减。

- 余数非负：下一步做减法，商位写 1。
- 余数为负：下一步做加法，商位写 0。
- 吞吐率可达到每周期产生 1 个商位。

4.6.5　有符号除法 (Signed Division)

最简单思路是先记录 dividend 与 divisor 的符号，对绝对值做无符号除法，再修正符号：

- 若两个操作数符号相反，则商为负。
- 若两个操作数符号相同，则商为正。
- 非零余数的符号应与 dividend 相同。

按“向 0 截断”的整数除法习惯：

- $(-7) \div 2 = -3$，余数 -1。
- $7 \div (-2) = -3$，余数 1。
- $(-7) \div (-2) = 3$，余数 -1。

4.6.6　4 x 4 阵列除法器

补充页给出 4 位除法器的抽象算法。对 A/B：

$$A/B = Q + R/B$$

算法：

1. $R' = 0$。
2. 对 $i = N-1$ 到 0：

$$R = \{R' << 1,\ A_i\}$$

$$D = R - B$$

3. 若 $D < 0$，则 $Q_i = 0,\ R' = R$。
4. 否则 $Q_i = 1,\ R' = D$。
5. 结束后得到商 Q 与余数 R。

4.7　浮点运算单元 (Floating-Point Operations)

4.7.1　浮点加法步骤 (Floating-Point Addition)

浮点加法的核心是：**先对阶，再加尾数，再规格化，再舍入**。

1. **Alignment**：比较指数，将指数较小的数的尾数右移，直到两个指数相同。
2. **Add Significands**：对齐后加/减尾数。
3. **Normalize**：若结果不符合规格化形式，移动尾数并调整指数。
4. **Over/Underflow Check**：检查指数是否溢出或下溢。
5. **Rounding**：按浮点舍入规则保留有限位数。
6. **Renormalization**：若舍入导致进位，可能需要再次规格化。

十进制示例

系统精度为 4 位，计算：

$$9.999 \times 10^1 + 1.610 \times 10^{-1}$$

对阶：

$$1.610 \times 10^{-1} = 0.01610 \times 10^1 \rightarrow 0.016 \times 10^1$$

加尾数：

$$9.999 \times 10^1 + 0.016 \times 10^1 = 10.015 \times 10^1$$

规格化：

$$10.015 \times 10^1 = 1.0015 \times 10^2$$

舍入：

$$1.0015 \times 10^2 \rightarrow 1.002 \times 10^2$$

二进制示例：$0.5 + (-0.4375)$

$$0.5_{10} = 1.000_2 \times 2^{-1}$$

$$-0.4375_{10} = -1.110_2 \times 2^{-2}$$

对阶：

$$-1.110_2 \times 2^{-2} \rightarrow -0.111_2 \times 2^{-1}$$

加尾数：

$$1.000_2 \times 2^{-1} + (-0.111_2 \times 2^{-1}) = 0.001_2 \times 2^{-1}$$

规格化：

$$0.001_2 \times 2^{-1} = 1.000_2 \times 2^{-4}$$

结果：

$$1.000_2 \times 2^{-4} = 0.0625_{10}$$

4.7.2　浮点加法硬件

FP Adder 的硬件步骤与算法一致：

- 比较指数，计算指数差。
- 右移较小数的尾数。
- 用大 ALU 对尾数做加/减。
- 规格化，必要时调整指数。
- 进入 rounding hardware。

课件强调：浮点加法器远比整数加法器复杂。若强行一个时钟周期完成，会显著拉长时钟周期，拖慢所有指令。因此 FP adder 通常多周期完成，并且常采用流水线。

4.7.3　浮点乘法步骤 (Floating-Point Multiplication)

浮点数可拆成符号、尾数、指数分别处理：

$$(s_1 \cdot 2^{e_1})(s_2 \cdot 2^{e_2}) = (s_1 s_2) \cdot 2^{e_1+e_2}$$

浮点乘法流程：

1. **Sign**：结果符号为两个操作数符号异或。
2. **Add Exponents**：指数相加；若使用偏置指数，需减去一次 bias。
3. **Multiply Significands**：尾数相乘。
4. **Normalize**：规格化乘积。
5. **Over/Underflow Check**：检查指数范围。
6. **Rounding**：舍入并在必要时再次规格化。

十进制示例

计算：

$$1.110 \times 10^{10} \times 9.200 \times 10^{-5}$$

指数相加：

$$10 + (-5) = 5$$

尾数相乘：

$$1.110 \times 9.200 = 10.212$$

得到：

$$10.212 \times 10^{5} = 1.0212 \times 10^{6}$$

舍入后：

$$1.021 \times 10^{6}$$

偏置指数示例

课件例子：

$$1\ 10000010\ 000\ldots000 = -1 \times 2^{3}$$

$$0\ 10000011\ 000\ldots000 = +1 \times 2^{4}$$

符号：$1 \oplus 0 = 1$，结果为负。尾数均为 1，乘积仍为 1。单精度 bias 为 127：

$$10000010_2 + 10000011_2 - 01111111_2 = 10000110_2$$

即 $130 + 131 - 127 = 134 = 127 + 7$。结果：

$$1\ 10000110\ 000\ldots000 = -1 \times 2^{7}$$

二进制乘法示例

计算 $0.5_{10} \times (-0.4375_{10})$：

$$0.5_{10} = 1.000_2 \times 2^{-1}$$

$$-0.4375_{10} = -1.110_2 \times 2^{-2}$$

指数：

$$-1 + (-2) = -3$$

尾数：

$$1.000_2 \times 1.110_2 = 1.110_2$$

符号为负，因此：

$$-1.110_2 \times 2^{-3} = -0.21875_{10}$$

4.7.4　浮点乘法、除法与硬件复杂度

FP multiplier 与 FP adder 复杂度相近，只是用尾数乘法器替代尾数加法器。浮点运算硬件通常支持：

- 加法、减法、乘法、除法。
- 倒数 (reciprocal)、平方根 (square-root)。
- 浮点数与整数之间的转换。

  这些操作通常需要多个周期完成，也适合流水线化。

4.7.5　浮点除法概要 (Floating-Point Division)

浮点除法与乘法类似，只是指数和尾数处理方向相反：

1. **Subtract Exponents**：指数相减；若使用 bias，需要做偏置修正。
2. **Divide Significands**：尾数相除。
3. **Normalize**：规格化结果。
4. **Rounding**：舍入。
5. **Sign**：结果符号由两个操作数符号异或决定。

4.8　全章总结 (Summary)

本讲的主线可以压缩为“用同一套硬件思想构造多种算术操作”：

1. **加法器是基础**：半加器、全加器构成多位加法器；RCA 简单但慢，CLA/Prefix 通过提前计算进位提速。
2. **补码统一加减法**：减法可转成 $A + \bar{B} + 1$，同一套加法器既服务加法又服务减法。
3. **标志位必须按语义解释**：无符号看 CF，带符号看 OF，ZF 和 SF 则用于结果状态判断。
4. **ALU 是可控的组合计算核心**：逻辑、算术、比较、移位等功能并行或复用实现，最后由 MUX 选择输出。
5. **乘法是移位加法的系统化**：逐次乘法节省硬件但多周期；Booth 编码减少连续 1 的部分积；阵列乘法器用面积换速度。
6. **除法依赖试减与余数修正**：恢复除法失败时加回除数，非恢复除法用下一轮加法避免恢复步骤。
7. **浮点运算复杂得多**：对阶、规格化、舍入、溢出/下溢检查使 FP 单元通常多周期并流水化。

“算术单元的设计，本质是在正确性、速度、面积和控制复杂度之间做权衡。”

4.9　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. **[Quiz]** Using one 4-bit full adder and necessary gates, design a controllable 4-bit increment/decrement unit. For control $S = 0$ the circuit increments the 4-bit input by 1; for $S = 1$ it decrements the input by 1. Draw the circuit, label every adder input and carry, and state how overflow/carry should be interpreted.

2. **[Exam]** A combinational multiplier multiplies two 5-bit unsigned numbers. At most how many bits does the product need?
   A. 5　B. 8　C. 9　D. 10

3. **[Exam]** In two's complement arithmetic, when does overflow occur?
   A. adding a positive and a negative number　B. carry-out $c_n = 1$　C. the two operands have different sign bits　D. $c_n \oplus c_{n-1} = 1$

4. **[Extra]** Use Booth's algorithm (radix-2 or radix-4) to compute $13 \times (-21)$; show the step-by-step process.

5. **[Extra]** A CPU takes 10 ns per multiply and 1 ns per subtract. How long does computing $d = a \times b - a \times c$ take? Can you rewrite the expression to make it faster?

6. **[Extra]** Use two's complement subtraction to compute $01010100_2 - 01000011_2$ and $01000011_2 - 01010100_2$; state the carry out of the MSB, whether correction is needed, and the final signed result.

7. **[Extra]** Given floats $s1 > s0 > 0$, `t3=0xFF000000`, `t4=0x00800000`. **Without using `fadd.s`**, implement $s2 = s1 + s0$ using only RV32I/RV64I integer instructions (i.e. perform exponent alignment, mantissa addition, and normalization by hand).

第五章　时序逻辑 (Sequential Logic)

本章考点

高频任务：锁存器/触发器特性、setup/hold 时序计算、状态表/状态图、Moore/Mealy、状态化简与编码、序列检测器、寄存器与计数器。**复习路线**：分析电路时固定写“激励方程 → 次态方程 → 输出方程 → 状态表/图”；设计题则反向进行。

5.1　时序逻辑基础 (Foundations of Sequential Logic)

5.1.1　时序电路的组成 (Sequential Circuit Model)

时序电路的核心区别在于：输出不仅取决于**当前输入**，还取决于**过去输入形成的当前状态**。因此，一个典型时序电路由两部分构成：

- **存储元件 (Storage Elements)**：锁存器 (Latches) 或触发器 (Flip-flops)，负责保存状态。
- **组合逻辑 (Combinational Logic)**：根据外部输入和当前状态产生输出与下一状态。

常用数学描述如下：

$$\text{Next State} = f(\text{Inputs}, \text{Present State})$$

$$\text{Output}_{\text{Mealy}} = g(\text{Inputs}, \text{State}), \qquad \text{Output}_{\text{Moore}} = h(\text{Present State})$$

时序电路的三类信号

- **Inputs**：来自外部环境的输入信号。
- **State / Present State**：存储元件当前保存的状态。
- **Next State**：组合逻辑送回存储元件、准备在下一时刻保存的新状态。

5.1.2　同步与异步时序电路 (Synchronous vs. Asynchronous)

时序电路可按存储元件观察输入、改变状态的时间方式分类：

- **同步时序电路 (Synchronous Sequential Circuits)**：行为只在离散时刻定义，存储元件只在时钟信号相关的时刻观察输入并改变状态。同步抽象让复杂数字系统可设计、可分析。
- **异步时序电路 (Asynchronous Sequential Circuits)**：行为由任意时刻输入变化及变化顺序决定。如果把时钟也看作普通输入，严格来说所有电路都可视为异步；但工程设计中通常采用同步抽象。

5.1.3　门延迟、反馈与状态 (Delay, Feedback and State)

课件用一个 2 输入多路选择器说明了延迟的重要性。理想功能为：

$$Y = \begin{cases} A, & S = 0 \\ B, & S = 1 \end{cases}$$

由于反相器和门电路存在传播延迟，选择信号切换时可能出现短暂毛刺 (Glitch)。如果把输出 Y 反馈到输入 A，电路在 $S = 0$ 时会变成：

$$Y(t) \text{ depends on } Y(t - \Delta)$$

此时 Y 不再只是组合逻辑输出，而是保存了过去的值，成为**状态**。

- 当 $S = 1$ 时，$Y = B$，输出跟随输入。
- 当 $S = 0$ 时，Y 维持先前值，表现出“记忆”。
- 若反馈路径中再加入反相器，$S = 0$ 时电路会不断翻转，形成不稳定振荡器，可粗略产生时钟，但不可作为严谨同步设计的基础。

5.2　基本存储元件 (Basic Storage Elements)

5.2.1　S-R 锁存器 (S-R Latches)

S-R 锁存器通过两个门的交叉耦合实现，是最基本的双稳态存储结构。不同实现方式的输入有效电平不同。

NAND S-R Latch

NAND 型 S-R 锁存器通常是**低有效输入**：

S	R	Q^+	$\overline{Q}^+$	含义
1	1	Q	$\overline{Q}$	保持原状态
0	1	1	0	Set，置 1
1	0	0	1	Reset，清 0
0	0	1	1	禁止输入，释放后可能不稳定

NOR S-R Latch

NOR 型 S-R 锁存器通常是**高有效输入**：

S	R	Q^+	$\overline{Q}^+$	含义
0	0	Q	$\overline{Q}$	保持原状态
1	0	1	0	Set，置 1
0	1	0	1	Reset，清 0
1	1	0	0	禁止输入，释放后可能不稳定

易错点：NAND 与 NOR 的禁止输入不同

- NAND S-R Latch：$S = R = 0$ 禁止。
- NOR S-R Latch：$S = R = 1$ 禁止。
- 禁止输入会破坏 Q 与 $\overline{Q}$ 互补关系，释放后状态可能不可预测。

5.2.2　带时钟的 S-R 锁存器 (Clocked S-R Latch)

在基本 NAND S-R 锁存器前加入两个 NAND 门，可得到带控制端 C 的 S-R 锁存器。只有当 $C = 1$ 时，S 和 R 才被观察；当 $C = 0$ 时保持状态。

C	S	R	$Q(t+1)$
0	X	X	No change
1	0	0	No change
1	1	0	1, Set
1	0	1	0, Clear
1	1	1	Undefined

5.2.3　D 锁存器 (D Latch)

D 锁存器在 S-R 锁存器基础上增加反相器，使 $S = D$，$R = \overline{D}$，从结构上消除 S-R 同时有效的未定状态。

$$Q(t+1) = \begin{cases} Q(t), & C = 0 \\ D(t), & C = 1 \end{cases}$$

- $C = 1$：锁存器透明，输出跟随 D。
- $C = 0$：锁存器关闭，保持原状态。
- D 锁存器也可用传输门 (Transmission Gates) 实现。

5.2.4　锁存器的时序问题 (Latch Timing Problem)

锁存器是**电平敏感**器件。只要控制端处于有效电平，输入到输出路径就可能保持打开。如果输出再经组合逻辑反馈到输入，就可能在一个时钟脉冲内多次变化。

设计目标

时序存储元件在一个时钟脉冲内应只改变一次状态。为打断输入到输出的内部路径，常用两种方案：主从触发器 (Master-Slave Flip-Flop) 和边沿触发器 (Edge-Triggered Flip-Flop)。

5.2.5　主从触发器 (Master-Slave Flip-Flop)

主从 S-R 触发器由两个带时钟的 S-R 锁存器串联构成，第二级使用反相时钟。

- 主锁存器在 $C = 1$ 时观察输入。
- 从锁存器在 $C = 0$ 时更新输出。
- 通过主从两级使用相反时钟电平，打断从输入到输出的直接通路。
- 输出变化会被推迟到时钟脉冲结束，导致电路速度变慢。

主从 S-R 触发器还存在 **1s catching** 问题：当 $C = 1$ 时，S 或 R 的短暂 1 可能被主锁存器捕获，并在随后传递给从锁存器。

5.2.6　边沿触发 D 触发器 (Edge-Triggered D Flip-Flop)

边沿触发器只在时钟跳变瞬间采样输入，在时钟保持稳定电平期间忽略输入变化。D 触发器是同步时序设计中最常用的标准存储元件：

$$\boxed{Q(t+1) = D(t)}$$

- **负边沿触发 D 触发器**：输出在时钟下降沿更新。
- **正边沿触发 D 触发器**：输出在时钟上升沿更新，是课件后续同步电路的默认选择。
- D 输入替代 S/R 后，可避免 S-R 主从触发器中的 1s catching 行为。

5.2.7　直接输入与初始化 (Direct Inputs)

上电或复位时，时序电路通常需要进入已知状态。许多触发器提供异步直接置位/清零输入，绕过时钟立即控制内部锁存器。

R	S	**Clock**	D	**结果**
0	1	X	X	异步 Reset，$Q = 0$
1	0	X	X	异步 Set，$Q = 1$
1	1	↑	0/1	正常边沿触发，$Q^+ = D$
0	0	X	X	状态未定义

5.2.8　触发器时序参数 (Flip-Flop Timing Parameters)

必须掌握以下参数：

- **建立时间 t_s (Setup Time)**：触发边沿到来之前，输入必须提前稳定的时间。
- **保持时间 t_h (Hold Time)**：触发边沿之后，输入仍需保持稳定的时间，常接近 0。
- **时钟脉宽 t_w (Clock Pulse Width)**：高/低电平持续时间需满足最小要求。
- **传播延迟 t_{pd}**：从触发边沿到输出改变的时间，常用

$$t_{pd} = \max(t_{PHL}, t_{PLH})$$

主从触发器的建立时间常等于触发脉冲宽度，而边沿触发器的建立时间通常远小于脉冲宽度，因此边沿触发器更利于高速设计。

5.2.9　J-K 与 T 触发器 (J-K and T Flip-Flops)

J-K 触发器类似 S-R 触发器，但允许 $J = K = 1$，此时状态翻转。

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$\overline{Q(t)}$

T 触发器只有一个输入 T，等价于令 J-K 触发器 $J = K = T$：

T	$Q(t+1)$
0	$Q(t)$，保持
1	$\overline{Q(t)}$，翻转

补充：J-K 和 T 触发器都可用边沿触发 D 触发器作为核心实现，例如 $D = T \oplus Q$。T 输入本身不能把触发器初始化到指定状态，因此 Reset 通常必不可少。

5.3　同步时序逻辑分析 (Synchronous Sequential Logic Analysis)

5.3.1　分析任务与一般流程

时序电路分析的目标是：给定逻辑图，推导状态表、状态图和外部行为。

1. 写出输出方程、触发器输入方程和下一状态方程。
2. 建立带状态的真值表：输入为外部输入和当前状态，输出为电路输出和下一状态。
3. 列出所有状态。

4. 画出状态图。

5. 分析外部功能。

6. 验证正确性，检查自恢复能力，并分析时序约束。

5.3.2　状态表与状态图 (State Table and State Diagram)

状态表通常包含四部分：

- **Present State**：当前状态变量取值。
- **Input**：外部输入组合。
- **Next State**：由当前状态和输入决定的下一状态。
- **Output**：由当前状态以及有时由输入共同决定的输出。

状态图由状态圆圈、有向边和标注构成。Mealy 型输出通常标在边上，格式为“输入/输出”；Moore 型输出通常标在状态中。

5.3.3　分析例 1：两个 D 触发器电路

课件例 1 的状态变量为 $A(t), B(t)$，输入为 $x(t)$，输出为 $y(t)$。由逻辑图可得：

$$D_A = x(A + B), \qquad D_B = x\overline{A}$$

$$A(t+1) = x(A + B), \qquad B(t+1) = x\overline{A}$$

$$y(t) = \overline{x}(A + B)$$

A	B	x	A^+	B^+	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

因为输出 y 与输入 x 直接相关，所以这是 **Mealy 型**电路。

5.3.4　Moore、Mealy 与混合输出

- **Moore Model**：输出只由状态决定，状态图中常写作 S/y。
- **Mealy Model**：输出由状态和输入共同决定，状态图中常写作 x/y。

- **混合模型**：实际设计中部分输出可为 Moore，部分输出可为 Mealy，以简化输出描述。

Moore 与 Mealy 的复习比较

- Mealy 输出可在输入变化后更快响应，通常状态数较少。
- Moore 输出只随状态变化，行为更稳定，通常需要更多状态。
- 序列检测器中常见记忆口诀：**Moore is More**。

5.3.5　等价状态 (Equivalent States)

两个状态等价，当且仅当对每一种可能的输入序列，它们产生完全相同的输出序列。等价地说：

- 对每个输入符号，两个状态产生的输出相同；
- 对每个输入符号，两个状态的下一状态相同或等价。

等价状态可以合并，合并后可能暴露新的等价状态，因此状态化简通常是迭代过程。

5.3.6　分析例 2：三触发器无输入电路

课件例 2 中输入为空，状态变量为 A, B, C，输出 $Z = A$，Reset 初始化到 000。由逻辑图得：

$$A^+ = BC, \qquad B^+ = \overline{B}C + B\overline{C}, \qquad C^+ = \overline{A}\,\overline{C}, \qquad Z = A$$

A	B	C	A^+	B^+	C^+	Z
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	0	0	0	1
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	1	0	0	1

从 Reset 后的状态序列为：

$$000 \to 001 \to 010 \to 011 \to 100 \to 000$$

因此它产生一个周期为 5 的状态循环，并在状态 100 时令输出 $Z = 1$。未用状态 $101, 110, 111$ 分别进入 $010, 010, 100$，具有自恢复能力。

5.3.7　系统级时序约束 (Circuit and System Level Timing)

同步系统中，数据从一个触发器输出，经组合逻辑传播到下一个触发器输入。若时钟周期太短，数据无法在建立时间窗口前稳定，系统会出错。

关键约束为：

$$t_p = t_{\text{slack}} + t_{pd,FF} + t_{pd,COMB} + t_s$$

正确工作要求所有触发器到触发器路径满足：

$$\boxed{t_p \geq \max(t_{pd,FF} + t_{pd,COMB} + t_s)}$$

其中 $t_{\text{slack}} \geq 0$。更精确分析可分别使用 t_{PHL} 与 t_{PLH}，并考虑路径反相次数。

课件计算例

设 $t_{pd,FF} = 1.0$ ns，时钟频率 250 MHz，故 $t_p = 4.0$ ns。

- **边沿触发器**：$t_s = 0.3$ ns

$$4.0 \geq 1.0 + t_{pd,COMB} + 0.3 \Rightarrow t_{pd,COMB} \leq 2.7 \text{ ns}$$

若平均每个门延迟 0.3 ns，约可容纳 9 级门。

- **主从触发器**：$t_s = t_{wH} = 2.0$ ns

$$4.0 \geq 1.0 + t_{pd,COMB} + 2.0 \Rightarrow t_{pd,COMB} \leq 1.0 \text{ ns}$$

约只能容纳 3 级门。

5.4　同步时序逻辑设计 (Synchronous Sequential Logic Design)

5.4.1　设计流程 (Design Procedure)

同步时序电路设计的标准流程如下：

1. **Specification**：明确电路功能、输入、输出、复位和时序要求。
2. **Formulation**：得到状态图或状态表。
3. **State Assignment**：为抽象状态分配二进制编码。
4. **Flip-Flop Input Equations**：选择触发器类型，并由下一状态表推导触发器输入方程。
5. **Output Equations**：由输出表推导输出方程。
6. **Optimization**：用卡诺图或布尔代数优化方程。
7. **Technology Mapping**：映射到具体触发器和门电路技术。
8. **Verification**：验证功能、时序、自恢复和非法状态行为。

5.4.2　状态图构造：序列识别器 (Sequence Recognizer)

状态是对过去输入历史的抽象。对于序列识别器，状态用于记住“目标序列的哪个前缀已经出现”。

构造步骤：

1. 从初始状态开始，表示还没有识别到目标序列的任何有效前缀。
2. 为目标序列的第一个符号建立状态。
3. 继续为每一个连续前缀建立状态。
4. 最终状态或最终转移表示完整序列被识别。
5. 补全所有非目标输入的转移。
6. 特别处理**重叠序列**，保证目标序列无论从输入流的哪个位置开始都能被识别。

5.4.3　例：识别序列 1101 的 Mealy 模型

设输出 1 表示刚刚识别到序列 1101。Mealy 模型状态含义：

- A：未出现任何有效前缀。
- B：已经出现前缀 1。
- C：已经出现前缀 11，或连续两个以上的 1。
- D：已经出现前缀 110。

Present State	**Next State**		**Output**	
	$x=0$	$x=1$	$x=0$	$x=1$
A	A	B	0	0
B	A	C	0	0
C	D	C	0	0
D	A	B	0	1

从 D 状态遇到 1 时识别出 1101，但最后这个 1 又可作为新一次识别的前缀，因此转到 B，支持重叠识别，例如 1101101 中的两次出现。

5.4.4　例：识别 1101 的 Moore 模型

Moore 模型输出必须绑定在状态上，因此需要额外状态 E 表示“完整序列已经出现，当前输出为 1”。

Present State	Next State		Output y
	$x=0$	$x=1$	
A	A	B	0
B	A	C	0
C	D	C	0
D	A	E	0
E	A	C	1

状态 E 和 B 对未来输入的响应相似，但当前输出不同，因此不能合并。

5.4.5　状态编码 (State Assignment)

若有 m 个状态，最少需要 n 位编码：

$$n = \lceil \log_2 m \rceil$$

使用 n 位编码时有 $2^n - m$ 个未用状态。不同编码会改变输出方程、激励方程和电路复杂度。

状态编码四条经验规则

1. 对相同输入具有相同下一状态的当前状态，尽量分配相邻编码。
2. 同一当前状态在相邻输入下得到的下一状态，尽量分配相邻编码。
3. 具有相同输出的状态，尽量分配相邻编码。
4. 初始状态或最常用状态优先分配为 0。

优先级上，第一条通常最重要；若状态对在多条规则中多次出现，应优先让这些状态相邻。

5.4.6　状态编码例：Counting vs. Gray

对于 1101 的 Mealy 状态表，若采用 Counting Order：

$$A = 00, \quad B = 01, \quad C = 10, \quad D = 11$$

设状态位为 Y_1Y_2，输入为 X，使用 D 触发器时可得一组方程：

$$D_1 = Y_1\overline{Y_2} + X\overline{Y_1}Y_2$$

$$D_2 = \overline{X}Y_1\overline{Y_2} + X\overline{Y_1Y_2} + XY_1Y_2$$

$$Z = XY_1Y_2$$

若采用 Gray Code：

$$A = 00, \quad B = 01, \quad C = 11, \quad D = 10$$

可得到更简单的一组方程：

$$D_1 = Y_1Y_2 + XY_2, \qquad D_2 = X, \qquad Z = XY_1\overline{Y_2}$$

这说明状态编码会显著影响电路成本。优化后还需进行技术映射，例如映射到带 Reset 的 D 触发器、NAND 门和反相器。

5.4.7　状态编码规则例

课件给出状态转移/输出表：

Present State	$X=0$	$X=1$
A	$C/0$	$D/0$
B	$C/0$	$A/0$
C	$B/0$	$D/0$
D	$A/1$	$B/1$

按规则分析：

- 规则 1 倾向：$A-B$、$A-C$ 相邻。
- 规则 2 倾向：$C-D$、$A-C$、$B-D$、$A-B$ 相邻。
- 规则 3 倾向：$A-B$、$A-C$、$B-C$ 相邻。
- 规则 4：令 $A=00$。

  一种合适编码为：

$$A=00, \quad B=01, \quad C=10, \quad D=11$$

对应编码后的表为：

Present State	$X=0$	$X=1$
00 (A)	10/0	11/0
01 (B)	10/0	00/0
10 (C)	01/0	11/0
11 (D)	00/1	01/1

5.4.8　设计例 3：模 3 累加器

目标：设计一个 2 位操作数的模 3 累加器。

- **Modulo n Adder**：输出为加法结果除以 n 的余数。例如 $2+2 \pmod 3 = 1$。
- **Accumulator**：随时间累计输入操作数，并把累计和保存在状态中。
- 状态/输出：$(Y_1, Y_0) = (Z_1, Z_0)$，表示当前模 3 和。

- 有效输入：$00, 01, 10$。输入 11 对模 3 操作数而言作为 don't care。

Present State	**Next State for X_1X_0**				**Output**
Y_1Y_0	00	01	11	10	Z_1Z_0
00 (A)	00	01	X	10	00
01 (B)	01	10	X	00	01
11 (unused)	X	X	X	X	11
10 (C)	10	00	X	01	10

使用 D 触发器，并把无效输入和未用状态作为 don't care 优化，可得到一组可用方程：

$$D_1 = \overline{Y_1}\,\overline{Y_0}X_1 + Y_0X_0 + Y_1\overline{X_1}\,\overline{X_0}$$

$$D_0 = \overline{Y_1}\,\overline{Y_0}X_0 + Y_0\overline{X_1}\,\overline{X_0} + Y_1X_1$$

$$Z_1 = Y_1, \qquad Z_0 = Y_0$$

5.4.9　未用状态与自启动 (Unused States and Self-Starting)

状态编码可能产生未用状态。电路上电、噪声或干扰可能使 FSM 进入这些状态。

- 若未用状态指向自己或互相循环，电路可能"冻结"在非法区域。
- 若所有未用状态都能回到合法状态，称为 **self-starting**。
- 可通过 Reset 逻辑强制进入初始状态。
- 若非法状态不可接受，可设计 ERROR 状态并输出错误码。
- 若使用 don't care 优化，应在最终硬件实现后检查非法状态行为，而不能只看抽象状态表。

5.5　经典时序逻辑电路 (Classic Sequential Logic Circuits)

5.5.1　寄存器 (Registers)

寄存器是一组二进制存储元件的集合，可保存多位数据。它既可被看作时序逻辑电路，也可被看作存储一个二进制向量的硬件模块。

- 一个触发器只能存 1 位；n 位寄存器由 n 个触发器组成。
- n 位寄存器有 2^n 个状态、2^n 种输入组合和 2^n 种输出组合。
- 简单并行寄存器中，下一状态等于输入向量，输出等于当前状态向量。
- 寄存器是现代处理器中的核心部件，作为比主存更快的临时存储。

2 位寄存器示例

设输入为 In_1In_0，状态为 A_1A_0，输出为 Y_1Y_0。则：

$$A_1(t+1)A_0(t+1) = In_1In_0, \qquad Y_1Y_0 = A_1A_0$$

这是 Moore 型，因为输出只由寄存器当前状态决定。

5.5.2　使能与时钟门控 (Enable and Clock Gating)

课件展示了 4 位寄存器的 Clock Gating 以及带 EN 的触发器。复习时可抓住两种实现思想：

- **Clock Gating**：关闭某些寄存器的时钟以节能，但必须避免门控时钟产生毛刺，实际芯片中通常使用专门的时钟门控单元。
- **Flip-Flop with EN**：用使能控制是否加载新值，常见逻辑为

$$D_{\text{in}} = \begin{cases} D_{\text{new}}, & EN = 1 \\ Q, & EN = 0 \end{cases}$$

即用多路选择器在新输入和保持值之间选择。

5.5.3　寄存器传送与微操作 (Register Transfers and Microoperations)

寄存器传送操作描述寄存器中数据的移动和处理，由三部分指定：

- 系统中的寄存器集合。
- 对寄存器数据执行的操作。
- 控制这些操作发生顺序的控制信号。

微操作是寄存器数据上的基本操作，例如加载、计数、加法、减法、移位。常见四类：

- **Transfer**：寄存器间移动数据。
- **Arithmetic**：加、减、乘、除等算术操作。
- **Logic**：按位 OR、AND、XOR、NOT 等逻辑操作。
- **Shift**：寄存器内数据左右移动。

控制表达式

控制表达式写在操作左侧，用冒号分隔：

$$P : R_2 \leftarrow R_1$$

当 $P = 1$ 时操作发生，$P = 0$ 时操作不发生。

加减法示例：

$$\overline{X}K_1 : R_1 \leftarrow R_1 + R_2$$

$$XK_1 : R_1 \leftarrow R_1 + \overline{R_2} + 1$$

当 $X = 0$ 时执行加法；当 $X = 1$ 时加上 R_2 的补码，实现减法。

5.5.4　寄存器传送结构 (Register Transfer Structures)

常见结构包括：

- **专用多路选择器 (Dedicated Multiplexers)**：每个寄存器前有自己的 MUX，常用于移位寄存器和计数器。
- **总线型传送 (Bus-Based Transfers)**：多个输入经共享 MUX 驱动总线，再送到多个寄存器。
- **三态总线 (Three-State Bus)**：多个三态驱动器连接到同一总线，同一时刻只允许一个驱动器有效。
- **混合结构**：多个 MUX、多个总线和三态结构组合使用。

5.5.5　移位寄存器 (Shift Registers)

移位寄存器能把保存的位向 MSB 或 LSB 方向移动。最简单形式是把一排 D 触发器串联：

$$In \to A \to B \to C \to Out$$

- In：串行输入，也称 shift right input。
- Out：串行输出。
- (A, B, C, Out)：若可同时访问，则称为并行输出。

串行移位行为

初始未知用 ? 表示，输入序列对应课件表格：

时刻	In	A	B	C	Out
T_0	0	?	?	?	?
T_1	1	0	?	?	?
T_2	1	1	0	?	?
T_3	0	1	1	0	?
T_4	1	0	1	1	0
T_5	1	1	0	1	1
T_6	1	1	1	0	1

并行加载移位寄存器

在每一级 D 触发器前加入多路选择器，可在“移位”和“并行加载”之间选择。

Shift	Load	Operation
0	0	No Change
0	1	Parallel Load
1	X	Shift Down

若每级前使用 4 输入 MUX，还可统一支持保持、下移、上移、并行加载：

S_1	S_0	**Register Operation**
0	0	No change
0	1	Shift down
1	0	Shift up
1	1	Parallel load

5.5.6　计数器 (Counters)

计数器是按特定状态序列循环的时序电路。它可以向上计数、向下计数，或按任意固定序列计数。其状态值本身通常就是输出。

- 可用作简单时钟或事件计数器。
- 可记录发送/接收了多少位、计算进行了多少步。
- 所有处理器都有程序计数器 PC，用于记录当前执行指令位置，通常随时钟递增。

5.5.7　波纹计数器 (Ripple Counters)

波纹计数器中，时钟只直接连接到最低位触发器；更高位触发器由前一级输出触发。因此它并非真正同步。

- 最低位每个时钟边沿翻转。
- 每个更高位的翻转频率是前一位的一半。
- 状态变化从 LSB 向 MSB 逐级传播，称为 ripple。
- n 位最坏传播延迟约为 $n \cdot t_{PHL}$ 或对应方向传播延迟之和。
- 优点是结构简单、功耗较低；缺点是高位变化滞后，易产生中间暂态。

3 位向上计数示例：

$$000 \to 001 \to 010 \to 011 \to 100 \to 101 \to 110 \to 111 \to 000$$

5.5.8　同步计数器 (Synchronous Counters)

同步计数器让所有触发器共享同一时钟，通过组合逻辑生成下一状态。对于二进制向上计数器，可使用增量器：

- XOR 用于决定某一位是否翻转。
- 某位翻转的条件是所有低位均为 1。
- Count Enable 可强制保持状态。
- Carry Out 可连接到更高位计数器的 Count Enable，实现级联。

若用串联 AND 形成进位链，会产生较长路径延迟，称为 **serial gating**。可改用并行 AND 或类似超前进位思想实现 **parallel gating**，减少大规模计数器的长路径。

5.5.9　模 N 计数器与 BCD 计数器

模 N 计数器按 N 个状态循环，计数值相当于除以 N 的余数。N 不一定是 2 的幂；模 10 计数器即 BCD 计数器。

同步 BCD 计数器：D 触发器实现

BCD 计数器合法状态为 0000 到 1001，其中 $Q_8Q_4Q_2Q_1$ 表示十进制 0 到 9；状态 1010 到 1111 可作为 don't care 优化。常见优化方程为：

$$D_1 = \overline{Q_1}$$

$$D_2 = \overline{Q_8}\,\overline{Q_2}Q_1 + Q_2\overline{Q_1}$$

$$D_4 = \overline{Q_4}Q_2Q_1 + Q_4\overline{Q_2} + Q_4\overline{Q_1}$$

$$D_8 = Q_8\overline{Q_1} + Q_4Q_2Q_1$$

终端计数/进位输出常取：

$$Y = Q_8Q_1$$

非法状态行为

由上述方程可得 6 个非法状态的实际下一状态：

Present State	Decimal	Next State
1010	10	1011
1011	11	0110
1100	12	1101
1101	13	0100
1110	14	1111
1111	15	0010

该设计若进入非法状态，会在两个时钟周期内回到合法状态。是否足够取决于应用和设计规范。可选措施包括：

- 增加非法状态检测信号，例如

$$Invalid = Q_8(Q_4 + Q_2)$$

- 修改逻辑使非法状态一个周期内回到合法状态。
- 强制非法状态回到指定状态，如 0000。

同步 BCD 计数器：T 触发器实现

若用 T 触发器，利用 $T = Q \oplus Q^+$，可得到一组常用激励方程：

$$T_1 = 1, \qquad T_2 = \overline{Q_8} Q_1, \qquad T_4 = Q_2 Q_1, \qquad T_8 = Q_1(Q_8 + Q_4 Q_2)$$

同样需要处理 1010 到 1111 的 don't care 与自恢复问题。

5.5.10　模 N 计数的实现方法

课件总结了用 n 位二进制计数器构造模 N 计数器的几类方法：

- 检测终端计数 N，异步 Clear 到 0 或异步 Load 0。该方法会让终端状态只存在极短时间，可能因时序条件失败。
- 检测终端计数 $N-1$，同步 Clear 到 0。
- 检测终端计数 $N-1$，同步 Load 0。
- 检测某个终端计数，并用 Load 预置为“终端计数值减去 $N-1$”。
- 完全按状态表自定义设计模 N 计数器，例如 BCD 计数器。

模 7：检测 7 后异步清零

用带异步 Clear 的 4 位同步二进制计数器检测 7 并清零，会得到：

$$0, 1, 2, 3, 4, 5, 6, 7(\text{short}), 0, \ldots$$

状态 7 只短暂出现，课件称为 **suicide counter**。这种设计可能因门延迟和异步清零竞争而失败，复习时记住：**不要这样做**。

模 7：检测 6 后同步加载 0

更可靠方法是检测终端合法状态 6，在下一个时钟同步 Load 0：

$$0, 1, 2, 3, 4, 5, 6, 0, 1, \ldots$$

若把 7 以上状态作为 don't care，检测 6 可简化为：

$$Load = Q_2 Q_1$$

模 6：预置 9，检测 14 后同步加载 9

用 4 位同步二进制计数器实现模 6 的另一种思路是让它循环：

$$9, 10, 11, 12, 13, 14, 9, \ldots$$

Reset 时预置 9，检测到 14 后同步 Load 9。若终端计数为 15，许多计数器内部已有 Carry Out 可用。

60 进制分钟计数

分钟的十进制显示可用两个 BCD/模计数器级联：

- 个位：模 10，计数 0 到 9。
- 十位：模 6，计数 0 到 5。
- 个位从 9 回到 0 时给十位一个进位。
- 整体从 59 回到 00，实现模 60。

5.6　全章总结 (Summary)

本章围绕“数字系统如何记住过去”展开。复习时建议抓住以下主线：

1. **状态来自反馈与存储**：组合逻辑一旦引入反馈和延迟，就可能保存过去的值；稳定存储需要锁存器和触发器。
2. **锁存器是电平敏感，触发器是边沿敏感**：D 锁存器透明时会持续跟随输入；边沿触发 D 触发器只在时钟边沿采样，公式 $Q^+ = D$ 是同步设计核心。
3. **S-R、J-K、T、D 各有状态表**：S-R 有禁止输入；J-K 的 11 翻转；T 的 1 翻转；D 直接保存输入。
4. **同步分析从方程到状态图**：先写触发器输入方程、下一状态方程和输出方程，再建状态表、状态图，最后分析功能与自恢复。
5. **同步设计从规格到硬件**：规格说明、状态图、状态编码、触发器方程、输出方程、优化、技术映射和验证缺一不可。
6. **状态编码影响成本**：相邻编码可简化卡诺图；Gray 编码、初始状态取 0、相同输出相邻等规则都是工程经验。
7. **时序约束决定最高频率**：所有触发器到触发器路径必须满足

$$t_p \geq t_{pd,FF} + t_{pd,COMB} + t_s$$

8. **寄存器、移位寄存器、计数器是常用积木**：寄存器保存向量，移位寄存器移动位，计数器按固定状态序列循环。
9. **异步清零构造模计数器风险高**：检测 N 后异步清零会产生短暂非法脉冲；更推荐检测 $N-1$ 后同步清零或同步加载。
10. **不要忽视非法状态**：don't care 可降低成本，但最终必须检查非法状态是否能自恢复，必要时加入 Reset、Invalid 或 ERROR 逻辑。

“组合逻辑回答现在该做什么；时序逻辑记住过去，从而决定下一步。”

5.7　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. **[Exam]** For an SR latch built from NOR gates, which input drives it into the invalid state?
   A. S=0, R=0　B. S=0, R=1　C. S=1, R=0　D. S=1, R=1

2. **[Exam]** Which of the following is correct?
   A. A D flip-flop can be built from combinational logic alone.
   B. In complex synchronous systems, level-triggered latches usually have better timing robustness than edge-triggered flip-flops.
   C. To realize the same function, a Moore machine generally needs more states than a Mealy machine.
   D. None of the above.

3. **[Exam]** A traffic-light controller has three states: 00 green, 01 yellow, 11 red; when input X=1 it stays on yellow. Implement it with two D flip-flops.
   (a) Write the complete state-transition table; (b) draw the state diagram; (c) use K-maps to find the next-state equations D_1, D_0; (d) draw the complete sequential circuit.

4. **[Quiz]** Design a controlled 3-bit counter with input C. Starting from 000, when $C = 1$ it repeats
   $$000 \to 100 \to 110 \to 111 \to 011 \to 000;$$
   when $C = 0$ it repeats
   $$000 \to 100 \to 110 \to 010 \to 011 \to 000.$$
   Use D flip-flops. Give the state diagram, complete state table (including unused states), simplified next-state equations, output equations, and logic circuit; state how unused states are handled.

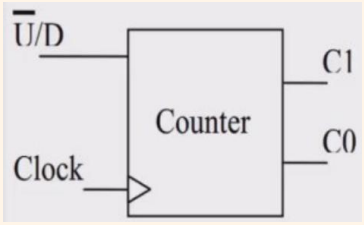
5. **[Extra]** A sequential circuit has one D flip-flop (output S) with $D = X \oplus Y \oplus S$. Find the state table and state diagram.

6. **[Extra]** Design a circuit with two D flip-flops A, B and input X: when X=0 hold; when X=1 cycle through $00 \to 10 \to 11 \to 01 \to 00$.

7. **[Extra]** Using D flip-flops and gates, design a counter with the cyclic sequence $0, 2, 1, 3, 4, 6, 5, 7$.

8. **[Extra]** Design a Mealy machine that detects the sequences **101** and **110**: output `10` on detecting 101, `11` on detecting 110, otherwise `00`. Draw the state diagram, state table, next-state equations, output equations, and circuit.

9. **[Extra]** Design a 2-bit up/down saturating counter (D flip-flops; the module is shown below): direction input $U/\overline{D}$ and clock, outputs C_1C_0; if $U/\overline{D}$=0 and not yet at 11 then increment, if $U/\overline{D}$=1 and not yet at 00 then decrement. Give the state diagram, state table, simplified equations, and circuit.



10. **[Extra]** Handshake-detection FSM: synchronous inputs R, A and an asynchronous `RESET`, output E. When the handshake transitions occur in the correct order E=0; if out of order, E=1 and stays until `RESET`. Design this detector.

11. **[Extra]** Fill in the blanks: (1) The next state of a sequential circuit depends not only on the current input but also on __; (2) the setup time of a master-slave flip-flop is generally __ that of an edge-triggered flip-flop; (3) a circuit whose output depends only on the current state is a __ machine; (4) encoding 108 keyboard symbols needs at least __ binary bits; (5) for a basic SR latch built from two NOR gates, when both inputs are 0, Q __.

12. **[NumLogic]** Which function does a basic SR flip-flop **not** provide? A. hold B. toggle C. set D. reset.
Which one has the "ones-catching" problem? A. SR latch B. D latch C. edge-triggered D flip-flop D. SR master-slave flip-flop.

13. **[Extra]** In an 8-bit binary up-counter, how many flip-flops toggle when counting from `10100011` to the next number?

14. **[Extra]** Register micro-operations: $R1 = (10110101)_2$, $R2 = (01110110)_2$. Execute in order (a) $R1 \leftarrow sr\, R1$ (shift right) (b) $R1 \leftarrow R1 \oplus R2$; find $R1$.
74LS153 (dual 4-to-1 MUX): $I_0 \sim I_3 = 1010$, $S_1S_0 = 10$, output Y =__; for Y to be a square wave, S_1S_0 may be __.

15. **[NumLogic]** Modulo-counter multiple choice: (a) To design a mod-8 counter from a 4-bit synchronous binary counter, the Load signal can be asserted as: A. $\overline{Q_2Q_1Q_0}$ B. $Q_2Q_1Q_0$ C.

$\sim Q_2 \sim Q_1 \sim Q_0$ D. $Q_2 + Q_1 + Q_0$. (b) For a synchronous BCD counter (Q_3 is MSB), what are the output period and positive pulse width of Q_3? A. 10 cycles / 1 B. 10 cycles / 2 C. 16 cycles / 2 D. 16 cycles / 10. (c) When designing a mod-N counter from a synchronous binary counter, Load makes the counter: A. load a nonzero initial value once B. initialize all flip-flops once C. load the initial value on the next clock edge when it reaches $N-1$ (or before start) D. none. (d) At least how many 4-bit binary counters are needed to build a counter with $N = 2010$? A. 12 B. 11 C. 3 D. 2.

16. **[Class]** Draw the logic diagram of a 4-bit register with mode-select inputs S_1, S_0 operating per the table below (build the multiplexer from "AND-gate enable + OR gate", with select control S_1S_0):

S_1	S_0	Operation
0	0	Hold (no change)
0	1	Parallel load
1	0	Shift right
1	1	Shift left

17. **[Class]** Design a register cell for an 8-bit register A that realizes the register transfers:

$$C_0: \ A \leftarrow A \cap B, \qquad C_1: \ A \leftarrow A \cup \overline{B}$$

Use AND/OR/NOT gates to find the optimal logic for the D input of the flip-flop in each cell (analyze the conflict when $C_0C_1 = 11$ and 00, and give the improved K-map and expression).

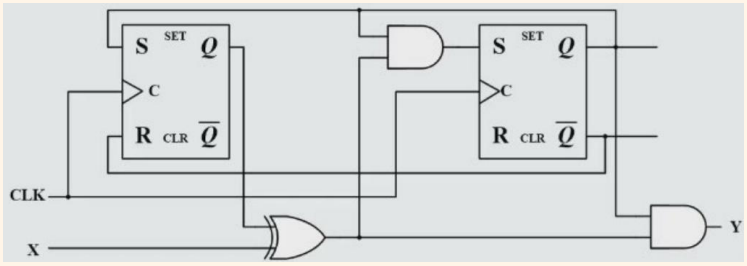
18. **[Class]** Given two register-transfer statements ($R1$ unchanged otherwise):

$$C_1: \ R1 \leftarrow R1 + R2 \text{ (add)}, \qquad C_2: \ R1 \leftarrow R1 + 1 \text{ (increment)}$$

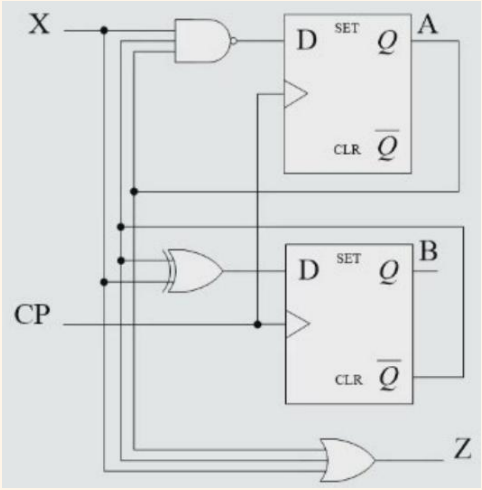
(a) Draw a logic diagram realizing both transfers using one 4-bit counter with parallel load and one 4-bit adder; (b) instead realize it using one 4-bit adder plus the necessary external gates, and compare with (a).

19. **[Class]** A sequential circuit has the device delays: inverter $t_{pd} = 0.01\,\text{ns}$; XOR gate $t_{pd} = 0.04\,\text{ns}$; flip-flop $t_{pd} = 0.08\,\text{ns}$, $t_s = 0.02\,\text{ns}$, $t_h = 0.01\,\text{ns}$. Find:(a) the longest path delay from an external input through gates only to an external output; (b) the longest path delay from an external input to a clock rising edge; (c) the longest path delay from a clock rising edge to an output; (d) the longest path delay from clock rising edge to clock rising edge; (e) the maximum operating frequency (MHz/GHz).

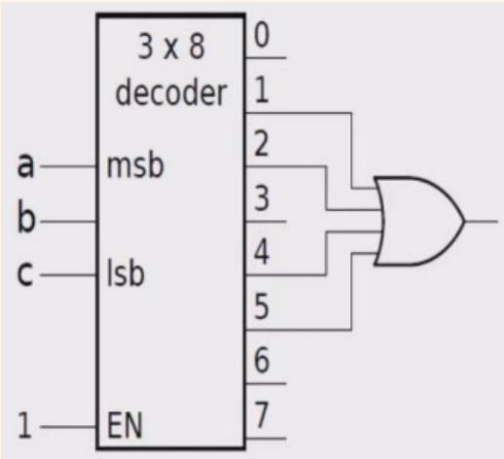
20. **[Recall]** In the sequential circuit below, XOR gate $t_{pd} = 3.0\,\text{ns}$, AND gate $t_{pd} = 1.5\,\text{ns}$, flip-flop $t_{pd} = 3.0\,\text{ns}$, $t_s = 1.0\,\text{ns}$, $t_h = 0.25\,\text{ns}$. How long before the clock rising edge must the external input be stable? A. 8.5 ns　B. 7.5 ns　C. 6.5 ns　D. 5.5 ns



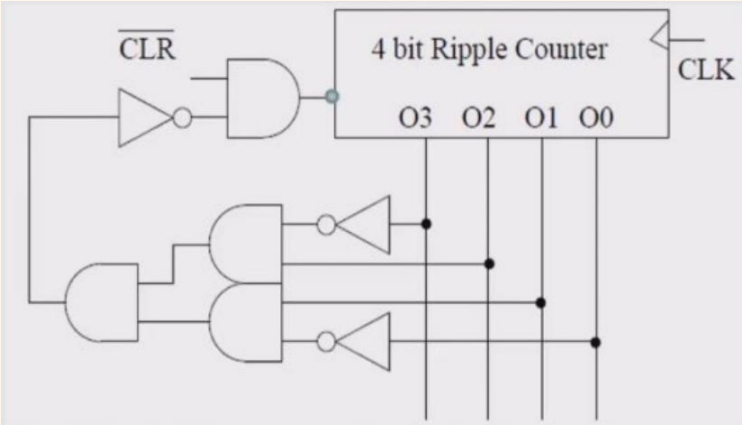
21. **[Recall]** Analyze the circuit below (two D flip-flops, input X, output Z, clock CP): write the output equation and next-state equations, build the state table, and draw the state diagram.



22. **[Recall]** What output function does the circuit below realize? (3×8 decoder, inputs a, b, c, EN=1, outputs combined through an OR gate.)



23. **[Recall]** The circuit below is a mod-N counter with a synchronous reset signal. What is N? A. 6　B. 7　C. 8　D. 9



第六章　指令集体系结构与寻址 (ISA & Addressing)

本章考点

高频任务：ISA 与微体系结的边界、零至三地址指令、有效地址 EA、各种寻址方式、指令字段位宽、CISC/RISC 权衡。**复习路线**：寻址题先写 EA，再判断取到的是操作数、地址还是地址的地址。

6.1　ISA 基础 (Foundations of ISA)

6.1.1　ISA 在计算机系统中的位置 (System Stack)

指令集体系结构 (Instruction Set Architecture, ISA) 是软件与硬件之间最重要的接口。课件给出的层次可以概括为：

层次	说明
Application	应用程序，面向最终用户的功能。
Compiler / Libraries	编译器、宏库、过程库，把高级语言映射到机器相关代码。
Operating System	操作系统，管理资源并提供运行环境。
Instruction Set Architecture	**软硬件契约：软件可见的机器接口。**
Computer Organization / Microarchitecture	ISA 的具体实现方式，如流水线、缓存、执行单元组织。
RTL / Digital Logic / Device Technology	寄存器传输级、数字逻辑电路和器件技术。

ISA 的核心定位

- ISA 位于操作系统和微体系结构之间，是软件能直接“看见”的硬件抽象。
- ISA 定义机器语言；汇编语言是机器语言的人类可读表示。
- 同一个 ISA 可以有多个不同实现。例如同属 x86 ISA 的 Xeon 和 Celeron 可以采用不同微体系结构。

6.1.2　什么是指令 (What is an Instruction)

一条指令由**操作码 (opcode)** 和 **操作数 (operands)** 组成。指令告诉处理器要做什么、对什么数据做、结果放在哪里，以及下一步执行哪里。

- **Operation Code (Opcode)**：指定操作类型，例如加、减、装载、跳转。
- **Source Operand Reference(s)**：源操作数位置，即“对谁操作”。
- **Result Operand Reference(s)**：结果操作数位置，即“答案放哪里”。
- **Next Instruction Reference**：下一条指令的位置。通常由程序计数器 (PC) 自动递增得到，分支和跳转指令会改变它。

6.1.3　操作数的位置 (Where Are the Operands)

操作数可以来自多个地方：

- **Main Memory**：主存中的某个地址。
- **CPU Register**：CPU 内部寄存器，速度快、编号短。
- **I/O Device**：输入输出设备。
- **Instruction Itself**：指令中直接携带的立即数。

为了说明“哪个寄存器、哪个内存位置、哪个 I/O 设备”，ISA 必须设计对应的**寻址方式 (addressing scheme)**。

6.1.4　指令格式与指令集 (Instruction Formats & Instruction Set)

指令格式是机器码中各个字段的安排方式，设计时通常需要考虑：

- **Instruction Length**：短指令、长指令或可变长指令。
- **Number of Operands**：0、1、2 或 3 个显式操作数。
- **Number of Addressable Registers**：可编码多少寄存器。
- **Memory Organization**：内存按字节寻址还是按字寻址。
- **Addressing Modes**：直接、间接、变址等寻址方式如何编码。

指令集 (Instruction Set) 是 CPU 能理解的全部指令集合。它最终以二进制机器码，也称目标代码 (object code)，存在于程序中；程序员通常用汇编语言表示这些机器码模式。

6.1.5　ISA 的定义与机器状态 (ISA Specifies State Change)

ISA 不只是“指令列表”，还包括程序员可见的机器视图：

- 指令集本身。
- 寄存器数量、名称和角色。
- 内存组织方式，例如地址长度、字节序、字节/字寻址。
- 异常和中断处理模型。

从状态机角度看，一条指令把计算机从一个机器状态转换到另一个机器状态：

$$\text{After State} = \text{Instruction}(\text{Before State})$$

机器状态至少包含：

- **PC**：程序计数器，指向当前或下一条指令。
- **Memory State**：主存内容，例如按字节寻址、小端序、32 位地址空间。
- **Register State**：寄存器文件内容，例如 $R0$ 到 $R31$，每个 32 位。

因此，指令格式、指令类型、寻址方式共同决定一次状态变化如何发生。

6.1.6　ISA 像“方言”(Different ISAs as Dialects)

不同 ISA 可以完成同样的计算任务，但表达方式不同，类似不同地区的方言。早期 von Neumann 体系结构思想指出：从形式逻辑上说，某些简单指令集已经足以控制任意运算序列；但实际选择 ISA 时，决定性因素是硬件简单性、程序表达清晰性和执行速度。也就是说，**理论上能做**不等于**工程上好用**。

6.1.7　指令格式设计思路 (Thoughts of Instruction Formats)

课件列出的指令格式设计建议可以整理为：

- 指令长度尽量短，减少取指带宽与存储空间。
- 为未来新操作码预留足够位数，避免 ISA 过早耗尽编码空间。
- 编码要清晰、可区分，便于硬件译码和文档说明。
- 合理选择操作数个数。
- 注意指令对齐，例如字节级或字级对齐。
- 尽量保持规则性：固定指令长度、固定操作码位数、固定操作数字段位置等。

6.1.8　四条设计原则 (Design Principles)

Hennessy 和 Patterson 总结了 ISA 设计中反复出现的四条原则：

ISA 设计四原则

1. **Simplicity favors regularity**：简单性偏好规则性。例如固定指令大小、固定格式、统一数据格式能简化硬件。
2. **Make the common case fast**：让常见情况更快。常用数据应尽量放在寄存器中，减少读、移动和写的位数。
3. **Smaller is faster**：更小通常更快。小常数很常见，因此立即数字段可以较小；寄存器编号字段也远短于内存地址字段。
4. **Good design demands good compromises**：好设计需要折中。重要例外可能需要特殊格式，例如跳转到很远目标时，小立即数字段不够用。

6.2　ISA 设计问题 (Issues in ISA Design)

6.2.1　基本设计问题 (Basic Design Issues)

ISA 设计必须回答以下问题：

- **Data Types**：支持哪些数据类型？大小是多少？
- **Operations**：提供多少操作？每个操作有多复杂？理论上 `LD/ST/INC/BRN`，甚至“减法 + 分支”也能编码任意计算，但程序会很长，实用性差。
- **Operands**：每条指令如何说明操作数？大多数操作是二元操作，例如 $A \leftarrow B + C$；也有一元操作，例如 $A \leftarrow\sim B$。
- **Operand and Result Locations**：操作数和结果能否在内存中？有多少显式操作数？内存操作数如何定位？哪些操作数必须在寄存器中？
- **Instruction Encoding**：如何把操作码、寄存器编号、立即数和地址编码成一致的指令格式？

典型指令集常假设：32 位字长、32 位基本地址、32 位整数等基本操作数；一般情形下可能要引用 3 个操作数，即 $A := B + C$。核心挑战是在较少位数中编码足够多的信息。

6.2.2　设计权衡决策 (Design Tradeoff Decisions)

不同 ISA 会在以下维度上做取舍：

- **Operation Repertoire**：操作数量、能力和复杂度。
- **Data Types**：操作作用于整数、浮点数、字符串、向量还是特殊领域数据。
- **Registers**：寄存器数量、寄存器角色，以及哪些操作能直接作用于寄存器。
- **Addressing**：地址如何指定，是立即数、直接地址、寄存器、间接地址还是变址地址。

6.3　操作数个数 (Number of Operands)

6.3.1　为什么很少见四地址指令

以加法为例，如果要把两个内存位置相加，需要两个源地址；结果还需要一个目的地址；下一条指令若也显式指定，则还要第四个地址。若这些都是内存地址，指令会非常长。

实践中几乎不会看到四地址指令，原因是：

- CPU 控制逻辑和译码复杂度过高。
- 指令字太长，取指成本高。
- 所有操作数并非总是频繁使用。
- 下一条指令通常由 PC 递增得到，只有分支或跳转才显式改变 PC。

因此，大多数 ISA 使用 1、2 或 3 个操作数地址。下面统一用

$$Y = \frac{A - B}{C + (D \times E)}$$

比较不同操作数个数的表达方式。

6.3.2　三操作数指令 (Three-Operand Instructions)

三操作数指令可以同时指定两个源操作数和一个目的操作数：

指令	含义
`SUB R1, A, B`	$R1 \leftarrow A - B$
`MUL R2, D, E`	$R2 \leftarrow D \times E$
`ADD R2, R2, C`	$R2 \leftarrow R2 + C$
`DIV R1, R1, R2`	$R1 \leftarrow R1 / R2$

三地址格式的优点是灵活，结果位置可显式指定；原始的 A, B, C, D, E 不会被破坏。代价是每条指令需要编码更多操作数字段，指令可能更长。

6.3.3　二操作数指令 (Two-Operand Instructions)

二操作数指令通常把第一个操作数同时作为源和目的，即结果覆盖第一个操作数：

指令	含义
`SUB A, B`	$A \leftarrow A - B$
`MUL D, E`	$D \leftarrow D \times E$
`ADD D, C`	$D \leftarrow D + C$
`DIV A, D`	$A \leftarrow A / D$

这样指令较短，但会破坏 A 和 D 的原始值。如果后续还需要原值，必须先复制：

指令	目的
`MOV R1, A`	保存 A 的副本。
`MOV R2, D`	保存 D 的副本。
`SUB R1, B`	$R1 \leftarrow R1 - B$
`MUL R2, E`	$R2 \leftarrow R2 \times E$
`ADD R2, C`	$R2 \leftarrow R2 + C$
`DIV R1, R2`	$R1 \leftarrow R1/R2$

6.3.4　一操作数指令 (One-Operand Instructions)

一操作数指令把另一个操作数隐含在累加器 (Accumulator, ACC) 中：

指令	含义
`LDA D`	$ACC \leftarrow D$
`MUL E`	$ACC \leftarrow ACC \times E$
`ADD C`	$ACC \leftarrow ACC + C$
`STO R1`	$R1 \leftarrow ACC$
`LDA A`	$ACC \leftarrow A$
`SUB B`	$ACC \leftarrow A - B$
`DIV R1`	$ACC \leftarrow ACC/R1$

早期计算机大量使用一地址指令，因为 CPU 设计简单；但程序表达会变得更繁琐，累加器也容易成为瓶颈。

6.3.5　零操作数指令与栈 (Zero-Operand Instructions and Stack)

零操作数指令通常基于栈。栈是后进先出 (LIFO) 的数据结构：

- `PUSH A`：把 A 放入栈顶。
- `POP A`：弹出栈顶元素并放入 A。
- `ADD`：弹出栈顶两个元素，相加后把结果压回栈顶。

对同一个表达式，可以使用如下栈序列：

指令	栈内容变化
`PUSH B`	B
`PUSH A`	B, A
`SUB`	$A - B$
`PUSH E`	$(A - B), E$
`PUSH D`	$(A - B), E, D$
`MUL`	$(A - B), E \times D$
`PUSH C`	$(A - B), E \times D, C$
`ADD`	$(A - B), E \times D + C$
`DIV`	$(A - B)/(E \times D + C)$

6.3.6　操作数个数的取舍

- **更多操作数**：指令更强，程序指令数更少，但指令长、硬件复杂。
- **更多寄存器**：寄存器间操作更快，也便于编译器保存临时变量。
- **更少操作数**：指令短、取指和执行可能更快，但程序指令数更多，表达复杂。

6.4　寻址方式 (Addressing Modes)

6.4.1　有效地址 (Effective Address)

寻址方式说明一条指令的操作数字段如何引用真正想要的数据。它可以指定常量、寄存器或内存位置。操作数真实所在位置称为**有效地址 (effective address, EA)**。某些寻址方式允许在运行时动态计算 EA，这对数组遍历、指针和过程调用非常重要。

6.4.2　立即寻址 (Immediate Addressing)

立即寻址中，操作数字段直接包含要使用的值：

```
ADD #5
```

表示把常数 5 加到某个隐含或显式目的操作数上。数字 5 的补码表示直接存放在指令中。立即数必须在汇编时已知，优点是无需访问内存取操作数。

6.4.3　直接寻址 (Direct Addressing)

直接寻址中，操作数字段给出内存地址：

```
ADD 100
```

表示取内存地址 100 中的内容参与运算，即 $M[100]$。缺点是完整地址必须放入指令，可能限制地址空间。例如 32 位指令字若还要容纳 32 位地址，就几乎没有空间放操作码和其他字段。常见解决办法包括只编码偏移量，或使用隐含段寄存器。

6.4.4 间接寻址 (Indirect Addressing)

间接寻址中，操作数字段给出的地址里保存了真正数据的地址：

$$\texttt{ADD [100]} \quad \Rightarrow \quad M[M[100]]$$

缺点是需要额外一次内存访问；优点是第二级地址可在运行时改变，适合动态访问不同地址，例如遍历数组或处理指针。间接寻址也可以看成取指周期之外增加了一个寻址子周期。

6.4.5 寄存器直接与寄存器间接 (Register Direct / Indirect)

- **Register Direct**：`ADD R5` 表示取寄存器 $R5$ 的内容。寄存器数量有限，编号字段短，不需要像内存地址那样占用很多位。
- **Register Indirect**：`ADD [R3]` 表示 $R3$ 中保存的是内存地址，先到该地址取数据，再参与运算，即 $M[R3]$。

6.4.6 相对、变址与基址寻址 (Relative, Indexed and Based)

- **Relative Addressing**：以 PC 为基准，加上指令中的偏移量得到 EA。常用于分支跳转，因为目标通常离当前位置不远。
- **Indexed Addressing**：使用一个索引寄存器作为偏移量，与指令中的地址相加得到 EA，适合数组访问。
- **Based Addressing**：使用基址寄存器保存基地址，指令中的地址字段表示相对该基地址的位移。
- **Indexed 与 Based 的区别**：索引寄存器更像“相对给定地址的偏移”，基址寄存器更像“区域起点”，指令字段是相对起点的 displacement。

6.4.7 寻址方式总结表 (Summary of Addressing Modes)

Addressing Mode	**Syntax**	**Meaning**
Immediate	$\#K$	K
Direct	K	$M[K]$
Indirect	(K)	$M[M[K]]$
Register	(Rn)	$M[Rn]$
Register Indexed	$(Rm + Rn)$	$M[Rm + Rn]$
Register Based	$(Rm + X)$	$M[Rm + X]$
Register Based Indexed	$(Rm + Rn + X)$	$M[Rm + Rn + X]$

课件强调了四类常见地址计算来源：汇编时已知常数、寄存器内容、两个寄存器之和、寄存器和常数之和。实际 ISA 会给这些组合命名为不同寻址方式。

6.4.8　寻址示例 (Addressing Example)

设内存和寄存器状态如下：

$$M[800] = 900, \quad M[900] = 1000, \quad M[1000] = 500,$$
$$M[1100] = 600, \quad M[1600] = 700, \quad R1 = 800.$$

对装入累加器的 `Load` 指令，各寻址方式得到：

寻址方式	形式	装入 AC 的值
Immediate	`Load 800`	800
Direct	`Load 800`	$M[800] = 900$
Indirect	`Load 800`	$M[M[800]] = M[900] = 1000$
Indexed	`Load R1[800]`	$M[R1 + 800] = M[1600] = 700$

如果写成 `Load R1`，在寄存器直接寻址下会装入 $R1$ 的值 800；若是寄存器间接寻址 `Load [R1]`，则会装入 $M[800] = 900$。

6.5　操作类型与指令编码 (Operations and Encodings)

6.5.1　操作类型 (Types of Operations)

ISA 通常包含以下操作类型：

- **Arithmetic and Logic**：整数、十进制或浮点加减乘除，逻辑与、或、非等。
- **Shift**：左移、右移、循环左移、循环右移。
- **Data Transfer**：`MOV`、`LOAD`、`STORE`。
- **String**：字符串搜索、转换等。
- **Control**：`BRANCH`、`JMP`、`CALL`、`RET`。
- **System**：`HALT`、打开/关闭中断、模式切换等。
- **Input/Output**：输入、输出。

6.5.2　典型操作集合 (Typical Operations)

自 1960 年代以来，主流 ISA 的操作类型变化不大：

类别	典型操作
Data Movement	从内存装载、存入内存、内存到内存移动、寄存器到寄存器移动、I/O 输入输出、栈的 push/pop。
Arithmetic	整数或浮点的 Add、Subtract、Multiply、Divide。
Shift	左/右移，左/右循环移位。
Logical	not、and、or、set、clear。
Control	无条件跳转和条件分支。
Subroutine Linkage	call、return。
Interrupt	trap、return。
Synchronization	test-and-set 等原子读-改-写操作。
String / Graphics	字符串搜索与转换；MMX 等并行子字操作，例如 4 个 16 位数据并行相加。

6.5.3 编码类型 (Types of Encodings)

指令编码大体可分为：

- **Variable Length**：可变长指令。优点是代码密度高，适合强调代码大小的场景；缺点是译码复杂。
- **Fixed Length**：固定长指令。优点是取指、译码和流水线更简单，性能更容易做高；缺点是短操作也占完整指令宽度。
- **Hybrid**：混合长度。兼顾性能和代码密度。

若**代码大小**最重要，倾向使用可变长指令；若**性能**最重要，倾向使用固定长指令。近代嵌入式机器如 ARM、MIPS 增加了可选的 16 位子集模式，例如 Thumb 和 MIPS16，可按过程选择性能或密度。有些架构还探索运行时解压缩指令，以提升代码密度。

6.6 ISA 演化与 CISC/RISC (Evolution and CISC/RISC)

6.6.1 指令集的演化 (Evolution of Instruction Sets)

课件给出的演化路径可概括为：

阶段	代表与特点
Single Accumulator	EDSAC, 1950, Maurice Wilkes。以单累加器为核心。
Accumulator + Index Registers	Manchester Mark I、IBM 700 系列，1953。加入索引寄存器改善地址计算。
Separation of Programming Model from Implementation	程序员模型与内部实现逐渐分离。
High-level Language Based	B5000, 1963。面向高级语言支持。
Concept of a Family	IBM 360, 1964。一个 ISA 家族支持多种实现。
General Purpose Register Machines	通用寄存器机器成为主流基础。
Complex Instruction Sets	VAX、Intel 432，1977–1980，走向 CISC。
Load/Store Architecture	CDC 6600、Cray 1，1963–1976，走向 RISC。
CISC / RISC	CISC 代表 x86、Pentium；RISC 代表 MIPS、SPARC、HP-PA、IBM RS6000、PowerPC 等。

6.6.2 CISC：复杂指令集计算机

CISC (Complex Instruction Set Computers) 的主要思想是缩小编程语义与机器执行之间的差距：

- 指令更复杂，能直接表达较高级操作。
- 代码尺寸更小，这在内存昂贵的时代很重要。
- 编译器可能更简单，因为复杂工作可交给指令和微码。
- 机器内部常有另一个由微码控制的状态机。
- 典型例子：x86、Intel 432、IBM 360、DEC VAX。

x86 例子

- `MOVSD` 是移动双字的 1 字节指令，语义可理解为 $m32[DS:EDI] \leftarrow m32[DS:ESI]$。
- `REP` 是 1 字节前缀，用于重复字符串操作，例如 `REP MOVSD`，重复次数由 `ECX` 设置。
- `LOCK ADD ds:[esi+ecx*2+0x67452301], 0xEFCDAB89` 可编码为 13 字节，包含前缀、复杂地址计算和 32 位立即数，体现了 CISC 的复杂编码能力。

6.6.3 RISC：精简指令集计算机

RISC (Reduced Instruction Set Computer) 的出发点来自 IBM 的观察：可用指令很多，但程序实际频繁使用的指令很少。

- CISC 可能出现“$n+1$”现象：加入一条新指令若需要额外译码逻辑，可能拖慢整个 ISA 的译码路径。
- RISC 起源于 IBM 1975 年的电话项目，目标是达到 12 MIPS，用于每秒约 300 次呼叫、每次约 20k 条指令。
- IBM 801 于 1978 年出现，是早期 RISC 代表。
- RISC 依靠简单指令和更强编译器来换取性能。

6.6.4 典型 RISC 特征 (A Typical RISC)

- 指令数量较少。
- 固定格式指令，例如 32 位。
- 三地址、寄存器到寄存器算术指令。
- 执行阶段常追求单周期操作。
- Load-store 架构：只有 load/store 访问内存，ALU 操作只在寄存器之间进行。
- 简单寻址方式，例如 base + displacement。
- 无间接寻址或极少使用复杂间接寻址。
- 分支条件简单。
- 硬布线控制，无微码。
- 需要更多编译器优化工作。
- 代表：Berkeley RISC I/II、Stanford MIPS、IBM RISC Technology、Sun SPARC、HP PA-RISC、ARM。

6.6.5 MIPS 指令格式示例

MIPS 是典型 RISC ISA，课件列出 5 类格式：

格式	字段	例子
R-format	Op, Rs, Rt, Rd, Shamt, Funct	`add $1, $2, $3`
I-format (Register-Immediate)	Op, Rs, Rt, immediate	`addi $1, $2, -5`
I-format (Load/Store)	Op, Base, Dest, immediate	`lw $1, 24($9)`
I-format (Branch)	Op, Rs, Rt, immediate	`beq L1, $4, $0`
J-format	Op, target	`j L2`

6.6.6 CISC 与 RISC 对比

CISC	**RISC**
可变长指令。	固定长指令，常追求单周期操作。
指令和寻址方式丰富。	指令和寻址方式较少。
译码长且复杂。	译码简单。
可包含内存到内存操作。	Load/store 架构。
使用微码。	无微指令，由硬件逻辑直接译码执行。
语义差距小，把复杂度转移到微码。	需要聪明编译器，或硬件动态重排指令。
IBM 360、DEC VAX、x86、Motorola 68030。	IBM 801、MIPS、RISC I、IBM POWER、Sun SPARC。

6.6.7 CISC/RISC 的现实复杂性

早期机器的现实数据并非完全二分：

CISC/过渡代表	IBM 370/168	VAX 11/780	Xerox Dorado
Year	1973	1978	1978
# Instructions	208	303	270
Microcode	54KB	61KB	17KB
Instruction Size	2–6B	2–57B	1–3B
Execution Model	Reg-reg/Reg-mem/Mem-mem	Reg-reg/Reg-mem/Mem-mem	Stack

RISC 代表	IBM 801	Berkeley RISC1	Stanford MIPS
Year	1980	1981	1983
# Instructions	120	39	55
Microcode	0	0	0
Instruction Size	4B	4B	4B
Execution Model	Reg-reg	Reg-reg	Reg-reg

Colwell 等人与 Patterson、Hennessy 等人围绕 CISC/RISC 的争论说明：**CISC/RISC 不应被看成严格二分法**。例如 DEC 的 MicroVAX-32 是单芯片实现，子集化 VAX 指令但仍有 175 条指令，能模拟复杂指令，具有可变长指令、非 load/store 机器、微码控制和 VAX 寻址方式。它到底算 RISC 还是 CISC，并不容易一句话判定。

6.6.8 现代 x86 与近期趋势

现代 x86 常被称为“CISC 外衣，RISC 内核”：

- Intel P6、Netburst、Core，AMD Athlon、Phenom、Opteron 等都会把 x86 指令动态译码成 μop 或 RISC-op。
- 内部微体系结构采用很多 RISC 设计思想。

- 处理器动态调度 μop，提高并行度。
- 编译器调度仍然有价值，因为硬件重排并不能替代所有静态优化。

近期 ISA 趋势体现为“按应用增加复杂性”:

- MIPS 中也可能出现复杂指令，例如 `CABS.LE.PS $fcc0, $f8, $f10`，用于比较复数/向量绝对值关系。
- 无线通信和 DSP 中会加入 Viterbi 指令。
- SSE 或 DSP 中常见 sum of absolute differences (PSAD)，用于 MPEG 运动估计：$C = \sum |A-B|$。
- 嵌入式领域代码大小很关键。
- 为降低编程工作量、优化性能，可能使用专用硬件加速器、协处理器，或可扩展 ISA plug-in。

6.7　ISA 分类与操作数位置 (Classification of ISAs)

6.7.1　按操作数位置分类

课件按照 ALU 操作数主要来自哪里，把 ISA 分成几类：

类别	典型指令	含义与代表
Accumulator	`add A`	1 地址，$acc \leftarrow acc + mem[A]$。1960 年以前常见，如 68HC11。
Stack	`add`	0 地址，$tos \leftarrow tos + next$。1960–1970 年代常见。
Memory-Memory	`add A, B` 或 `add A, B, C`	2/3 地址，直接以内存为源和目的。1970–1980 年代常见。
Register-Memory	`add R1, A`; `load R1, A`	一个寄存器操作数和一个内存操作数，代表 80x86。
Register-Register (Load/Store)	`add R1, R2, R3`; `load R1, R2`; `store R1, R2`	ALU 只在寄存器间操作；load/store 访问内存，代表 MIPS。

操作数位置图的核心信息是：从 stack、accumulator 到 register-memory，再到 register-register/load-store，通用寄存器 (GPR) 的使用越来越突出。GPR 让操作数更多停留在处理器内部，减少内存访问。

6.7.2　代码序列：C = A + B 在四类指令集中的表示

Stack	Accumulator	Register-Memory	Load-Store
`Push A`	`Load A`	`Load R1, A`	`Load R1, A`
`Push B`	`Add B`	`Add R1, B`	`Load R2, B`
`Add`	`Store C`	`Store C, R1`	`Add R3, R1, R2`
`Pop C`			`Store C, R3`

这张表说明：栈和累加器指令短，但隐含状态强；register-memory 指令数较少但内存参与 ALU；load-store 指令数可能更多，但结构规则、流水线友好。

6.7.3　为什么几乎所有新架构都使用 GPR

通用寄存器几乎成为新 ISA 的标配，原因是：

- 寄存器比内存甚至缓存更快，寄存器值可立即获得。
- 若内存尚未准备好，处理器会等待，形成 stall。
- 寄存器方便保存变量，编译器可以把常用变量分配到寄存器中。
- 指令中寄存器编号字段很短，代码更紧凑；内存地址字段通常更长。

从处理器到存储层次看，速度与容量大致呈反向关系：寄存器最快、容量最小；缓存次之；主存更慢；磁盘最慢但容量最大。

6.7.4　栈架构 (Stack Architectures)

栈架构使用 First-In Last-Out (FILO) 或 Last-In First-Out (LIFO) 数据结构。ALU 操作数隐含在栈顶，算术逻辑操作通常无需显式操作数；`push`/`pop` 需要一个显式操作数。

- **优点**：指令短，代码密度好；硬件要求低；简单编译器容易编写。
- **缺点**：栈顶成为瓶颈；并行性和流水线能力弱；需要的数据不一定在栈顶，常需 `TOP`、`SWAP` 等额外指令；优化编译器难写。
- **补救思路**：为栈中值提供随机访问，或用寄存器文件/缓存保存栈顶若干项，缓解栈大小和访问延迟问题。
- **例子**：1960 年代 Burroughs B5500/B6500、HP 3000/70；现代 Java VM 采用栈式字节码模型。

6.7.5　累加器架构 (Accumulator Architectures)

累加器架构只有一个核心工作寄存器 ACC。指令通常包括：

- **ALU**：$Acc \leftarrow Acc + *M$。
- **Load to accumulator**：$Acc \leftarrow *M$。
- **Store from accumulator**：$*M \leftarrow Acc$。

  其特点是一个显式操作数通常为内存地址，另一个隐式操作数为累加器。
- **优点**：硬件需求很低，设计简单，容易理解；指令较短，内部状态最少。
- **缺点**：累加器成为瓶颈；并行和流水线能力弱；内存流量高，需要大量 load/store。
- **例子**：早期 IBM 7090、DEC PDP-8；现代部分 DSP 架构仍有累加器思想。

6.7.6　内存-内存架构 (Memory-Memory Architectures)

内存-内存架构中，所有 ALU 操作数都可以直接来自内存地址。例如：

```
mul D <- C*D,     add D <- D+B
```

可直接完成 $D = B + (C \times D)$ 一类操作。

- **优点**：不浪费寄存器；若支持三操作数，指令数最低；编译器相对容易生成代码。
- **缺点**：内存流量极高；指令长度变化大；每条指令时钟数变化大；两操作数形式会产生更多数据移动。
- **例子**：VAX。

6.7.7　寄存器-内存架构 (Register-Memory Architectures)

寄存器-内存架构的 ALU 操作通常包含一个内存地址和一个寄存器，常见为二操作数形式。例如：

$$R1 \leftarrow R1 \times M[D], \qquad R1 \leftarrow R1 + M[B]$$

- **优点**：某些数据无需先 load 就可参与运算；指令格式相对容易编码；代码密度好，指令数较少。
- **缺点**：操作数不对等，正交性差；结果可能破坏一个源操作数；指令长度和每条指令周期数变化；更难流水线化；可能限制寄存器数量。
- **例子**：IBM 360/370、VAX、80x86。

6.7.8　装载-存储架构 (Load-Store Architectures)

Load-store 架构中，ALU 操作没有内存地址，通常是三操作数寄存器指令；只有 load 和 store 指令访问内存：

```
load R1 <- C,  load R2 <- D,  mul R4 <- R1*R2,  add R5 <- R4+R3
```

- **优点**：简单固定长指令编码；各指令周期数相近；容易流水线化和超标量执行；三操作数格式简化寄存器分配。
- **缺点**：指令条数较多；并非所有指令都需要三个操作数；依赖优秀编译器。
- **例子**：CDC6600、CRAY-1、大多数 RISC。

6.7.9　寄存器的优缺点 (Registers: Advantages and Disadvantages)

寄存器的优势与代价

优势：

- 比缓存和主存更快，不需要复杂寻址方式或标签匹配。
- 访问确定，没有 cache miss。
- 可复制多读端口，支持多个操作数同时读取。
- 标识符短，通常只需 3 到 8 位。
- 减少内存流量。

代价：

- 过程调用和上下文切换时需要保存与恢复。
- 不能像内存变量那样直接“取寄存器地址”，因此不适合作为指针目标。
- 大小固定，不适合高效存储字符串或结构体。
- 必须由编译器管理。
- 数量有限。

课件最后强调：1980 年以后设计的每一种 ISA 都采用 load-store ISA，也就是 RISC 风格，以简化 CPU 设计。

6.8　复习抓手与补充知识 (Review Notes)

6.8.1　ISA 与微体系结构的区别

- **ISA** 是软件可见接口，回答“程序员能使用哪些指令、寄存器和内存模型”。
- **Microarchitecture** 是 ISA 的实现，回答“硬件内部如何取指、译码、执行、访存、提交”。
- 同一 ISA 可有多个实现；同一程序的机器码可在这些实现上运行，但性能、功耗和内部组织不同。

6.8.2　评估 ISA 的常用指标

复习时可用以下指标判断一个设计选择的影响：

- **Code Size**：程序占用的指令字节数，受指令长度、操作数个数、寻址方式影响。
- **Instruction Count**：完成同一任务需要多少条指令。
- **Cycles per Instruction (CPI)**：每条指令平均执行周期。
- **Clock Cycle Time**：硬件复杂度会影响时钟周期长度，例如复杂译码可能拖慢时钟。
- **Memory Traffic**：访问内存次数越多，越容易受缓存未命中和主存延迟影响。

- **Compiler Complexity**：简单硬件常把优化压力转移给编译器。

6.8.3　常见易混点

- **立即数不是地址**：立即寻址使用的是指令中写死的值；直接寻址使用的是该字段指向的内存内容。
- **间接寻址多取一层**：直接寻址是 $M[K]$，间接寻址是 $M[M[K]]$。
- **RISC 不等于指令永远少**：现代 ISA 会为了新应用加入复杂指令；CISC/RISC 是设计倾向，不是绝对二分。
- **Load-store 的重点不是“有 load/store 指令”**：关键是 ALU 运算只在寄存器间发生，访存只由 load/store 完成。
- **寄存器更快但不是免费资源**：数量有限，过程调用和上下文切换有保存恢复成本。

6.9　全章总结 (Summary)

本讲围绕“指令集体系结构如何定义软硬件接口”展开。复习时抓住以下主线：

1. **ISA 是契约**：它定义机器语言、寄存器、内存组织和异常模型，是高级语言经编译器、汇编器到硬件执行之间的关键接口。
2. **指令导致状态变化**：一条指令根据 opcode、操作数和寻址方式，把 PC、内存、寄存器等机器状态从 before state 变成 after state。
3. **设计必须折中**：指令短、规则、易译码有利于硬件；复杂指令和丰富寻址方式可能降低代码大小和编译难度，但增加硬件复杂性。
4. **操作数个数影响表达方式**：三地址灵活但字段多；二地址短但破坏源操作数；一地址依赖累加器；零地址依赖栈。
5. **寻址方式决定操作数定位**：立即、直接、间接、寄存器、相对、变址、基址等方式本质上都是计算有效地址的规则。
6. **CISC/RISC 是连续谱**：CISC 强调复杂指令和代码密度，RISC 强调简单规则、load-store、固定格式和流水线友好；现代处理器常混合两者思想。
7. **GPR 与 load-store 成为主流**：寄存器快、编号短、减少内存流量；1980 年后新 ISA 普遍采用 load-store 风格以简化 CPU 设计。

“*ISA 不是某个处理器内部怎么做，而是软件能够依赖硬件一定会做什么。*”

6.10　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. **[Extra]** The table below lists 9 addressing modes. Write the effective-address E formula for each (Disp = displacement, B = base register, I = index register, S = scale factor, PC = program counter): (1) immediate (2) register (3) direct (4) base (5) base + displacement (6) scaled index + displacement (7) base + index + displacement (8) base + scaled index + displacement (9) PC-relative.

2. **[Extra]** A machine has instruction format: X(2 bits) | OP(6 bits) | source register(3 bits) | destination register(3 bits) | address. There are 8 general-purpose registers (16 bits each) and a main memory of 256K words.
   (1) If each memory cell is accessed directly without going through a register and OP = 6 bits, how many bits should the address field be? What should the instruction word length be?
   (2) If X=11 designates a general-purpose register as a base register, give a hardware scheme that lets it access every cell of a 1M memory space.

3. **[Exam]** A 32-bit fixed-length ISA: 130 instructions, 64 general-purpose registers; each instruction has an opcode, one destination register *rd*, and one immediate *imm*, in a single word (4 bytes).
   (a) Minimum bits for the opcode? (b) Minimum bits for *rd*? (c) Maximum bits for *imm*? (d) If *imm* is an unsigned byte-address offset, what is the largest representable offset? (e) If *imm* is two's complement, what are the maximum and minimum representable values?

4. **[Extra]** Address = PC plus a constant in the instruction. Which addressing mode is this? A. register B. immediate C. PC-relative D. base/displacement.

5. **[Extra]** Modern computers are built on two key principles: ① instructions are represented as numbers; ② programs can be stored in memory like data; ③ make the common case fast; ④ every instruction can be executed conditionally. The correct ones are: A. ①③ B. ②④ C. ①② D. ③④.

6. **[Extra]** True or false: (1) "Good design requires no compromise." (2) A callee-saved register is "a register saved by the program that initiates the procedure call."

第七章　RISC-V 指令、汇编与机器码

本章考点

高频任务：寄存器约定、R/I/S/B/U/J 格式、立即数拼接与符号扩展、load/store、分支/跳转、C 与汇编互译、调用约定与栈帧。**复习路线**：机器码题先根据 opcode 判格式，再切字段；B/J 立即数必须按位号重排，不能直接连读。

7.1　RISC-V ISA 概览 (RISC-V ISA Overview)

7.1.1　什么是 RISC-V?

RISC-V 读作 “risk-five”，是一套开放的精简指令集计算机架构标准 (ISA, Instruction Set Architecture)。这里的 ISA 是软硬件之间的接口：软件看到的是寄存器、指令、异常和内存访问规则；硬件可以用不同的微结构去实现同一套规则。

- **RISC-V 是 ISA 标准**：它规定指令如何编码、寄存器如何命名、程序如何跳转和访问内存，而不是某一个固定芯片。
- **RISC-V 是开放 ISA**：任何个人、学校或公司都可以实现兼容的处理器和软件生态。
- **与常见商业 ISA 对比**：x86、ARM、Power、MIPS、SPARC 等通常受商业授权、专利或生态控制约束；RISC-V 设计目标之一是减少这些限制。
- **起源**：2010 年由 UC Berkeley 的 Krste Asanović、David Patterson 及学生发起。
- **规范版本**：课件提到非特权规范 Unprivileged Spec 20191213 (v2.2)，特权规范 Privileged Spec 20211203 (v1.11)。更多规范见

  `https://riscv.org/` 与 `https://github.com/riscv/riscv-isa-manual`。

ISA 与实现的关系

ISA 关心“程序员可见”的抽象，例如 `add x9,x20,x21` 的语义；微结构关心“如何实现”，例如流水线、乱序执行、缓存和分支预测。RISC-V 的目标是避免为某一种微结构或某一种工艺过度定制。

7.1.2　RISC-V 的设计目标与原则

RISC-V 在定义时追求以下目标：

- **完全开放**：对学术界和工业界免费开放。
- **真实可实现**：不是只用于模拟或二进制翻译，而是适合直接硬件实现。
- **不过度架构化**：不绑定微码、顺序、解耦、乱序等某一种实现风格，也不绑定 full-custom、ASIC、FPGA 等某一种技术。
- **小而可扩展**：基础整数 ISA 很小，可单独用于教学、研究和定制加速器；通用软件再按需加入标准扩展。
- **支持 IEEE 754-2008 浮点标准**：浮点扩展与现代科学计算需求配合。

RISC-V 的命名方式通常为：

$$\text{RV} + \text{字长} + \text{扩展集合}$$

例如 `RV32I` 表示 32 位 RISC-V，支持基础整数指令集 `I`；`RV64G` 表示 64 位 RISC-V，支持通用集合 `G`。

7.1.3　用户级 ISA 与标准扩展

用户级 ISA 定义普通计算所需的指令：

- **基础整数 ISA `I`**：ALU 运算、分支/跳转、Load/Store。
- `M`：整数乘法与除法。
- `A`：原子指令，用于并发同步。
- `F`：单精度浮点。
- `D`：双精度浮点。
- `C`：压缩指令，16 位长度。
- `G = IMAFD`：基础整数指令加四个常用标准扩展。
- **自定义扩展**：用于领域专用加速器或研究。

7.1.4　基础 RISC-V 与处理器状态

RISC-V 有 32 位和 64 位地址空间变体：

- **RV32**：32 位地址空间，寄存器宽度通常为 32 位。
- **RV64**：64 位地址空间，寄存器宽度通常为 64 位。
- 常见组合包括 `RV32IM`、`RV32IMA`、`RV32IMAFD`、`RV32G` 等。

RISC-V 处理器状态包括：

- **程序计数器 PC**：保存当前/下一条指令地址。
- **32 个整数寄存器 `x0`–`x31`**：
  - `x0` 永远为 0，写入会被丢弃。

– `x1` 常作为返回地址寄存器 `ra`。

- **32 个浮点寄存器 `f0`–`f31`**：可保存单精度或双精度 IEEE 浮点数。
- **浮点状态寄存器 `fsr`**：记录浮点舍入模式与异常信息。

7.1.5　RV32I 指令速览 (RV32I Instruction Quick Reference)

RV32I 是 32 位 RISC-V 的基础整数指令集，课件用四页表格先给出整体轮廓：整数计算、Load/Store、控制转移与少量杂项系统指令。这里先做一个总览，后续章节再展开编码、寻址和调用约定。

RV32I 基础类别

类别	典型指令族与含义
Integer Computation	`add/addi`、`sub`；`and/andi`、`or/ori`、`xor/xori`；`sll/slli`、`sra/srai`、`srl/srli`；`lui`，`auipc`；`slt/slti`、`sltu/sltiu`。
Loads and Stores	`lb/lh/lw` 与 `sb/sh/sw` 访问 byte、halfword、word；`lbu/lhu` 对 byte、halfword 做零扩展读取。RV64 还会加入 `ld/sd/lwu` 等 doubleword 相关指令。
Control Transfer	`beq/bne` 比较相等或不等；`blt/bge` 做有符号小于/大于等于判断；`bltu/bgeu` 做无符号比较；`jal` 与 `jalr` 完成跳转和函数调用/返回。
Miscellaneous	`fence` 约束 load/store 的内存可见顺序；`fence.i` 同步指令流与数据写入；`ecall/ebreak` 进入执行环境或调试断点；CSR 指令读写、置位或清位控制状态寄存器。

RV32I 的复习边界

RV32I 的核心是“寄存器计算 + 显式内存访问 + PC 相对控制流”。乘除法、原子操作和浮点运算不属于 RV32I 本体，而是分别放在 `M/A/F/D` 等扩展中。

ALU 指令示例

基础 ALU 指令既可使用寄存器-寄存器操作数，也可使用一个立即数操作数。课件表格中的典型例子如下：

Example instruction	Instruction name	Meaning
`add x1,x2,x3`	Add	$x1 \leftarrow x2 + x3$
`addi x1,x2,3`	Add immediate	$x1 \leftarrow x2 + 3$
`lui x1,42`	Load upper immediate	把 20 位立即数放入高位，低 12 位补 0；在 RV64 中结果还会按第 31 位向高位符号扩展。
`sll x1,x2,5`	Shift left logical	$x1 \leftarrow x2 \ll 5$
`slt x1,x2,x3`	Set less than	若 $x2 < x3$，则 $x1 = 1$，否则 $x1 = 0$。

补充：严格的基础 RISC-V 中，常数移位使用 I-Type 的 `slli/srli/srai`；`sll rd,rs1,rs2` 的移位量来自寄存器 `rs2` 的低位。许多教材表格会用 `sll x1,x2,5` 表达“左移 5 位”的语义，实际汇编时通常写作 `slli x1,x2,5`。

Load/Store 指令示例

所有内存引用都使用基址加位移形式 `imm(rs1)`。短于寄存器宽度的 Load 分为符号扩展和零扩展两类；Store 只把寄存器低位写回内存。

Example instruction	Instruction name	Meaning
`ld x1,80(x2)`	Load doubleword	`x1` ← Mem[80 + `x2`]。RV64 指令。
`lw x1,60(x2)`	Load word	读取 32 位 word，并符号扩展到寄存器宽度。
`lwu x1,60(x2)`	Load word unsigned	读取 32 位 word，并零扩展到 64 位。RV64 指令。
`lb x1,40(x3)`	Load byte	读取 8 位 byte，并符号扩展。
`lbu x1,40(x3)`	Load byte unsigned	读取 8 位 byte，并零扩展。
`lh x1,40(x3)`	Load halfword	读取 16 位 halfword，并符号扩展。
`flw f0,50(x3)`	Load FP single	读取单精度浮点到浮点寄存器。
`fld f0,50(x2)`	Load FP double	读取双精度浮点到浮点寄存器。
`sd x2,400(x3)`	Store doubleword	把 `x2` 的 64 位写入内存。RV64 指令。
`sw x3,500(x4)`	Store word	把 `x3` 的低 32 位写入内存。
`fsw f0,40(x3)`	Store FP single	把浮点寄存器中的单精度值写入内存。
`fsd f0,40(x3)`	Store FP double	把浮点寄存器中的双精度值写入内存。
`sh x3,502(x2)`	Store halfword	把 `x3` 的低 16 位写入内存。
`sb x2,41(x3)`	Store byte	把 `x2` 的低 8 位写入内存。

注意：RV32I 本体只包含整数寄存器和 32 位地址空间中的基础 Load/Store；上表中的 `ld/sd/lwu` 属于 RV64，`flw/fld/fsw/fsd` 属于浮点扩展。课件把它们放在同一张表中，是为了展示 RISC-V Load/Store 家族的统一寻址风格。

控制转移指令示例

Example instruction	Instruction name	Meaning
`jal x1,offset`	Jump and link	`x1` ← `PC` + 4，`PC` ← `PC` + (`offset` ≪ 1)。
`jalr x1,x2,offset`	Jump and link register	`x1` ← `PC` + 4，`PC` ← `x2` + `offset`。
`beq x3,x4,offset`	Branch equal	若 `x3` = `x4`，则 `PC` ← `PC`+(`offset` ≪ 1)。
`bgt x3,x4,name`	Branch greater than	若 `x3` > `x4`，则跳转到 `name`。这是常见伪指令，通常可由 `blt x4,x3,name` 展开。

除了 `jalr` 这种寄存器间接跳转外，RISC-V 控制流指令通常都是 PC-relative，因此目标位置随程序整体搬移而自然改变，便于生成位置无关代码。

7.1.6　动态指令混合 (Dynamic Instruction Mix)

SPECint2006 中 RISC-V 动态指令大致呈现如下分布。该表的意义是帮助理解为什么 Load/Store、分支和 ALU 是基础 ISA 的核心。

Program	Loads	Stores	Branches	Jumps	ALU ops
astar	28%	6%	18%	2%	46%
bzip	20%	7%	11%	1%	54%
gcc	17%	23%	20%	4%	36%
gobmk	21%	12%	14%	2%	50%
h264ref	33%	14%	5%	2%	45%
hmmer	28%	9%	17%	0%	46%
libquantum	16%	6%	29%	0%	48%
mcf	35%	11%	24%	1%	29%
omnetpp	23%	15%	17%	7%	31%
perlbench	25%	14%	15%	7%	39%
sjeng	19%	7%	15%	3%	56%
xalancbmk	30%	8%	27%	3%	31%

典型整数程序中，Load 频率常为 Store 频率的 1/5 到 3 倍；`gcc` 中 Store 比 Load 更高，是课件指出的一个异常现象。

7.2　指令编码与格式 (Instruction Encoding and Formats)

7.2.1　混合长度编码

RISC-V 支持 16、32、48、64 位以及更长的混合长度指令编码。基础 RV32 指令固定为 32 位，并且最低两位为 11_2。压缩扩展 `C` 使用 16 位指令，因此分支和跳转目标统一按 16 位粒度对齐，即使在只使用 32 位基础指令时也如此。

- 16 位指令：低 2 位不为 11_2。
- 32 位指令：低 2 位为 11_2，但低 3 位不全为 1。
- 48/64 位及更长指令：用特定低位模式继续扩展。

7.2.2　核心字段位置

RISC-V 指令编码的一个关键设计是：寄存器字段位置尽量固定，立即数符号位总在指令第 31 位。

- **`opcode`**：低 7 位，决定指令大类；基础 32 位指令中低 2 位为 11_2。
- **`rd`**：目的寄存器。
- **`rs1`, `rs2`**：源寄存器。
- **`funct3`, `funct7`**：在同一 opcode 大类下进一步区分具体运算。
- **立即数字段**：为保持寄存器字段固定，S/B/J 类型的立即数会被拆开重排。

7.2.3　六类常用指令格式

格式	31..25	24..20	19..15	14..12	11..7	6..0	用途
R-type	funct7	rs2	rs1	funct3	rd	opcode	寄存器-寄存器算术/逻辑
I-type	imm[11:0]		rs1	funct3	rd	opcode	Load、立即数算术、jalr
S-type	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	Store
B-type	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode	条件分支
U-type	imm[31:12]				rd	opcode	高 20 位立即数
J-type	imm[20,10:1,11,19:12]				rd	opcode	无条件跳转 jal

编码复习口诀

寄存器字段 rd/rs1/rs2 的位置尽量不变；立即数为适配硬件与对齐规则被拆散；所有立即数的符号位放在 inst[31]，便于统一符号扩展。

7.2.4　立即数编码变体

I、S、B、U、J 格式都含立即数，但使用方式不同：

- **I-immediate**：$\{\texttt{inst[31]}\ 扩展, \texttt{inst[30:20]}\}$，常用于 addi、Load、jalr。
- **S-immediate**：$\{\texttt{inst[31]}\ 扩展, \texttt{inst[30:25]}, \texttt{inst[11:7]}\}$，用于 Store。
- **B-immediate**：$\{\texttt{inst[31]}\ 扩展, \texttt{inst[7]}, \texttt{inst[30:25]}, \texttt{inst[11:8]}, 0\}$，最低位恒为 0，表示按 2 字节对齐的分支偏移。
- **U-immediate**：$\{\texttt{inst[31:12]}, 12'b0\}$，用于构造高位立即数。
- **J-immediate**：$\{\texttt{inst[31]}\ 扩展, \texttt{inst[19:12]}, \texttt{inst[20]}, \texttt{inst[30:21]}, 0\}$，用于 jal 的 PC 相对跳转。

7.2.5　RISC-V 寻址方式

课件总结了四类常见寻址方式：

1. **立即数寻址 (Immediate Addressing)**：操作数直接来自指令中的立即数字段，例如 addi x1,x2,3。
2. **寄存器寻址 (Register Addressing)**：操作数来自寄存器，例如 add x1,x2,x3。
3. **基址/位移寻址 (Base/Displacement Addressing)**：有效地址为 rs1 + imm，Load/Store 使用这一种内存寻址方式。
4. **PC 相对寻址 (PC-relative Addressing)**：目标地址为 PC + offset，用于分支、jal、auipc 等。

7.3　RV32I 指令系统 (RV32I Instruction Set)

7.3.1　RV32I 指令类别总览

RV32I 的基础指令可以按功能分为：

- **整数计算**：`add/addi`、`sub`、`and/andi`、`or/ori`、`xor/xori`、`sll/slli`、`srl/srli`、`sra/srai`、`slt/slti`、`sltu/sltiu`、`lui`、`auipc`。
- **控制转移**：`beq`、`bne`、`blt`、`bge`、`bltu`、`bgeu`、`jal`、`jalr`。
- **Load/Store**：`lb/lh/lw/lbu/lhu` 与 `sb/sh/sw`；RV64 还包括 `ld/sd/lwu`。
- **杂项与系统**：`fence`、`fence.i`、`ecall`、`ebreak`、CSR 读写/置位/清位及其立即数形式。

7.3.2　ALU 指令：R-Type

R-Type 指令使用两个源寄存器和一个目的寄存器：

$$\texttt{rd} \leftarrow \texttt{rs1}\ \text{op}\ \texttt{rs2}$$

典型指令包括：

- **`add/sub`**：整数加法与减法。
- **`slt/sltu`**：若 `rs1` $<$ `rs2` 则 `rd=1`，否则 `rd=0`；`slt` 按有符号比较，`sltu` 按无符号比较。
- **`sll/srl/sra`**：逻辑左移、逻辑右移、算术右移。
- **`and/or/xor`**：按位与、或、异或。

机器码例子：`add x9,x20,x21`

RISC-V 汇编：

```
add x9, x20, x21
```

字段十进制形式：

$$|\ 0\ |\ 21\ |\ 20\ |\ 0\ |\ 9\ |\ 51\ |$$

字段二进制形式：

$$|\ 0000000\ |\ 10101\ |\ 10100\ |\ 000\ |\ 01001\ |\ 0110011\ |$$

完整机器码：

$$0000\ 0001\ 0101\ 1010\ 0000\ 0100\ 1011\ 0011_2 = \texttt{0x015A04B3}$$

7.3.3　按位逻辑与移位

`and`：掩码操作

`and` 常用于选择某些位并把其他位清零：

```
and x9, x10, x11
```

只有两个操作数同一位都为 1 时，结果对应位才为 1。

xor：差异与取反

xor 可用于翻转部分位。当掩码全 1 时，异或相当于按位取反：

```
xor x9, x10, x12     # if x12 = -1, x9 = ~x10
```

移位运算

- **逻辑左移 sll/slli**：低位补 0。左移 i 位相当于乘以 2^i（不考虑溢出）。
- **逻辑右移 srl/srli**：高位补 0，适合无符号数。
- **算术右移 sra/srai**：高位补符号位，适合有符号数。

  例：$9_{10} = 0000\cdots1001_2$，左移 4 位得到 $0000\cdots1001\,0000_2 = 144_{10}$：

```
slli x11, x19, 4     # x11 = x19 << 4
```

7.3.4　ALU 指令：I-Type

I-Type 使用一个源寄存器和一个 12 位立即数。所有 RISC-V 指令中的立即数都会按规则进行符号扩展。

- **addi**：rd = rs1 + sign-extended imm。
- **slti/sltiu**：与立即数比较并设置 0/1。
- **andi/ori/xori**：逻辑立即数运算。
- **slli/srli/srai**：按常数位数移位，移位量来自立即数字段的低位。

7.3.5　ALU 指令：U-Type 与大立即数

lui 与 auipc 使用 U-Type，用 20 位立即数构造高位：

- **lui rd,imm20**：把 20 位立即数写入目的寄存器高 20 位，低 12 位清 0。RV64 中还会把第 31 位符号扩展到高 32 位。
- **auipc rd,imm20**：rd = PC + (imm20 << 12)，常用于 PC 相对地址生成。

  由于 I-Type 立即数只有 12 位且会符号扩展，加载 32 位常量常用两条指令完成：

```
lui  s3, 976
addi s3, s3, 2304
```

其中 lui 生成高 20 位，addi 加入低 12 位。若低 12 位最高位为 1，要注意 addi 的符号扩展可能影响高位，实际汇编器会在伪指令 li 中处理这些细节。

7.4　内存访问、字节序与对齐 (Memory Access, Endianness and Alignment)

7.4.1　Load/Store 架构

RISC-V 是典型 Load/Store 架构：算术逻辑运算只能在寄存器之间进行，内存访问由专门的 Load/Store 指令完成。

$$\text{Load:}\quad \texttt{rd} \leftarrow \text{MEM}[\texttt{rs1} + \texttt{imm}]$$

$$\text{Store:}\quad \text{MEM}[\texttt{rs1} + \texttt{imm}] \leftarrow \texttt{rs2}$$

- Load 使用 I-Type。
- Store 使用 S-Type，立即数被拆成 `imm[11:5]` 与 `imm[4:0]` 两段。
- RISC-V 使用字节地址，连续 doubleword 地址相差 8。

7.4.2　常用 Load/Store 指令

类别	指令	含义
Load	`lb/lh/lw`	读取 byte/halfword/word，并按符号扩展到寄存器宽度。
Load unsigned	`lbu/lhu`	读取 byte/halfword，并用 0 扩展。RV64 还有 `lwu`。
RV64 Load	`ld`	读取 doubleword，64 位。
Store	`sb/sh/sw`	将寄存器低 8/16/32 位写入内存。
RV64 Store	`sd`	将寄存器 64 位写入内存。
浮点 Load/Store	`flw/fld/fsw/fsd`	读取或写入单精度/双精度浮点寄存器。

例子：

```
ld x9, 64(x22)      # x9 = MEM[x22 + 64]
sd x9, 64(x22)      # MEM[x22 + 64] = x9
```

7.4.3　字节序 (Endianness)

当一个多字节数据放入内存时，字节序决定高低字节与地址大小的对应关系。

- **大端序 (Big Endian)**：高字节放在低地址，低字节放在高地址。PowerPC 是课件示例之一。
- **小端序 (Little Endian)**：低字节放在低地址，高字节放在高地址。RISC-V 通常采用小端序。

以 32 位数 `0x12345678` 从地址 `0x100` 开始存放为例：

字节序	0x100	0x101	0x102	0x103
Big Endian	12	34	56	78
Little Endian	78	56	34	12

7.4.4　内存对齐 (Memory Alignment)

对齐约束要求某些数据必须放在特定边界上，例如 4 字节整数通常放在 4 字节边界，8 字节 doubleword 通常放在 8 字节边界。

- 对齐访问通常更快，硬件一次内存访问即可读出完整数据。
- 未对齐访问可能需要多次内存访问，甚至在某些实现上触发异常。
- 编译器会在结构体成员之间插入填充字节 (padding)，以满足对齐要求。

课件中的结构体例子：

```
struct {
    int a;
    char b;
    char c[2];
    char d[3];
    float e;
};
```

若不考虑对齐，`float e` 可能跨越自然边界，无法一次读出；加入 padding 后可保证 `e` 对齐，但会牺牲部分空间。复习时要能判断“哪些空洞是为了对齐而插入的”。

7.4.5　数组寻址例：`g = h + A[i]`

假设 `A` 是包含 100 个 doubleword 的数组，`g,h,i` 分别在 `x1,x2,x4`，数组基地址在 `x3`。由于每个 doubleword 为 8 字节，需要先计算 $8i$：

```
add x5, x4, x4     # x5 = 2 * i
add x5, x5, x5     # x5 = 4 * i
add x5, x5, x5     # x5 = 8 * i
add x5, x5, x3     # x5 = &A[i]
ld  x6, 0(x5)      # x6 = A[i]
add x1, x2, x6     # g = h + A[i]
```

补充：也可用 `slli x5,x4,3` 更直接地计算 $8i$。

7.5　控制转移指令 (Control Transfer Instructions)

7.5.1　跳转与分支的共同特点

RISC-V 控制转移指令没有架构可见的延迟槽 (delay slot)。除 `jalr` 使用寄存器间接跳转外，分支和 `jal` 通常使用 PC 相对寻址：

$$\text{target} = \texttt{PC} + \text{offset}$$

由于目标按 16 位粒度对齐，B-Type 和 J-Type 立即数最低位隐含为 0。

7.5.2　jal 与 jalr

- **jal rd,label**：保存返回地址 PC + 4 到 rd，然后跳转到 PC 相对目标。函数调用常用 jal x1,ProcedureLabel。
- **jalr rd,imm(rs1)**：保存返回地址 PC + 4 到 rd，目标地址为 rs1 + imm。返回常用 jalr x0,0(x1)。
- **丢弃返回地址**：若 rd=x0，返回地址被丢弃，可用于无条件跳转。

常见伪指令：

```
j label       # jal x0, label
ret           # jalr x0, 0(x1)
```

7.5.3　条件分支 B-Type

条件分支比较两个寄存器，根据条件决定是否跳转：

指令	含义
beq rs1,rs2,L	相等则跳转
bne rs1,rs2,L	不等则跳转
blt rs1,rs2,L	有符号 $rs1 < rs2$ 则跳转
bge rs1,rs2,L	有符号 $rs1 \geq rs2$ 则跳转
bltu rs1,rs2,L	无符号 $rs1 < rs2$ 则跳转
bgeu rs1,rs2,L	无符号 $rs1 \geq rs2$ 则跳转

课件例子：bne x10,x11,2000 中，偏移字段编码的是以半字为单位的 PC 相对偏移。若目标地址差为 12 字节，编码字段应表示 $12/2 = 6$，不是 12。

7.5.4　编译 if 语句

C 代码：

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

假设 f 到 j 分别在 x19 到 x23：

```
bne x22, x23, ELSE
add x19, x20, x21
beq x0, x0, EXIT
ELSE:
sub x19, x20, x21
EXIT:
```

其中 beq x0,x0,EXIT 是永真分支，也可写成伪指令 j EXIT。

7.5.5　更多条件与边界检查技巧

- `if (a > b) a += 1;` 可改写为先测试退出条件：

```
bge  x23, x22, Exit     # if b >= a, skip
addi x22, x22, 1
Exit:
```

边界检查快捷写法：

```
bgeu x20, x11, IndexOutofBounds
```

它能同时捕捉 `x20` $\geq$ `x11` 和 `x20` < 0。原因是负数按无符号数解释会变成很大的数。

7.5.6　编译 while 循环

C 代码：

```
while (save[i] == k)
    i = i + 1;
```

假设 `i,k` 在 `x22,x24`，数组 `save` 基地址在 `x25`：

```
Loop:
    slli x10, x22, 3
    add  x10, x10, x25
    ld   x9, 0(x10)
    bne  x9, x24, Exit
    addi x22, x22, 1
    beq  x0, x0, Loop
Exit:
```

7.5.7　远距离分支

若条件分支目标太远，超出 B-Type 可编码范围，可以反转条件，让短分支跳过一个长跳转：

```
# original
beq x10, x0, L1

# rewritten
bne x10, x0, L2
jal x0, L1
L2:
```

7.5.8　switch 与跳转表

对于密集的 `case`，编译器可使用跳转地址表。假设 `k` 在 `x25`，`x5` 中有 4，跳转表基地址在 `x6`：

```
blt  x25, x0, Exit
bge  x25, x5, Exit
slli x7, x25, 3
```

```
add  x7, x7, x6
ld   x7, 0(x7)
jalr x1, 0(x7)
Exit:
```

每个 case 代码段执行完后可用 `jalr x0,0(x1)` 返回到 switch 之后。这里 `jalr` 也体现了它可用于 computed jump。

7.5.9　NOP 与机器码解码

RISC-V 的 NOP 通常是：

```
addi x0, x0, 0
```

它不改变任何可见状态，因为 `x0` 恒为 0 且写入无效。

机器码解码例：`0x00578833` 的二进制为：

$$0000\ 0000\ 0101\ 0111\ 1000\ 1000\ 0011\ 0011$$

低 7 位 `0110011` 表示 R-Type 算术指令；`funct7=0000000` 且 `funct3=000` 表示 `add`；字段为 `rs2=x5`、`rs1=x15`、`rd=x16`，因此：

```
add x16, x15, x5
```

7.5.10　立即数编码变体 (Immediate Encoding Variants)

RISC-V 为了让寄存器字段位置尽量固定，把不同格式的立即数拆分到不同指令位中。复习时不要只背“格式图”，还要理解两组关键对比：**S-Type vs. B-Type** 和 **U-Type vs. J-Type**。

指令格式中的立即数字段

格式	31..25	24..20	19..15	14..12	11..7	6..0	用途
R-type	funct7	rs2	rs1	funct3	rd	opcode	寄存器算术/逻辑
I-type	imm[11:0]		rs1	funct3	rd	opcode	Load、立即数运算、jalr
S-type	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	Store
B-type	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode	条件分支
U-type	imm[31:12]				rd	opcode	lui/auipc
J-type	imm[20,10:1,11,19:12]				rd	opcode	jal

从指令位恢复真实立即数

硬件解码时，会从指令位 `inst[31:0]` 中重新拼接立即数：

立即数类型	拼接方式
I-immediate	{inst[31] 符号扩展, inst[30:20]}
S-immediate	{inst[31] 符号扩展, inst[30:25], inst[11:8], inst[7]}
B-immediate	{inst[31] 符号扩展, inst[7], inst[30:25], inst[11:8], 0}
U-immediate	{inst[31], inst[30:20], inst[19:12], $12'b0$}
J-immediate	{inst[31] 符号扩展, inst[19:12], inst[20], inst[30:25], inst[24:21], 0}

S-Type vs. B-Type

S-Type 与 B-Type 都使用 12 位立即数字段，并且都保留 `rs1`、`rs2`、`funct3`、`opcode` 的位置。区别在于：

- **S-Type** 用于 Store，立即数表示普通字节偏移：

$$\texttt{imm[11:0]} = \{\texttt{inst[31:25]}, \texttt{inst[11:7]}\}$$

- **B-Type** 用于分支，分支目标按 2 字节粒度对齐，因此最低位恒为 0：

$$\texttt{offset} = \{\texttt{inst[31]}, \texttt{inst[7]}, \texttt{inst[30:25]}, \texttt{inst[11:8]}, 0\}$$

- 这样设计避免在硬件中把整个立即数字段统一左移 1 位；中间位 `imm[10:1]` 和符号位保持在相近位置，而 S-Type 的最低字段位 `inst[7]` 在 B-Type 中变成较高位 `imm[11]`。

U-Type vs. J-Type

U-Type 与 J-Type 都含 20 位立即数，但用途不同：

- **U-Type** 用于构造高位立即数，真实值为：

$$\texttt{imm[31:12]} << 12$$

即低 12 位补 0，常见于 `lui` 和 `auipc`。

- **J-Type** 用于 `jal`，真实跳转偏移为：

$$\{\texttt{imm[20]}, \texttt{imm[10:1]}, \texttt{imm[11]}, \texttt{imm[19:12]}, 0\}$$

即左移 1 位，目标仍按 2 字节边界对齐。

- J-Type 的立即数位顺序看起来“跳跃”，是为了最大化与 U-Type 以及其他格式的重叠，降低解码和布线复杂度。

易错点：编码值不等于字节偏移

分支和跳转立即数的最低位隐含为 0，因此机器码中保存的是按 2 字节粒度重排后的偏移信息。做题时要先判断目标指令地址与当前 PC 的差值，再考虑是否需要除以 2 以及如何拆分到 `inst[31]`、`inst[7]`、`inst[30:25]`、`inst[11:8]` 等位置。

7.6　指令编码速查 (Encoding Reference)

7.6.1　常用 R-Type 与 I-Type 编码

格式	指令	Opcode	Funct3	Funct6/7
R	`add`	0110011	000	0000000
R	`sub`	0110011	000	0100000
R	`sll`	0110011	001	0000000
R	`xor`	0110011	100	0000000
R	`srl`	0110011	101	0000000
R	`sra`	0110011	101	0100000
R	`or`	0110011	110	0000000
R	`and`	0110011	111	0000000
R/A	`lr.d`	0110011	011	0001000
R/A	`sc.d`	0110011	011	0001100
I	`addi`	0010011	000	n.a.
I	`slli`	0010011	001	000000
I	`xori`	0010011	100	n.a.
I	`srli`	0010011	101	000000
I	`srai`	0010011	101	010000
I	`ori`	0010011	110	n.a.
I	`andi`	0010011	111	n.a.
I	`jalr`	1100111	000	n.a.

7.6.2　Load/Store、Branch、U/J-Type 编码

格式	指令	Opcode	Funct3	Funct6/7
I	`lb`	0000011	000	n.a.
I	`lh`	0000011	001	n.a.
I	`lw`	0000011	010	n.a.
I	`ld`	0000011	011	n.a.
I	`lbu`	0000011	100	n.a.
I	`lhu`	0000011	101	n.a.
I	`lwu`	0000011	110	n.a.
S	`sb`	0100011	000	n.a.
S	`sh`	0100011	001	n.a.
S	`sw`	0100011	010	n.a.
S	`sd`	0100011	111	n.a.
B	`beq`	1100011	000	n.a.
B	`bne`	1100011	001	n.a.
B	`blt`	1100011	100	n.a.
B	`bge`	1100011	101	n.a.
B	`bltu`	1100011	110	n.a.
B	`bgeu`	1100011	111	n.a.
U	`lui`	0110111	n.a.	n.a.
J	`jal`	1101111	n.a.	n.a.

7.7　特权 ISA 与软件栈 (Privileged ISA and Software Stack)

7.7.1　特权模式

RISC-V 特权规范定义了三个主要特权级别，Machine mode 是最高特权级，也是唯一必需模式。

Level	Encoding	Name	Abbrev.
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	–
3	11	Machine	M

高特权模式通常能访问低特权模式的功能，并额外支持中断处理、异常处理和 I/O 等能力。处理器大部分时间运行在最低可用特权级；中断和异常会把控制转交给更高特权级。

7.7.2　实现层次与特权软件栈

不同实现可提供 1 到 4 个特权层次，以隔离能力换取实现成本：

层数	支持模式	用途
1	M	简单嵌入式系统
2	M, U	安全嵌入式系统
3	M, S, U	类 Unix 操作系统

课件中的软件栈包含 Application、ABI、AEE、OS、SBI、SEE、Hypervisor、HBI、HEE 等层次。需要理解的是：越靠下越接近硬件和特权资源，越靠上越接近应用程序。

常见特权指令包括：

- Machine/Supervisor trap return：从陷入处理返回。
- Supervisor-mode fence virtual memory address：虚拟内存相关 fence。
- Wait for interrupt：等待中断。

7.8　汇编语言与调用约定 (Assembly Language and Calling Convention)

7.8.1　函数调用角色

函数调用中有两个角色：

- **Caller**：调用者，例如 `main` 调用 `sum` 时，`main` 是 caller。它负责传参并跳转到被调函数。
- **Callee**：被调用者，例如 `sum`。它完成任务，把返回值放到约定位置，然后返回调用点。

7.8.2　函数调用六步

RISC-V 函数调用通常可分为六步：

1. 把参数放到 callee 能访问的位置，通常是 `x10`–`x17`。
2. 用 `jal` 跳转到函数，并把返回地址放入 `x1`。
3. 在函数入口分配局部存储，必要时保存寄存器。
4. 执行函数主体。
5. 把返回值放到 caller 能访问的位置，通常是 `x10`–`x17`，恢复寄存器并释放局部存储。
6. 用 `ret` 或 `jalr x0,0(x1)` 返回调用点。

7.8.3　函数入口与出口

- **Prologue**：函数入口代码。若参数和局部变量太多，或需要保护寄存器，就在栈上创建 stack frame。
- **Epilogue**：函数出口代码。撤销栈帧、恢复寄存器并返回。

调用与返回的核心指令：

```
jal  x1, ProcedureLabel    # x1 = PC + 4; PC = target
jalr x0, 0(x1)             # return; x0 discards link
```

7.8.4　寄存器约定

名字	编号	用途	调用后保持?
x0(zero)	0	常数 0	n.a.
x1(ra)	1	返回地址 / link register	需维护返回路径
x2(sp)	2	栈指针	yes
x3(gp)	3	全局指针	yes
x4(tp)	4	线程指针	yes
x5-x7(t0-t2)	5–7	临时寄存器	no
x8(s0/fp)	8	保存寄存器 / 帧指针	yes
x9(s1)	9	保存寄存器	yes
x10-x17(a0-a7)	10–17	参数 / 返回值	no
x18-x27(s2-s11)	18–27	保存寄存器	yes
x28-x31(t3-t6)	28–31	临时寄存器	no

Saved 与 Temporary 的责任

`temporary` 寄存器可被 callee 随意使用，返回后 caller 不能假设其值还在。`saved` 寄存器若被 callee 使用，callee 必须保存并恢复。非叶函数若会再次调用其他函数，也必须保护自己的返回地址 `ra`，因为新的 `jal` 会覆盖它。

7.8.5　栈与栈指针

栈适合保存溢出的寄存器、返回地址和局部变量。

- 栈指针 `sp` 为 `x2`。
- RISC-V 栈从高地址向低地址增长。
- Push 8 字节：`sp = sp - 8`。
- Pop 8 字节：`sp = sp + 8`。

7.8.6　叶函数示例

C 代码：

```
long long leaf_example(long long g, long long h,
                       long long i, long long j) {
    long long f;
    f = (g + h) - (i + j);
    return f;
}
```

参数 `g,h,i,j` 在 `x10,x11,x12,x13`，返回值放入 `x10`。课件示例展示了把寄存器压栈、计算并恢复：

```
addi sp, sp, -24
sd   x5, 16(sp)
sd   x6, 8(sp)
sd   x20, 0(sp)
add  x5, x10, x11      # x5 = g + h
add  x6, x12, x13      # x6 = i + j
sub  x20, x5, x6       # f = x5 - x6
addi x10, x20, 0       # return value
ld   x20, 0(sp)
ld   x6, 8(sp)
ld   x5, 16(sp)
addi sp, sp, 24
jalr x0, 0(x1)
```

补充：按常见 ABI，`x5/x6` 是临时寄存器，callee 通常不必为 caller 保存；`x20` 是 saved register，使用后必须恢复。课件示例更强调“保存现场”的栈操作过程。

7.8.7　非叶函数与递归

非叶函数会调用其他函数，因此需要在栈上保存：

- 自己的返回地址 `x1/ra`；
- 调用后还要继续使用的参数和临时值；
- 被使用的 saved registers。

递归阶乘例子：

```
long long fact(long long n) {
    if (n < 1) return 1;
    else return n * fact(n - 1);
}
```

核心汇编结构：

```
fact:
    addi sp, sp, -16
    sd   x1, 8(sp)
    sd   x10, 0(sp)
    addi x5, x10, -1
    bge  x5, x0, L1
    addi x10, x0, 1
    addi sp, sp, 16
    jalr x0, 0(x1)
L1:
    addi x10, x10, -1
    jal  x1, fact
    add  x6, x10, x0
    ld   x10, 0(sp)
    ld   x1, 8(sp)
    addi sp, sp, 16
    mul  x10, x10, x6
    jalr x0, 0(x1)
```

- 保存 `x10`：递归调用会把返回值也放到 `x10`，原来的 n 会丢失。
- 保存 `x1`：递归调用中的 `jal x1,fact` 会覆盖当前函数返回到 caller 的地址。
- 基例 `n<1` 返回 1；递归例先调用 `fact(n-1)`，再乘以保存的 n。

7.8.8　递归的缺点

- 每层递归都要保存处理器状态，可能造成栈溢出 (stack overflow)。
- Push/Pop 消耗时间和内存空间。
- 可用循环或尾调用优化减少递归开销。

7.8.9　调用前后保持与不保持的内容

- **保持**：saved registers、`sp`、返回路径所需的 `ra`、栈指针以上的栈内容。
- **不保持**：temporary registers、参数寄存器 `x10`–`x17`、返回值寄存器 `x10`–`x17`、栈指针以下的区域。

7.8.10　C 变量存储类别与内存布局

C 变量常见存储类别：

- **automatic**：自动变量，通常在栈帧中。
- **static**：静态变量，通常在静态数据区。

内存布局自低地址到高地址通常为：

区域	内容
Reserved	低地址保留区。
Text	程序代码；PC 指向此区域中的指令。
Static data	全局变量、`static` 变量、常量数组和字符串；`gp(x3)` 初始化为可用正负偏移访问该区域的地址。
Dynamic data / heap	动态分配区，如 C 的 `malloc`、Java 的 `new`。
Stack	自动存储区，从高地址向低地址增长；`sp` 指向当前栈顶。
OS	用户地址空间顶部附近可能由操作系统保留。

7.8.11　栈帧 (Stack Frame)

栈帧是函数入口分配、函数退出释放的私有空间，也称 activation record。课件栈帧包含：

- 返回地址 (Return Address)。
- 旧帧指针 (Old FP)。
- 保存寄存器 (Saved Registers)。
- 局部变量 (Local Variables)。
- 传给被调函数的额外参数。

`fp(x8)` 可作为稳定基准访问栈帧内容，`sp` 随压栈/出栈变化。栈溢出与安全攻击密切相关，课件提到经典文章 “Smashing The Stack For Fun And Profit”。

7.9　从源代码到运行程序 (From Source Code to A Running Program)

7.9.1　整体流程

C 程序变为正在运行的进程，逻辑上经过以下步骤：

$$\texttt{foo.c} \xrightarrow{\text{Compiler}} \texttt{foo.s} \xrightarrow{\text{Assembler}} \texttt{foo.o} \xrightarrow{\text{Linker + libraries}} \texttt{a.out} \xrightarrow{\text{Loader}} \text{running process}$$

实际系统中某些步骤可能被合并以加快翻译，但理解这条链路有助于定位编译、链接和运行时错误。

7.9.2　编译器与预处理器

- **Preprocessor** `*.c` → `*.i`：展开宏定义、处理 `#include` 和其他以 `#` 开头的指令。
- **Compiler** `*.i` → `*.s`：把预处理后的 C 程序转换为汇编语言。

  课件中说明编译器内部细节不在本课展开。

7.9.3　汇编器与伪指令

汇编器不仅把处理器能直接执行的指令翻译成机器码，还支持对程序员和编译器更方便的伪指令 (pseudo-instructions)。伪指令会被展开为一条或多条基础指令。

伪指令	可能展开	含义
`lla rd,symbol`	`auipc rd,symbol[31:12]` + `addi rd,rd,symbol[11:0]`	加载本地地址
`la rd,symbol`	PIC: 经 GOT 加载；Non-PIC: 类似 `lla`	加载地址
`li rd,imm`	若立即数较大，展开为多条指令	加载立即数
`mv rd,rs`	`addi rd,rs,0`	复制寄存器
`not rd,rs`	`xori rd,rs,-1`	按位取反
`seqz rd,rs`	`sltiu rd,rs,1`	等于 0 则置 1
`bgt rs,rt,L`	`blt rt,rs,L`	大于则分支
`ble rs,rt,L`	`bge rt,rs,L`	小于等于则分支
`j L`	`jal x0,L`	无条件跳转
`ret`	`jalr x0,0(x1)`	函数返回
`call offset`	`auipc x1,offset[31:12]` + `jalr x1,x1,offset[11:0]`	远距离函数调用

7.9.4　汇编指示符 (Assembler Directives)

汇编指示符指导汇编器组织目标文件，但本身不是处理器执行的指令。

- `.text`：进入代码段。
- `.align 2`：把后续代码对齐到 2^2 字节。
- `.globl main`：声明全局符号 `main`。
- `.section .rodata`：进入只读数据段。
- `.balign 4`：把数据段对齐到 4 字节。
- `.string "Hello, %s!\n"`：创建以 `NUL` 结尾的字符串。

7.9.5　Hello World 汇编结构

C 代码：

```
#include <stdio.h>
int main() {
    printf("Hello, %s!\n", "world");
    return 0;
}
```

对应汇编框架包含：

```
.text
.align 2
.globl main
main:
    addi sp, sp, -16
    sw   ra, 12(sp)
    lui  a0, %hi(string1)
    addi a0, a0, %lo(string1)
    lui  a1, %hi(string2)
    addi a1, a1, %lo(string2)
    call printf
    lw   ra, 12(sp)
    addi sp, sp, 16
    li   a0, 0
    ret

.section .rodata
.balign 4
string1:
    .string "Hello, %s!\n"
string2:
    .string "world"
```

汇编器输出 `*.o` 时采用 ELF 格式。目标文件中会出现机器码、符号表、重定位信息等。

7.10　ELF、链接与装载 (ELF, Linking and Loading)

7.10.1　ELF 基础

ELF (Executable and Linkable Format) 是 Unix 和类 Unix 系统中常见的可执行文件、目标文件、共享库与 core dump 的标准格式。它灵活、可扩展、跨平台。

ELF 文件有两种视图：

- **Section view**：面向链接，section header 列出各 section，例如 `.text`、`.data`、`.rodata`、符号表、重定位信息等。
- **Segment view**：面向运行，program header 描述装载时使用的 segment。

7.10.2　三类目标文件

1. **Relocatable file**：包含适合与其他目标文件链接的代码和数据，例如 Linux 的 `.o`。

2. **Executable file**：可执行程序，描述操作系统如何创建进程映像。
3. **Shared object file**：共享对象，可参与静态链接生成其他目标文件，也可在运行时由动态链接器装入进程。

7.10.3　可重定位目标文件

汇编器输出的可重定位目标文件具有以下特点：

- 是二进制机器码，但还不能直接执行。
- 可能引用外部符号，因此需要符号表。
- 每个目标文件有自己的地址空间，地址之后需要由链接器修正。

典型内容包括：

- Header：文件各部分大小与位置。
- Text Section：指令。
- Data Section：静态数据，如局部/全局变量、字符串、常量。
- Debugging Information：源代码行号到代码地址映射等。
- Symbol Table：导出引用和未解析的导入引用。
- Relocation Information：哪些指令或数据依赖最终地址，需要之后修补。

7.10.4　链接器的作用

链接器把多个 `.o` 和库例程合成为一个可执行文件。这样一个文件改动时，只需重新编译/汇编改动模块，再与已有机器码模块重新链接。

链接要解决三个核心问题：

1. **合并目标文件**：把各目标文件的 text、data、rodata 等同类段合并。
2. **符号解析**：把未定义的外部引用绑定到某个模块中定义的符号。
3. **地址重定位**：修正代码和数据中依赖最终地址的位置。

7.10.5　地址修正与符号解析

目标文件中的地址通常相对于本文件代码段或数据段起点。当多个目标文件合并后，地址必须重新计算。

- **Fixing Addresses**：链接器根据重定位信息修补地址字段。
- **Symbol Resolution**：若模块 A 引用模块 B 中定义的 `x`，链接器必须确定合并后 `x` 的位置，并修改所有对 `x` 的引用。

每个可链接模块的符号表通常包含：

- 本模块定义的全局符号；
- 本模块引用但未定义的全局符号，即 externals；
- 段名，如 text、data、rodata；
- 可选的非全局符号和调试信息。

7.10.6　可执行文件格式

可执行目标文件包含：

- Header：记录 `main` 或更底层入口点位置。
- Text Segment：指令。
- Data Segment：静态数据。
- Relocation Information：若仍保留重定位需求，则指出依赖实际地址的指令和数据。
- Symbol Table：导出和导入引用。
- Debugging Information。

7.10.7　静态链接与动态链接

类型	优点	缺点
静态链接	程序启动简单；库代码在链接时确定；运行时依赖少。	若多个程序都链接同一库，会浪费磁盘/内存；库更新后已静态链接的程序仍使用旧版本。
动态链接	外部函数首次调用时才装入/链接；多个程序可共享同一库；每次启动可使用当前库版本。	首次调用有额外开销；运行时需要查符号表并解析；实现更复杂。

7.10.8　静态库与共享库

- **静态库**：对象文件集合，可类比压缩包。例如 `libc.a` 内含 `printf.o`、`read.o`、`exit.o` 等。链接器只选择解析未定义引用所需的对象文件。
- **共享库**：多个可执行文件指向磁盘上的同一个共享库，最终链接和重定位由 loader 完成。优化方法包括固定非零装载地址、跳转表、运行时 patch jump 等。

7.10.9　装载器 (Loader)

`a.out` 是保存在存储设备中的可执行文件；运行程序时，loader 的工作是把它装入内存并跳到起始地址。现代系统中，loader 通常是操作系统的一部分。

Loader 负责：

- 初始化寄存器、栈、程序参数和环境变量。
- 根据头部确定所需地址空间大小。

- 分配地址空间并读入代码/数据 segment。
- 把未初始化数据 `.bss` 清零。
- 创建栈段。
- 设置运行时信息并从入口点开始执行。

7.10.10　动态装载、PIC 与 GOT

动态链接程序的装载更复杂：操作系统先启动动态链接器，动态链接器再启动目标程序，并在第一次外部调用时装入和修补目标函数。课件特别提到 **GOT** 与 **PLT**：

- **PIC (Position-Independent Code)**：代码不依赖固定装载地址，适合共享库。
- **PC-relative addressing**：通过 PC 相对寻址获取当前位置，是生成 PIC 的基础。
- **GOT (Global Offset Table)**：链接器创建的全局偏移表，保存全局数据或函数地址入口。
- **PLT / stub**：程序先跳到短 stub；首次调用时 stub 进入动态链接器解析符号，之后可改写入口以快速跳到真实函数。

ELF 中数据页跟在代码页后，代码到数据的偏移不依赖实际装载地址。若程序能把自己的地址装入寄存器，就能用固定偏移访问 GOT，再由 GOT 找到数据。

7.10.11　Lazy Linkage

Lazy linkage 延迟符号解析：

1. 第一次调用动态库例程时，stub 装入例程 ID 并跳到 linker/loader。
2. 动态链接器检查目标例程是否已装入。
3. 若未装入，则装入并链接；若已装入，则直接绑定当前引用。
4. 后续调用通过已修补的间接表或跳转入口快速到达目标例程。

7.11　补充主题与课件结尾 (Additional Topics)

7.11.1　编程语言演进与自举

课件用编程语言演进图引出“编译器如何产生自己”的问题。自举 (bootstrapping) 指用某种语言编写该语言自己的编译器：

- 初始核心编译器可用另一种语言或汇编写成。
- 之后逐步扩展语言子集，并用已有编译器编译新版本。
- 这解决了编译器设计中的“先有鸡还是先有蛋”问题。

7.11.2　Reflections on Trusting Trust

课件提到 Ken Thompson 的 “Reflections on Trusting Trust”。核心思想是：即使源代码看起来干净，编译器本身也可能在编译时植入后门，并在编译新编译器时继续复制这种行为。因此，系统可信性不仅取决于源代码，还取决于工具链、构建过程和二进制来源。

7.11.3　真实入口点：`_start` 与 `crt0`

对于大多数 C/C++ 程序，真正入口点不是 `main`，而是 `_start`。它由 C runtime 启动代码实现，传统上称为 `crt0` 或 `c0`。

`crt0` 的任务包括：

- 初始化全局指针 `gp`。
- 清空 `.bss` 段。
- 注册退出时执行的终止函数。
- 运行全局初始化函数。
- 设置 `argc`、`argv`、`envp`。
- 调用 `main`，然后调用 `exit`。

课件中的 `crt0.s` 例子展示了 `.text`、`.global _start`、`la gp,__global_pointer$`、清 `bss`、调用 `atexit`、`call main` 与 `tail exit` 等启动流程。

7.11.4　Quiz：符号放在哪里?

题目代码包含：

```
#define ITEM_NUM 16
static int ITEM_SIZE = 4;

int main() {
    size_t buf_size = ITEM_NUM * ITEM_SIZE * sizeof(char);
    char* heap_buf = (char *) malloc(buf_size);
    if (heap_buf) {
        printf("Succeed to allocate: %d!\n", buf_size);
    }
    free(heap_buf);
    return 0;
}
```

选项：A. Text Section；B. Data Section；C. Exported reference in symbol table；D. Imported reference in symbol table；E. None of the above。

问题	答案	原因
`ITEM_NUM`	E	宏在预处理阶段被替换为常量 16，不作为目标文件符号存在。
`ITEM_SIZE`	B	`static int` 是具有静态存储期的已初始化数据，放入数据段；其符号通常是本地符号，不是导出引用。
`malloc`	D	当前目标文件引用但不定义 `malloc`，需要由链接器从库中解析，因此是导入引用。

7.12　全章总结 (Summary)

本讲从 RISC-V ISA 进入汇编语言与程序装载链路，复习时可以抓住以下主线：

1. **RISC-V 是开放、简单、可扩展的 ISA**：基础整数集小，通用能力通过 `M/A/F/D/C/G` 等扩展补足。
2. **编码格式是考试核心**：R/I/S/B/U/J 六类格式、寄存器字段固定、立即数符号位在第 31 位、B/J 偏移最低位隐含 0。
3. **Load/Store 是内存访问唯一通路**：ALU 只操作寄存器，Load/Store 使用基址加位移；注意符号扩展、零扩展、字节序和对齐。
4. **控制转移依赖 PC 相对寻址**：掌握 `jal/jalr`、条件分支、far branch 改写、while/if/switch 的汇编模式。
5. **调用约定规定谁保存什么**：`a` 寄存器传参和返回，`t` 寄存器不保持，`s` 寄存器由 callee 保持，非叶函数要保护 `ra`。
6. **栈帧连接高级语言和机器执行**：局部变量、保存寄存器、返回地址和参数溢出都通过栈帧管理。
7. **从源代码到运行程序是一条工具链**：预处理、编译、汇编、链接、装载分别解决不同问题；ELF、符号表、重定位、GOT/PLT 和 lazy linkage 是理解链接错误与运行时绑定的关键。
8. **真实入口点早于 `main`**：`_start/crt0` 完成运行时初始化，再调用 `main` 并进入 `exit`。

“ISA 是软件看见的硬件；汇编、链接和装载则把高级语言程序一步步落到这层硬件抽象上。”

7.13　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. **[Exam]** In a RISC-V R-type instruction, which field specifies the destination register? A. opcode B. funct3 C. rd D. rs1.

2. **[Exam]** Which format does the `jalr` instruction belong to? A. R B. I C. S D. J (or U).

3. **[Exam]** Memory contents are as follows, with initial `R2 = 0x0004`:

Address	Data
0x1000	0x1008
0x1004	0x100C
0x1008	0x1004
0x100C	0x100D

   After executing `lw R1, 0x1004(R2)`, what is the value of `R1`? A. 0x1004 B. 0x1008 C. 0x100C D. 0x100D.

4. **[Extra]** Give the instruction type and the assembly instruction for the following binary: `0000 0000 0001 0000 1000 0000 1011 0011`

5. **[Extra]** Translate the function `f` into RISC-V assembly (`g` is declared as `int g(int a,int b)`):

```
int f(int a, int b, int c, int d){
  return g(g(a,b), c+d);
}
```

6. **[Extra]** Expand the following pseudo-instructions into the **shortest** RISC-V instruction sequence:

Pseudo-instruction	Function
`li rd, BIG`	`rd = BIG` (32-bit two's complement constant)
`sne rd, rs1, rs2`	`rd = (rs1 != rs2) ? 1 : 0`

7. **[Extra]** Give the hexadecimal machine code of the following instructions (starting memory address `0x0040 0000`):

```
    ld  x12, 8(x10)
loop:  bne  x12, x13, label
    slli  x13, x12, 4
    jal  x0, loop
label:  jalr  x0, 0(x1)
```

8. **[Extra]** Decode `0x00578833` into a RISC-V assembly instruction (write out each field).

9. **[Quiz]** The same 32-bit pattern `0xFF53AE93` may be interpreted in several ways. Decode it as (a) a 32-bit two's-complement integer, (b) an IEEE 754 single-precision value, and (c) an RV32I instruction. Show all field boundaries and intermediate values.

10. **[Quiz]** The instruction at address `0x00200000` is `Loop: jal x0, L1`; the instruction at `0x00200008` is `beq x3, x4, Loop`; label `L1` is at `0x00200070`. Encode both instructions as 32-bit hexadecimal machine code.

11. **[Quiz]** Convert each pseudoinstruction to the shortest RV32I sequence: (a) `bnez rs, Label`; (b) `j Label`; (c) `mv rd, rs`; (d) `not rd, rs`. Then write the shortest branch sequence to label `IndexOutOfBounds` if $x20 \geq x11$ in signed comparison or if $x20$ is negative.

第八章　处理器性能与单周期 CPU

本章考点

高频任务：CPU Time/IC/CPI/时钟周期、加权 CPI、Amdahl 定律、各类指令数据流、主控制/ALU 译码、控制信号表、单周期关键路径。**复习路线**：数据通路题要沿真实数据流跟踪每个 MUX 和写使能；性能题始终回到 CPU Time 基本式。

8.1　计算机体系结构概览 (Computer Architecture Overview)

8.1.1　层次结构 (System Stack)

计算机系统是由多个抽象层次构成的栈式体系结构，ISA 处于软硬件分界面：

层次	说明
Problem	用户待解决的问题
Algorithm	解决问题的算法
Program	程序（高级语言）
Runtime System (VM/OS/MM)	运行时环境：虚拟机、操作系统、内存管理
ISA (Architecture)	**软硬件接口**：指令集体系结构
Microarchitecture	微体系结构：硬件具体实现方式
Logic / Circuits	逻辑门与电路
Electrons	物理层：电子信号

8.1.2　冯 · 诺依曼结构 (Von Neumann Structure)

冯 · 诺依曼结构核心原则

存储程序计算机 (Stored-Program Computer)：程序和数据都存储在同一存储器中。CPU 从存储器取指令和数据，执行运算后将结果写回存储器。

冯 · 诺依曼结构的三大核心组成：

- **中央处理器 (CPU)**：负责取指、译码、执行。

- **存储器 (Memory)**：存放指令和数据，按地址访问。
- **输入输出 (I/O)**：与外部设备交互。

8.1.3　多核处理器示例 (Multicore Example: AMD Barcelona)

AMD Barcelona 是早期多核处理器的代表，在单块芯片上集成了 **4 个处理器核心**，每个核心拥有独立的 L1/L2 Cache，并共享 L3 Cache。这是应对”三堵墙”、从单核转向多核时代的典型案例。

8.2　CPU 内部结构 (Inside the Processor)

8.2.1　CPU 的三大核心部件 (Datapath, Control & Cache)

CPU 三大核心部件

- **数据通路 (Datapath)**：对数据执行算术和逻辑操作的部件集合，包括寄存器、ALU、多路选择器 (MUX)、存储器等。
- **控制器 (Control Unit)**：根据指令指挥数据通路、存储器和 I/O 设备按序完成操作，产生控制信号。
- **高速缓存 (Cache)**：小而快的 SRAM，位于 CPU 芯片内部，缓存频繁访问的指令和数据，弥补 CPU 与主存之间的速度差异。

8.2.2　三种设计方法论 (Three Design Methodologies)

方法一：有限状态机 (Finite State Machine, FSM)

将数字系统建模为一组状态和状态转移函数，适用于**非可编程专用系统**（Non-programmable System）。控制单元直接编码所有状态转移逻辑，不涉及取指和执行指令。

方法二：寄存器传输控制 (Register Transfer Language, RTL)

通过描述寄存器间的数据传输来设计系统，三要素：

1. **寄存器集合 (Set of Registers)**：大部分在数据通路中，部分在控制单元中。
2. **基本操作 (Micromanipulation)**：寄存器间传输的原子操作。
3. **控制 (Control)**：监督和调度寄存器传输的执行顺序。

方法三：程序状态机 (Program State Machine, PSM)

通用系统 (General System) 的设计方法。输入中包含一段存储在存储器中、由程序计数器 (PC) 寻址的**指令序列**。控制单元负责取指、译码并执行指令，是现代 CPU 的核心设计范式。

PSM 的核心信号流：

- **控制信号 (Control Signals)**：控制单元输出，驱动数据通路。
- **状态信号 (Status Signals)**：数据通路输出给控制单元，用于条件判断（如 Zero 信号）。

- **控制输入/输出 (Control Input/Output)**：与外部 I/O 设备的控制交互。
- **数据输入/输出 (Data Input/Output)**：与存储器和 I/O 的数据交互。

8.2.3 可编程与非可编程系统 (Programmable vs Non-programmable)

非可编程系统 (Specific System)	可编程系统 (General System / CPU)
控制单元不取指/执行指令	控制单元从 PC 指向的存储器取指并执行
包含所有状态转移逻辑（基于输入和状态位）	通过程序灵活控制，功能通用
FSM 实现	PSM 实现（即 CPU）

8.2.4 CPU 的本质 (CPU as a General Digital System)

CPU 是一个**通用数字系统**，本质上是一台图灵机 (Turing Machine)，由寄存器传输控制技术实现：

- **数据通路 (Datapath)**：执行算术运算的部件。
- **控制器 (Control)**：根据程序指令指挥数据通路、存储器和 I/O。

8.3 CPU 关键部件详解 (Key CPU Components)

8.3.1 算术逻辑运算单元 (ALU)

ALU 是执行算术和逻辑运算的核心部件。在本课程的简化 RISC-V CPU 中，ALU 支持以下 5 种基本操作（由 3 位 ALU 控制信号决定）：

ALU 控制信号	操作	说明
000	AND	按位与
001	OR	按位或
010	Add	加法
110	Sub	减法
111	Slt	Set on less than：若 $A < B$ 则结果为 1，否则为 0

ALU 还输出 **Zero** 信号，用于分支指令（如 `beq`）的条件判断：当 ALU 执行减法后结果为 0，则 Zero=1，表示两操作数相等。

8.3.2 存储器 (Memory)

CPU 中涉及两类存储器：

	指令存储器 (Instruction Memory)	**数据存储器 (Data Memory)**
访问方式	只读 (Read-Only)	可读写
控制信号	无（地址输入后直接输出指令）	MemRead, MemWrite
作用	根据 PC 输出当前指令	执行 `lw`/`ld` (读) 和 `sw`/`sd` (写)

8.3.3 寄存器 (Register)

单个寄存器是一个**时序逻辑单元 (State Element)**，由 D 触发器构成，受 Write 信号控制：

- **Write = 0**：数据输出保持原状态不变。
- **Write = 1**：在有效时钟边沿（上升沿），数据输出 = 数据输入。

8.3.4 寄存器文件 (Register File)

寄存器文件由 **32 个 64 位寄存器** (x0–x31) 构成，x0 硬连线为 0（写入无效）。

- **输入**：两个 5 位读地址 (Rs1_addr, Rs2_addr)，一个 5 位写地址 (Wt_addr)，一个 64 位写数据 (Wt_data)，以及写控制信号 RegWrite。
- **输出**：两个 64 位读数据 (Rs1_data, Rs2_data)。
- **时序**：读是组合逻辑（异步立即输出）；写在时钟**上升沿**触发，且仅当 RegWrite=1 时有效。

寄存器文件的 Verilog 模块接口（32×64 bits）：

```
module RegFile(
    input  clk, rst, RegWrite,
    input  [4:0]  Rs1_addr, Rs2_addr, Wt_addr,
    input  [63:0] Wt_data,
    output [63:0] Rs1_data, Rs2_data
);
    reg [63:0] register [1:31];   // r1 - r31
    assign Rs1_data = (Rs1_addr == 0) ? 0 : register[Rs1_addr];
    assign Rs2_data = (Rs2_addr == 0) ? 0 : register[Rs2_addr];
    always @(posedge clk or posedge rst) begin
        if (rst) for (i=1; i<32; i=i+1) register[i] <= 0;
        else if ((Wt_addr != 0) && (RegWrite == 1))
            register[Wt_addr] <= Wt_data;
    end
endmodule
```

8.3.5 立即数生成单元 (Immediate Generation Unit, ImmGen)

ImmGen 根据指令类型从 32 位指令中提取并**符号扩展**为 64 位立即数，由 **ImmSel** 控制信号选择类型：

ImmSel	类型	典型指令	立即数位字段（Verilog 风格）
000	I-type	`lw/ld, addi`	`{{52{inst[31]}}, inst[31:20]}`
001	S-type	`sw/sd`	`{{52{inst[31]}}, inst[31:25], inst[11:7]}`
010	SB-type	`beq, bne`	`{{51{inst[31]}}, inst[31], inst[7], inst[30:25], inst[11:8], 1'b0}`
011	UJ-type	`jal`	`{{43{inst[31]}}, inst[31], inst[19:12], inst[20], inst[30:21], 1'b0}`

注意 SB 和 UJ 型指令的立即数末位硬编码为 0，即偏移量始终为**偶数**（字节对齐）。另外，S 型指令立即数散布于指令字的两个字段（inst[31:25] 和 inst[11:7]），ImmGen 负责拼接。

8.3.6 RISC-V 寄存器约定 (RISC-V Register Conventions)

RISC-V 共有 32 个 64 位通用寄存器 (x0–x31)，各有固定用途约定：

寄存器	别名	用途
x0	zero	常量 0（写入无效）
x1	ra	返回地址 (Return Address)
x2	sp	栈指针 (Stack Pointer)
x3	gp	全局指针 (Global Pointer)
x4	tp	线程指针 (Thread Pointer)
x5–x7	t0–t2	临时寄存器 (Temporaries)
x8–x9	s0–s1	保存寄存器 (Saved)
x10–x17	a0–a7	函数参数/返回值 (Arguments/Results)
x18–x27	s2–s11	保存寄存器 (Saved)
x28–x31	t3–t6	临时寄存器 (Temporaries)

8.3.7 RISC-V 指令字段 (RISC-V Instruction Fields)

每条 RISC-V 指令为 32 位，包含以下字段：

- **opcode**（7 位）：指令的基本操作类型。
- **rs1**（5 位）：第一个源寄存器编号。
- **rs2**（5 位）：第二个源寄存器编号。
- **rd**（5 位）：目标寄存器编号。
- **funct3**（3 位）：功能字段，区分同 opcode 下的不同操作。

- **funct7**（7 位）：扩展功能字段（R 型指令）。
- **immediate**：立即数/地址偏移量（位置因指令类型而异）。

8.3.8 RISC-V 汇编语言概览 (RISC-V Assembly Language)

类别	指令	功能
算术 (Arithmetic)	`add, sub, addi`	加法、减法、加立即数
数据传输 (Data Transfer)	`ld, sd`	加载/存储双字
逻辑 (Logical)	`and, or, xor, srl`	按位与/或/异或/逻辑右移
条件分支 (Conditional Branch)	`beq, bne`	相等/不等时跳转
无条件跳转 (Unconditional Branch)	`jal, jalr`	跳转并链接

8.3.9 指令编码示例 (Instruction Encoding Examples)

- **SB-type**：`bne x10, x11, 2000`：若 x10 ≠ x11，跳转至 PC+2000。立即数 2000 被编码为 SB 格式（末位补 0，各位散布于指令字中）。
- **UJ-type**：`jal x0, 2000`：无条件跳转至 PC+2000，返回地址写入 x0（即丢弃）。立即数 2000 编码为 UJ 格式（末位补 0）。

8.3.10 逻辑设计约定 (Logic Design Conventions)

- 信息用二进制编码，低电压 = 0，高电压 = 1；每条线传一位，多位数据用总线 (Bus) 传输。
- **组合逻辑单元 (Combinational Element)**：输出是输入的纯函数，无内部状态（如 ALU、MUX）。
- **时序逻辑单元 (State Element)**：具有内部存储状态（如寄存器、存储器），受时钟控制。
- **边沿触发方法 (Edge Triggered Methodology)**：在时钟**上升沿**采样并写入状态单元。

  典型执行周期：

1. 读取状态单元的输出（寄存器、PC 等）。
2. 数据流经组合逻辑（ALU、MUX、ImmGen 等）进行运算。
3. 在下一时钟上升沿，将结果写入状态单元（寄存器文件、PC、数据存储器）。

8.4 CPU 性能分析 (CPU Performance Analysis)

8.4.1 基本性能指标 (Key Metrics)

响应时间与吞吐量 (Latency & Throughput)

- **响应时间/延迟 (Latency / Response Time)**：完成一项任务所需的总时间，是单用户最关心的指标。

- **吞吐量 (Throughput / Bandwidth)**：单位时间内完成的任务总量（如每小时处理的事务数）。

  性能定义：$\text{Performance}_X = \dfrac{1}{\text{Execution Time}_X}$

  "X 比 Y 快 n 倍" 等价于：

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution Time}_Y}{\text{Execution Time}_X} = n$$

  示例：计算机 A 执行某程序需 10s，计算机 B 需 15s，则 A 比 B 快 $15/10 = 1.5$ 倍。

CPU 时间 (CPU Time)

- **经过时间 (Elapsed Time)**：包括 I/O、OS 开销、等待等所有时间，反映系统整体性能。
- **CPU 时间 (CPU Time)**：仅包含 CPU 处理该程序的时间，分为用户 CPU 时间（执行程序本身）和系统 CPU 时间（OS 代表程序执行）。

8.4.2　CPU 时钟与时间公式 (CPU Clock & Time Formula)

CPU 性能公式（必须掌握）

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Period} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}} \\ &= \text{Instruction Count} \times \text{CPI} \times \text{Clock Period} \\ &= \frac{\text{IC} \times \text{CPI}}{\text{Clock Rate (Hz)}}\end{aligned}$$

其中：

- **时钟周期 (Clock Period)**：一个时钟周期的时长，如 $250\,\text{ps} = 0.25\,\text{ns}$。
- **时钟频率 (Clock Rate)**：每秒周期数，如 $4.0\,\text{GHz} = 4 \times 10^9\,\text{Hz}$。两者互为倒数。
- **指令数 (Instruction Count, IC)**：由程序、ISA 和编译器决定，与硬件无关。
- **每指令周期数 (Cycles Per Instruction, CPI)**：由 CPU 硬件决定；若不同指令的 CPI 不同，则需加权平均。

  性能改进方向：减少时钟周期数，或提高时钟频率——两者常相互制约，设计者需权衡。

CPU 时间例题

例：计算机 A 时钟频率 2 GHz，执行某程序 CPU 时间为 10s。计算机 B 目标 CPU 时间为 6s，且时钟周期数为 A 的 1.2 倍，求 B 的时钟频率。

$$\text{Clock Cycles}_A = 2\,\text{GHz} \times 10\,\text{s} = 20 \times 10^9$$

$$\text{Clock Cycles}_B = 1.2 \times 20 \times 10^9 = 24 \times 10^9$$

$$\text{Clock Rate}_B = \frac{24 \times 10^9}{6\,\text{s}} = 4\,\text{GHz}$$

CPI 比较例题

例：计算机 A 时钟周期 250ps，CPI=2.0；计算机 B 时钟周期 500ps，CPI=1.2。哪台更快？快多少？

$$\text{CPU Time}_A = \text{IC} \times 2.0 \times 250\,\text{ps} = 500 \cdot \text{IC}\,\text{ps}$$

$$\text{CPU Time}_B = \text{IC} \times 1.2 \times 500\,\text{ps} = 600 \cdot \text{IC}\,\text{ps}$$

A 比 B 快 $600/500 = 1.2$ 倍。

加权平均 CPI (Weighted Average CPI)

$$\text{CPI} = \sum_{i=1}^{n} \left(\text{CPI}_i \times \frac{\text{IC}_i}{\text{IC}} \right) = \sum_{i=1}^{n} \text{CPI}_i \times \text{Relative Frequency}_i$$

例：三类指令 A、B、C，CPI 分别为 1、2、3：

指令类	CPI	序列 1 IC	序列 2 IC
A	1	2	4
B	2	1	1
C	3	2	1
合计	–	IC=5, 周期 =10, 均 CPI=2.0	IC=6, 周期 =9, 均 CPI=1.5

序列 2 虽然指令数更多，但平均 CPI 更低，总时钟周期数更少，因此执行更快。

性能影响因素汇总

因素	影响 IC	影响 CPI	影响 T_c
算法 (Algorithm)	✓	✓	
编程语言 (Language)	✓	✓	
编译器 (Compiler)	✓	✓	
ISA	✓	✓	✓

8.5　性能提升与三堵墙 (Performance Improvement & Three Walls)

8.5.1　性能改进方向 (Performance Improvement Directions)

单处理器性能的提升主要从以下几个方向入手：

- **I/O 发展**：提升输入输出设备的速度。
- **存储器组织结构**：联想存储器、通用寄存器组、Cache 等。
- **指令集发展**：CISC（复杂指令集）→ RISC（精简指令集）。
- **并行处理技术**：微操作级、指令级、线程级、进程级、任务级并行。

8.5.2　三堵墙 (Three Walls)

单处理器性能的提升遭遇了三大瓶颈，即” 三堵墙”：

三堵墙 (Three Walls)

1. **功耗墙 (Power Wall)**
   CMOS 功耗公式：$P \propto C \cdot V^2 \cdot f$（C 为电容，V 为电压，f 为频率）。
   频率翻倍则功耗翻倍，产生严重热点 (Hot-spot) 问题；漏电流 (Power Leakage) 也随之增加。
   功耗从 1980s 到 2000s 增加约 **1000 倍**；电压从 5V 降至约 1V，已难以继续降低；散热能力达到极限。

   功耗降低示例：假设新 CPU 电容负载为原来的 85%，电压和频率各降低 15%，则：

   $$P_{\text{new}} = 0.85C \times (0.85V)^2 \times 0.85f = 0.85^4 \times P_{\text{old}} \approx 0.52\, P_{\text{old}}$$

   即使如此，功耗墙仍无法突破——继续降低电压会影响电路稳定性，继续提高频率会使散热无法维持。

2. **存储墙 (Memory Wall)**
   CPU 速度提升远快于 DRAM 主存，延迟差距持续扩大。
   1980 年微处理器无片上 Cache；1989 年 Intel 首次集成 Cache；2001 年已有两级片上 Cache。
   Cache 增大带来的性能边际收益递减（L2 Cache 越来越大，但贡献越来越小）。

3. **指令级并行墙 (ILP Wall)**
   单进程指令流中可挖掘的 ILP 已趋于枯竭，仅约 **20%** 的 ILP 剩余可被开发。
   应对方向：从 ILP 转向线程级并行 (TLP) 和数据级并行 (DLP)，即多核处理器。

8.6　多处理器与 Amdahl 定律 (Multiprocessors & Amdahl's Law)

8.6.1　多处理器 (Multiprocessors)

多核微处理器 (Multicore Microprocessors) 是应对三堵墙的主要方案：在一块芯片上集成多个处理器核心，要求程序通过**显式并行编程**来利用多核优势。

- 与 ILP（对程序员透明，由硬件自动挖掘）不同，多核并行需要程序员**显式编写并行代码**。
- 主要挑战：**负载均衡 (Load Balancing)**、**通信与同步的开销优化 (Communication & Synchronization)**。

8.6.2　Amdahl 定律 (Amdahl's Law)

基本公式推导

Amdahl 定律的出发点：

$$T_{\text{new}} = T_{\text{cannot enhance}} + \frac{T_{\text{enhanced}}}{S_e}$$

设 F_e 为改进部分占总时间的比例，S_e 为该部分自身的加速比，则：

Amdahl 定律公式

$$\text{Speedup}_{\text{overall}} = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}}$$

其中：

- F_e（**改进比例 / Fraction Enhanced**）：改进部分占系统总执行时间的比例（$0 \le F_e \le 1$）。
  例如：一个程序共需 60s，其中 20s 可被加速，则 $F_e = 20/60$。
- S_e（**部件加速比 / Speedup Enhanced**）：改进部分自身的加速倍数，即改进前与改进后该部分执行时间之比。
  例如：改进后该部分从 5s 降至 2s，则 $S_e = 5/2 = 2.5$。

Amdahl 定律的关键推论

性能改进递减规律与上界推论

1. **性能改进递减规律 (Decreasing Rule)**：若只改进一部分计算任务，改进程度越高，对整体性能的提升就越受限。
2. **加速比上界 (Upper Bound)**：即使将改进部分的速度提升到无限快（$S_e \to \infty$），整体加速比不超过：
   $$\text{Speedup}_{\max} = \frac{1}{1 - F_e}$$
3. **实践意义**：优化应**优先针对执行时间占比最大的部分**（即 **Make the Common Case Fast**）。

”无法达到”的反例 (Counter-example: ”Can't be done”)

例：乘法运算占程序总运行时间的 80s/100s（即 $F_e = 80\%$）。想通过改进乘法性能使**整体**性能提升 5 倍，需要将乘法加速多少倍?

$$5 = \frac{1}{(1 - 0.8) + \frac{0.8}{S_e}} \Rightarrow 0.2 + \frac{0.8}{S_e} = 0.2 \Rightarrow \frac{0.8}{S_e} = 0 \Rightarrow S_e \to \infty$$

结论：这是不可能实现的！即使将乘法运算的速度提升到无限快，整体加速比上界也只有 $1/(1-0.8) = 5$ 倍，这要求非乘法部分的时间趋近于零——实际上无法达到。

典型例题

例 1：某功能的处理速度提升为原来的 20 倍，但该功能仅占系统总运行时间的 40%，整体性能提升为多少?

$$\text{Speedup} = \frac{1}{(1-0.4)+\frac{0.4}{20}} = \frac{1}{0.6+0.02} = \frac{1}{0.62} \approx 1.613 \text{ 倍}$$

例 2：采用浮点运算部件后，浮点运算速度提升 20 倍，整体性能提升 5 倍。求浮点运算占该程序的比例。

$$5 = \frac{1}{(1-F_e)+\frac{F_e}{20}} \Rightarrow F_e \approx 84.2\%$$

8.7　构建数据通路 (Building the Datapath)

8.7.1　本课程简化 RISC-V CPU 所支持的指令集

类型	指令	功能
访存 (Memory-reference)	`lw/ld`, `sw/sd`	加载/存储
算术逻辑 (Arithmetic-logical)	`add, sub, and, or, slt, addi, xor, srl`	R/I 型运算
控制流 (Control Flow)	`beq, bne, jal, jalr`	分支与跳转

8.7.2　指令执行的五个阶段 (Five Stages of Instruction Execution)

RISC-V 指令执行的五个阶段（单周期 CPU）

1. **取指 (IF, Instruction Fetch)**：从指令存储器读出 PC 指向的指令，PC ← PC+4。
2. **译码/读操作数 (ID, Instruction Decode)**：对指令译码，读取寄存器文件中的操作数（无论是否用到，都读出备用）。
3. **执行 (EX, Execute)**：ALU 执行计算——R 型做算术/逻辑运算；访存指令计算地址；分支指令做比较；跳转指令计算目标地址。
4. **访存 (MEM, Memory Access)**：仅 `lw/ld`（Load）和 `sw/sd`（Store）指令使用此阶段，其余指令直通。
5. **写回 (WB, Write Back)**：将结果写回寄存器文件——R 型写 ALU 结果；Load 写内存数据；jal 写 PC+4；同时更新 PC。

注意：所有指令的前两步（IF 和 ID）完全相同，后续步骤根据指令类型不同而分叉。

8.7.3　取指单元 (Instruction Fetch Unit)

取指阶段需要三个核心部件：

- **指令存储器 (Instruction Memory)**：根据 PC 地址输出对应的 32 位指令。
- **程序计数器 (Program Counter, PC)**：存储当前指令的地址，每次取指后自动更新。
- **加法器 (Adder)**：计算 PC+4，用于顺序执行时更新 PC。

为什么是 PC+4? RISC-V 指令固定为 32 位（4 字节），因此顺序执行时下一条指令地址为 PC+4。

指令寄存器 (Instruction Register, IR)：取指阶段还涉及指令寄存器，用于暂存从指令存储器中取出的指令。在单周期 CPU 中，由于整个指令在一个时钟周期内完成，IR 可以省略（指令存储器的输出直接驱动后续逻辑）；但在多周期和流水线 CPU 中，IR 是必要的。

8.7.4　各类型指令的数据流 (Data Paths by Instruction Type)

R 型指令数据流（以 `add rd, rs1, rs2` 为例）

1. 取指：从指令存储器读指令，PC ← PC+4。
2. 译码：从寄存器文件读 rs1 和 rs2。
3. 执行：ALU 执行相应运算（由 ALU 控制信号决定），得到结果。
4. 访存：跳过（R 型指令不访存）。
5. 写回：将 ALU 结果写入 rd（RegWrite=1，MemtoReg=00）。

Load 指令数据流（以 `ld x1, 200(x2)` 为例）

1. 取指：读指令，PC ← PC+4。
2. 译码：读 rs1 (x2) 的值；ImmGen 将 12 位偏移量符号扩展为 64 位。
3. 执行：ALU 计算内存地址 = rs1 + Imm（ALUSrc_B=1，使用立即数而非 rs2）。
4. 访存：从数据存储器按地址读出 64 位数据（MemRead=1）。
5. 写回：将读出的内存数据写入 rd（MemtoReg=01，RegWrite=1）。

Store 指令数据流（以 `sd x1, 200(x2)` 为例）

1. 取指：读指令，PC ← PC+4。
2. 译码：读 rs1 (x2) 和 rs2 (x1)；ImmGen 从 S 型字段提取并扩展立即数。
3. 执行：ALU 计算地址 = rs1 + Imm。
4. 访存：将 rs2 的值写入该地址（MemWrite=1）。
5. 写回：无（Store 指令不写寄存器文件）。

分支指令数据流（以 `beq x1, x2, 200` 为例）

1. 取指：读指令，同时计算 PC+4 备用。
2. 译码：读 rs1 (x1) 和 rs2 (x2)；ImmGen 提取并符号扩展分支偏移量（SB 型，末位补 0）。
3. 执行：ALU 做减法（rs1 - rs2）并输出 Zero 信号；同时另一加法器计算分支目标地址 = PC + Imm（偏移量已左移 1 位）。

4. 访存：跳过。

5. 写回：若 Zero=1 且 Branch=1，则 PC ← PC+Imm；否则 PC ← PC+4。

BNE 指令扩展（以 `bne rs1, rs2, imm` 为例）

`bne`（不等则跳转）与 `beq` 的区别在于条件取反。硬件实现需要额外的 **BranchN** 控制信号：

- 当 `beq`：Branch=1，BranchN=0；跳转条件为 Zero=1（rs1=rs2）。
- 当 `bne`：Branch=0，BranchN=1；跳转条件为 Zero=0（rs1≠rs2）。
- 最终 PC 更新逻辑：PC ← PC+Imm 当 (Branch AND Zero) OR (BranchN AND NOT Zero)。

电路实现使用两个 AND 门加一个 OR 门，将两种分支条件合并为统一的" 是否跳转" 信号。

跳转指令数据流（以 `jal x1, label` 为例）

1. 取指：读指令。
2. 译码：ImmGen 提取并扩展 UJ 型立即数（末位补 0）。
3. 执行：计算跳转目标地址 = PC + Imm（偏移量已左移 1 位）；同时 PC+4 作为返回地址。
4. 访存：跳过。
5. 写回：将 PC+4 写入 rd（x1）；PC ← PC+Imm（Jump=1）。

8.7.5　组合数据通路的原则 (Composing the Elements)

- 第一版数据通路在**一个时钟周期内**完成一条指令（单周期 CPU）。
- 每个功能部件在一个时钟周期内只能执行一次操作。
- 指令存储器和数据存储器**分开**（避免同一周期内读写冲突）。
- 使用**多路选择器 (MUX)** 将不同指令类型的数据流合并到统一数据通路上。

8.7.6　完整单周期数据通路 (Full Single-Cycle Datapath)

完整数据通路通过多路选择器 (MUX) 将不同指令类型的数据流合并，主要 MUX 的作用：

MUX	控制信号	作用
ALU 的 B 端输入	ALUSrc_B	选择寄存器 rs2（0）还是立即数（1）
写回数据选择	MemtoReg	选择 ALU 结果（00）、内存数据（01）或 PC+4（10，用于 jal）
PC 下一值	Branch/BranchN & Zero, Jump	选择 PC+4（顺序）、分支目标或跳转目标

8.8　控制器设计 (Controller Design)

8.8.1　控制器的设计思路 (Design Approach)

控制器设计的核心原则

- 所有控制逻辑均为**组合逻辑 (Combinational Logic)**：等待信号稳定后，数据通路输出正确结果。
- 写信号 (Write Signals) 配合时钟边沿决定何时将结果写入状态单元（寄存器文件、PC、数据存储器）。
- **时钟周期由最长路径决定 (Cycle time = longest path)**：即关键路径决定了整个 CPU 的时钟频率上限。

8.8.2　控制信号总览 (Control Signals Overview)

单周期 CPU 共有 **7 个主控制信号**（由主译码器产生）加上 **4 个 ALU 操作控制信号**（由 ALU 译码器产生）：

7 个主控制信号分为两组：

- **4 个 MUX 控制信号**：ALUSrc_B、MemtoReg（2 位）、Branch、Jump
- **3 个读写控制信号**：RegWrite、MemWrite、MemRead

信号	类型	作用
ALUSrc_B	MUX 控制	ALU B 端：0= 寄存器，1= 立即数
MemtoReg	MUX 控制	写回源：00=ALU 结果，01= 内存数据，10=PC+4（jal）
Branch	MUX 控制	1= 允许 beq 跳转（需结合 Zero 信号）
Jump	MUX 控制	1= 无条件跳转（jal 指令）
RegWR (RegWrite)	R/W 控制	1= 写入寄存器文件
MemWrite	R/W 控制	1= 写入数据存储器
MemRead	R/W 控制	1= 读取数据存储器（Load 指令）

注意：MemtoReg 在支持 `jal` 指令时需要 2 位（00/01/10），以区分三种写回来源。

8.8.3　两级译码结构 (Two-Level Decoder)

级别	模块	输入 → 输出
第一级	主译码器 (Main Decoder)	opcode[6:0] → 7 个控制信号 + ALUOp[1:0]
第二级	ALU 译码器 (ALU Decoder)	ALUOp[1:0] + funct3 + funct7 → ALU 控制信号 [2:0]

ALUOp 的含义

ALUOp	含义
00	执行加法（用于 Load/Store 计算地址）
01	执行减法（用于 Branch beq/bne 比较）
10	由 funct7&funct3 决定（R 型指令）
11	由 funct3 决定（I 型算术指令，如 addi）

ALU 控制信号真值表 (ALU Control Truth Table)

ALU 译码器使用 4 位组合字段 `Fun = {funct3, funct7[0]}` 进行操作选择：

ALUOp	funct7	funct3	ALU 控制	操作
00	X	X	010	Add（Load/Store 地址计算）
01	X	X	110	Sub（Branch 比较）
10	0000000	000	010	Add（add）
10	0100000	000	110	Sub（sub）
10	0000000	111	000	AND（and）
10	0000000	110	001	OR（or）
10	0000000	010	111	Slt（slt）
10	0000000	100	011	XOR（xor）
10	0000000	101	100	SRL（srl，逻辑右移）
11	X	000	010	Add（addi）

8.8.4　各指令的控制信号真值表 (Control Signals Truth Table)

指令类型	ALUSrc	MemtoReg	RegWR	MemWR	Branch	Jump	ALUOp
R-type	0	00	1	0	0	0	10
Load (ld)	1	01	1	0	0	0	00
Store (sd)	1	X	0	1	0	0	00
Branch (beq)	0	X	0	0	1	0	01
Jump (jal)	X	10	1	0	0	1	XX
I-ALU (addi)	1	00	1	0	0	0	11

8.8.5　控制器 Verilog 实现 (Controller Verilog Implementation)

主控制器 (Main Controller)

注意：RISC-V opcode 的低 2 位固定为 `11`，实际译码通常只用 `inst[6:2]` 的 5 位：

```
`define CPU_ctrl_signals \
    {ALUSrc_B, MemtoReg, RegWR, MemWrite, Branch, Jump, ALUop}

always @* begin
    case(OPcode)   // OPcode = inst[6:2], 5-bit
    5'b01100: CPU_ctrl_signals = {1'b0,2'b00,1'b1,1'b0,1'b0,1'b0,2'b10}; // R-type
    5'b00000: CPU_ctrl_signals = {1'b1,2'b01,1'b1,1'b0,1'b0,1'b0,2'b00}; // load
    5'b01000: CPU_ctrl_signals = {1'b1,2'b00,1'b0,1'b1,1'b0,1'b0,2'b00}; // store
    5'b11000: CPU_ctrl_signals = {1'b0,2'b00,1'b0,1'b0,1'b1,1'b0,2'b01}; // beq
    5'b11011: CPU_ctrl_signals = {1'b0,2'b10,1'b1,1'b0,1'b0,1'b1,2'b00}; // jal
    5'b00100: CPU_ctrl_signals = {1'b1,2'b00,1'b1,1'b0,1'b0,1'b0,2'b11}; // addi(I-ALU)
    default:  CPU_ctrl_signals = 9'bxxxxxxxxx;
    endcase
end
```

ALU 译码器 (ALU Decoder)

```
assign Fun = {funct3, funct7[5]};   // 4-bit: {funct3[2:0], funct7[5]}

always @* begin
    case(ALUop)
    2'b00: ALU_Control = 3'b010;    // add（计算地址）
    2'b01: ALU_Control = 3'b110;    // sub（比较条件）
    2'b10:
        case(Fun)
        4'b0000: ALU_Control = 3'b010; // add
        4'b0001: ALU_Control = 3'b110; // sub
        4'b1110: ALU_Control = 3'b000; // and
        4'b1100: ALU_Control = 3'b001; // or
        4'b0100: ALU_Control = 3'b111; // slt
        4'b1000: ALU_Control = 3'b011; // xor
        4'b1010: ALU_Control = 3'b100; // srl（逻辑右移）
        default: ALU_Control = 3'bxxx;
        endcase
    2'b11:
        case(funct3)
        3'b000: ALU_Control = 3'b010; // addi
        default: ALU_Control = 3'bxxx;
        endcase
    endcase
end
```

8.9　单周期实现的性能分析 (Single-Cycle Implementation Performance)

8.9.1　关键路径分析 (Critical Path Analysis)

在单周期 CPU 中，**时钟周期必须满足最慢指令的需求**。假设各部件延迟如下：

- 指令存储器：200 ps
- ALU / 加法器：200 ps
- 数据存储器：200 ps
- 寄存器文件读/写：各 100 ps

  各类指令的关键路径时间：

指令	路径	时间
Load (`ld`)	取指 (200) + 读寄存器 (100) + ALU(200) + 读内存 (200) + 写寄存器 (100)	**800 ps**
Store (`sd`)	取指 (200) + 读寄存器 (100) + ALU(200) + 写内存 (200)	700 ps
R-type	取指 (200) + 读寄存器 (100) + ALU(200) + 写寄存器 (100)	600 ps
Branch (`beq`)	取指 (200) + 读寄存器 (100) + ALU(200)	500 ps
Jump (`jal`)	取指 (200) + 写寄存器 (100)	300 ps

关键路径 (Critical Path) 为 Load 指令，共 800 ps。单周期 CPU 的时钟周期必须设置为 800 ps，时钟频率为 $1/800\,\text{ps} = 1.25\,\text{GHz}$。

8.9.2　单周期实现的缺陷 (Limitations of Single-Cycle)

单周期实现的主要问题

1. **时钟周期由最慢指令决定**：即使执行简单指令（如 `jal` 只需 300 ps），时钟也必须等待最长指令（800 ps），造成大量时间浪费。
2. **功能部件面积浪费（无法复用）**：每条指令独占所有功能部件。若某指令需要 ALU 执行多次操作（如乘法 `mult`），则 CPU 面积将急剧增加，且每个功能部件在一个时钟周期内只能使用一次。
3. **违反” 使常见情况快速” 原则 (Making the Common Case Fast)**：绝大多数指令不需要访问数据存储器，却被迫等待最慢的 Load 指令来决定时钟周期。

解决方案：引入**流水线 (Pipelining)** 技术，通过指令级并行显著提升吞吐量。

8.10　全章总结 (Summary)

本章围绕” 如何设计并分析一个处理器” 展开，核心脉络如下：

1. **冯 · 诺依曼结构是现代 CPU 的基础**：存储程序、统一数据/程序存储，CPU 通过取指-译码-执行-访存-写回循环运行。

2. **数据通路由寄存器文件、ALU、存储器和 ImmGen 构成**：不同指令类型通过 MUX 在统一数据通路上复用不同数据路径；beq 和 bne 通过 Branch/BranchN 信号区分跳转条件。
3. **控制器产生控制信号驱动数据通路**：采用两级译码（主译码器 + ALU 译码器）；7 个主控制信号分为 4 个 MUX 控制和 3 个读写控制；所有控制逻辑为纯组合逻辑。
4. **CPU 性能由 IC、CPI、时钟周期三者共同决定**：提升性能须综合考虑算法、编译器、ISA 和微体系结构设计。
5. **Amdahl 定律揭示性能提升的上界**：整体加速比 $= 1/[(1-F_e)+F_e/S_e]$；即使无限加速改进部分，加速比也不超过 $1/(1-F_e)$；优化应聚焦于执行时间占比最大的瓶颈。
6. **三堵墙推动多核时代的到来**：功耗墙（$P \propto CV^2f$）、存储墙（CPU 远快于 DRAM）、ILP 墙（约 20% 可挖掘）使单核性能提升面临天花板，多核并行成为必然趋势。
7. **单周期 CPU 是流水线的基础**：理解单周期设计的结构（关键路径 800ps = Load 指令）和性能局限，是学习流水线 CPU 的前提。

”处理器设计的核心是在正确性、性能和成本之间寻求最佳平衡。”

8.11　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. [**Exam**] A single-cycle CPU supports four instruction classes A, B, C, D with frequencies 10%/20%/30%/40% and CPIs 1, 4, 3, ? respectively. After optimization, D's CPI is halved and the overall CPI becomes 3. Find D's original CPI. A. 4 B. 5 C. 6 D. 8.

2. [**Exam**] Which component is **required** in a multi-cycle CPU but **not strictly required** in a single-cycle CPU?
A. instruction register IR　B. program counter PC　C. instruction memory　D. multiplexer MUX.

3. [**Exam**] Single-cycle CPU execution times by instruction class: R-type 750ps, Load 1000ps, Branch 875ps, I-type 625ps, Jump 600ps, each class occurring with equal frequency.
(a) Find the clock cycle time and clock frequency; (b) find the average CPI; (c) find the CPU execution time for 10^8 instructions; (d) if a redesign reduces the time in (c) to 80% of the original but increases the total cycle count by 20%, find the new clock frequency.

4. [**Extra**] P1, P2, P3 run the same instruction set: P1 2GHz/CPI 1.5, P2 1.5GHz/CPI 1.0,

P3 3GHz/CPI 2.5.
(1) Which has the highest performance? (2) If each runs a program for 10 seconds, find the number of clock cycles and instructions for each.

5. **[Extra]** CISC vs RISC performance: a task needs P CISC instructions or $2P$ RISC instructions; one CISC instruction takes $8T$ ns, one RISC instruction takes $2T$ ns. Which performs better?

6. **[Extra]** A function is sped up 20× but accounts for only 40% of total run time; what is the overall speedup?
If adding an FP unit speeds up FP operations 20× and the overall speedup is 5×, find the FP fraction F_e.

7. **[Extra]** Translate the following C functions into RISC-V assembly callable from a standard C program:

```
void main(int *z, int x, int y){ *z = sub(x, y); }
int sub(int n, int m){ return m - n; }
```

Then: (1) In a single-cycle design, can the instruction memory IM and data memory DM be merged into one MEM? If yes, draw the datapath for `lw/sw`; if not, explain why. (2) Given IM/DM read 2ns and write 1.5ns, control unit 4.6ns, ALU 1.7ns, register file 1.3ns, find the optimal clock cycle time.

8. **[Extra]** Add a register–memory instruction `Addm rd, rs, rt` (`rd = rs + Mem[rt]`) to the single-cycle datapath.
(a) Give the binary instruction format; (b) describe the datapath changes needed and draw the datapath for this instruction; (c) list all control signals and their values for this instruction.

9. **[Extra]** Examine the difficulty of adding `ss rs1, rs2, imm` (Store Sum: `Mem[Reg[rs1]] = Reg[rs2] + imm`) to RISC-V: (a) What new functional units are needed? (b) Which existing units must change? (c) What new datapaths are needed? (d) What new signals does the control unit need? (e) Sketch the implementation on the single-cycle datapath.

10. **[Exam]** A processor with separate instruction/data memories defines three instructions (word = 4 bytes):

Instruction	Semantics
`lw ri, imm`	`Reg[ri]=Mem[imm]`, `PC=PC+4`
`sw ri, imm`	`Mem[imm]=Reg[ri]`, `PC=PC+4`
`add ri, imm`	`Reg[ri]=Reg[ri]+imm`, `PC=PC+4`

(a) Complete the interface ports of the register file and data memory; (b) draw the complete datapath supporting all three instructions (you may add MUXes, sign extension, etc.); (c) design the control signals for the three instructions and give each signal's value in a table.

11. **[Quiz]** A single-cycle processor runs at 4 GHz and executes 100,000 instructions in 0.03 ms. Find its CPI. A new processor executes instruction classes A, B, C with counts $1500, 2000, 500$ and CPIs equal to the previous answer, 1.4, and 2.1, respectively; find its average CPI. Finally, for a multi-cycle RISC-V CPU whose R-type/Load/Store/Branch/Jump instructions require 4/5/4/3/3 cycles and occur with percentages 50%/10%/10%/15%/15%, find the average CPI and total cycle count for 10^6 instructions.

第九章　多周期处理器 (Multi-Cycle Processor)

本章考点

高频任务：单/多周期数据通路差异、IR/MDR/A/B/ALUOut、IF–ID–EX–MEM–WB 分阶段微操作、FSM 状态转移与控制信号、微程序控制、加权 CPI 和性能对比。**复习路线**：以“当前周期读什么、算什么、写到哪里”为主线，用临时寄存器连接跨周期数据。

9.1　为什么需要多周期? (Why Multi-Cycle?)

9.1.1　提升程序性能的三个方向

提升程序运行速度，从体系结构角度主要有三个方向：

- **减少指令数 (Reduce Instruction Count)**：设计功能更强的指令（CISC 思路），或使用更优的编译器。
- **减少每条指令所需的时钟周期 (Reduce CPI)**：RISC 指令简单化，配合多周期、流水线等技术；引入多个执行单元（多发射）。
- **提高时钟频率 (Increase Clock Rate)**：采用更新工艺，优化关键路径，或引入流水线。

多周期设计属于第二个方向——通过允许简单指令更快完成来降低平均 CPI。

9.1.2　单周期 vs. 多周期对比

	单周期 (Single-Cycle)	**多周期 (Multi-Cycle)**
时钟周期长度	由最慢指令决定 (lw=800ps)	较短，由最复杂的单步操作决定
简单指令速度	慢（被迫等待最长时间）	快（简单指令占用更少周期）
硬件复用	差（ALU 和存储器各需两份）	好（一个 ALU、一块存储器可复用）
时序开销	每条指令只付一次	每个阶段都需要付出寄存器开销

单周期与多周期的根本区别

- **单周期**：一个时钟周期执行**一条完整指令**。
- **多周期**：一个时钟周期执行**指令执行的一个阶段**。

设计流程相同：都要设计数据通路 (Datapath) 和控制器 (Controller)。

9.1.3　多周期 CPU 的基本设计原则

1. 将指令执行拆分为若干**时间近似相等**的阶段，每个阶段占用一个时钟周期。
2. 每个时钟周期内最多完成一次存储器访问、一次寄存器读写，或一次 ALU 运算。
3. 上一个周期的执行结果必须**暂存在专用时序寄存器**中，供下一周期使用。
4. 时钟周期长度由**最复杂的单步操作**决定（而非最长指令）。

9.2　多周期数据通路 (Multi-Cycle Datapath)

9.2.1　数据通路总体变化

与单周期 CPU 相比，多周期 CPU 的数据通路作了以下关键调整：

- **存储器合并**：只保留一块统一的存储器（指令和数据共用），这也正是冯 · 诺依曼结构的本来样子。单周期 CPU 需要将其拆分是因为一个周期内可能同时读指令和读/写数据；多周期 CPU 由于分阶段执行，不会产生冲突。
- **ALU 合并**：只保留一个 ALU，不再需要单独的地址加法器，通过分时复用完成不同阶段的计算。
- **新增临时寄存器**：为了将执行结果从一个周期传递到下一个周期，需要引入多个临时寄存器（下节详述）。

9.2.2　新增的临时寄存器 (New Temporary Registers)

多周期 CPU 中新增的 5 个临时寄存器

- **IR（指令寄存器，Instruction Register）**：取指阶段结束后，将从存储器取出的指令锁存于此，供后续译码阶段使用。
- **MDR（内存数据寄存器，Memory Data Register）**：访存阶段结束后，将从内存读出的数据暂存于此，供写回阶段使用。
- **A**：译码/取操作数阶段结束后，暂存从寄存器文件读出的 rs1 数据。
- **B**：译码/取操作数阶段结束后，暂存从寄存器文件读出的 rs2 数据。
- **ALUout**：执行阶段结束后，暂存 ALU 的计算结果，供访存或写回阶段使用。

核心思想：这些临时寄存器的作用是将指令执行在物理上划分为独立的阶段。每个阶段结束时，将结果锁入对应的临时寄存器，下一阶段从该寄存器读取，从而实现阶段间的” 接力”。

9.2.3　控制信号汇总 (Control Signals Summary)

多周期 CPU 中的控制信号比单周期更多，且同一信号在不同阶段的含义/取值可能不同。

1 位控制信号：

信号	为 0 时	为 1 时
RegDst	写入寄存器来自 rt 字段	写入寄存器来自 rd 字段
RegWrite	无操作	寄存器文件被写入
ALUSrcA	ALU 第一个操作数为 PC	ALU 第一个操作数为寄存器 A
MemRead	无操作	将 Address 处的内存数据送至内存数据输出
MemWrite	无操作	将写数据写入 Address 处内存
MemToReg	写回寄存器的值来自 ALUout	写回寄存器的值来自 MDR
IorD	PC 作为存储器地址	ALUout 作为存储器地址
IRWrite	无操作	将内存输出写入 IR
PCWrite	无操作	PC 被写入（来源由 PCSrc 决定）
PCWriteCond	无操作	若 ALU 的 Zero 信号有效，则写入 PC

2 位控制信号：

信号	值	含义
ALUOp	00	ALU 执行加法
	01	ALU 执行减法
	10	由 funct 字段决定操作
ALUSrcB	00	ALU B 端来自寄存器 B
	01	ALU B 端固定为 4（用于 PC+4）
	10	符号扩展后的立即数
	11	符号扩展后的立即数左移 2 位（分支目标偏移）
PCSrc	00	PC ← ALU 输出（PC+4，顺序执行）
	01	PC ← ALUout（分支目标地址）
	10	PC ← 跳转目标地址

9.3　指令执行的各阶段 (Instruction Execution Stages)

9.3.1　阶段一：取指 (IF, Instruction Fetch)

操作步骤：

1. 将 PC 的值作为地址送至存储器。
2. 从存储器中读取指令。
3. 将指令写入 IR（IRWrite=1），供下一周期使用。
4. 同时利用 ALU 计算 PC+4（ALUSrcA=0，ALUSrcB=01，ALUOp=00），并更新 PC（PCWrite=1，PCSrc=00）。

本周期控制信号： IorD=0, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSrc=00, IRWrite=1, PCWrite=1，MemRead=1，MemWrite=0，RegWrite=0。

9.3.2　阶段二：译码与读取操作数 (ID, Instruction Decode & Register Fetch)

操作步骤：

1. 对 IR 中的指令进行译码。
2. **乐观地读取寄存器操作数**：无论指令是否实际使用 rs1 和 rs2，都将其读出并存入寄存器 A 和 B。（因为读操作无副作用，乐观读取可节省时间。）
3. **乐观地计算分支目标地址**：ALU 计算 PC + (符号扩展立即数 × 4)（ALUSrcA=0，ALUSrcB=11，ALUOp=00）。此时我们还不知道是否是分支指令，但” 反正现在没别的事做”，提前计算以备用。

本周期控制信号： ALUSrcA=0，ALUSrcB=11，ALUOp=00。

9.3.3 阶段三：执行 (EX, Execute)

各类指令在此阶段出现分叉：

指令类型	执行操作
访存指令 (Load/Store)	ALU 计算内存地址 = A + 符号扩展立即数（ALUSrcA=1, ALUSrcB=10, ALUOp=00), 结果存入 ALUout
R 型指令	ALU 根据 funct 字段执行对应运算（A op B)，ALUSrcA=1，ALUSrcB=00，ALUOp=10
分支指令 (beq)	ALU 计算 A-B（ALUSrcA=1，ALUSrcB=00，ALUOp=01)，产生 Zero 信号；PC 更新使用 ID 阶段预算的分支目标
跳转指令 (J)	直接产生跳转目标地址，PCSrc=10，PCWrite=1

9.3.4 阶段四：访存 (MEM, Memory Access)

仅 Load 和 Store 指令使用此阶段：

- **Load**：以 ALUout 中的地址从存储器读数据（IorD=1，MemRead=1)，结果存入 MDR。
- **Store**：以 ALUout 中的地址向存储器写入 B 寄存器中的数据（IorD=1，MemWrite=1)。Store 指令在此阶段结束后即完成，不需要写回阶段。

  其他指令（R 型、分支、跳转）**跳过此阶段**，直接进入写回。

9.3.5 阶段五：写回 (WB, Write Back)

- **Load 指令**：将 MDR 中的数据写入目标寄存器（RegDst=0，MemToReg=1，RegWrite=1)。
- **R 型指令**：将 ALUout 中的结果写入 rd（RegDst=1，MemToReg=0，RegWrite=1)。
- **I 型算术指令(如 addi)**:将 ALUout 中的结果写入 rt(RegDst=0,MemToReg=0,RegWrite=1)。

9.3.6 各类指令的阶段数汇总

指令类型	所需阶段数	阶段序列
beq, j（跳转/分支）	3	IF → ID → EX
R 型, Store (sw/sd)	4	IF → ID → EX → WB / IF → ID → EX → MEM
Load (lw/ld)	5	IF → ID → EX → MEM → WB

9.4 控制器设计 (Controller Design)

9.4.1 控制器设计的分析框架

控制器的设计从**因果分析**入手，核心思路与单周期 CPU 一致，但需考虑阶段信息：

- **信息来源**：指令的 32 位机器码（opcode、funct3、funct7、rd、rs1、rs2 等字段）以及当前所处的执行阶段。
- **控制目标一：顺序部件的操作选择**（寄存器读/写使能、存储器读/写使能等）。
- **控制目标二：数据流控制**（多路选择器的选择信号，决定数据从哪里来、到哪里去）。
- **控制目标三：ALU 操作选择**（根据指令类型和 funct 字段确定 ALU 执行何种运算）。
- **控制目标四：分支/跳转指令的 PC 更新**（决定是否以及如何更新 PC）。

与单周期 CPU 的关键区别：在多周期 CPU 中，同一控制信号在不同周期（不同阶段）的取值可能完全不同，因此控制器必须**感知当前阶段**，而不能像单周期那样只看 opcode 就给出固定输出。

9.4.2　为什么单周期的控制器设计方式不适用?

在单周期 CPU 中，每条指令的每个控制信号在整个时钟周期内**保持不变**。但在多周期 CPU 中，**同一个控制信号在不同周期的取值可能完全不同**。

示例：执行 `add x10, x12, x14`:

控制信号	周期 1 (IF)	周期 2 (ID)	周期 3 (EX)	周期 4 (WB)
PCWrite	1	0	0	0
IorD	0	–	–	–
MemRead	1	0	0	0
IRWrite	1	0	0	0
PCSrc	00	–	–	–
ALUOp	00	00	**10**	–
ALUSrcB	01	11	**00**	–
ALUSrcA	0	0	**1**	–
RegWrite	0	0	0	**1**

可以看出，ALUOp 在周期 2 (ID) 是 00，在周期 3 (EX) 变为 10——**控制信号随执行阶段变化**，这是多周期 CPU 控制器设计的核心挑战。

多周期控制器的关键特征

多周期 CPU 的控制器必须是**时序逻辑**（有状态的），而非单周期 CPU 的纯组合逻辑。控制器需要记住当前处于哪个执行阶段，并据此产生正确的控制信号。

9.4.3　两种控制器实现方案

	硬连线控制器 (Hardwired)	**微程序控制器 (Microprogram)**
实现方式	有限状态机 (FSM) + PLA 电路 + 状态寄存器	控制存储器 (Control Storage/ROM) + 微指令
适用场景	RISC（指令简单、数量少）	CISC（指令复杂、数量多）
优点	控制信号产生延迟短，性能高	设计灵活，易于修改和扩展 ISA
缺点	ISA 复杂时难以维护和更新	执行时间比硬连线长，效率较低

9.5　硬连线控制器：有限状态机 (Hardwired Controller: FSM)

9.5.1　FSM 设计思路

硬连线控制器的核心是**有限状态机 (Finite State Machine, FSM)**：

- 每个状态 (State) 对应指令执行的某一个阶段，并确定该阶段的所有控制信号取值。
- 状态之间的**转移条件**通常依赖指令的 opcode 字段（从 IR 读取）。
- 电路实现：PLA（可编程逻辑阵列）实现组合逻辑，状态寄存器存储当前状态，每个时钟上升沿更新状态。

9.5.2　各状态定义与转移

以下为完整 FSM 的状态列表（状态编号 S0–S11）：

FSM 状态图（所有指令类型）

S0：取指 (IF)（复位后进入）

- 控制：IorD=0，ALUSrcA=0，ALUSrcB=01，ALUOp=00，PCSrc=00，IRWrite=1，PCWrite=1
- 无条件转移至 S1

S1：译码/读操作数 (ID)

- 控制：ALUSrcA=0，ALUSrcB=11，ALUOp=00（乐观计算分支目标）
- 根据 opcode 分叉：
  - Op=LW 或 Op=SW → S2（内存地址计算）
  - Op=R-type → S6（R 型执行）
  - Op=beq → S8（分支执行）

- Op=addi → S9（addi 执行）
- Op=J → S11（跳转）

S2：内存地址计算 (MemAdr)（lw 和 sw 共用）

- 控制：ALUSrcA=1，ALUSrcB=10，ALUOp=00
- Op=LW → S3；Op=SW → S5

S3：内存读取 (MemRead)（lw）

- 控制：IorD=1，MemRead=1
- 无条件转移至 S4

S4：Load 写回 (MemWriteback)（lw）

- 控制：RegDst=0，MemToReg=1，RegWrite=1
- 无条件转移至 S0

S5：内存写入 (MemWrite)（sw）

- 控制：IorD=1，MemWrite=1
- 无条件转移至 S0

S6：R 型执行 (Execute)

- 控制：ALUSrcA=1，ALUSrcB=00，ALUOp=10
- 无条件转移至 S7

S7：R 型写回 (ALUWriteback)

- 控制：RegDst=1，MemToReg=0，RegWrite=1
- 无条件转移至 S0

S8：分支执行 (Branch)（beq）

- 控制：ALUSrcA=1，ALUSrcB=00，ALUOp=01，PCSrc=01，PCWriteCond=1
- 无条件转移至 S0

S9：addi 执行 (AddiExecute)

- 控制：ALUSrcA=1，ALUSrcB=10，ALUOp=00
- 无条件转移至 S10

S10：addi 写回 (AddiWriteback)

- 控制：RegDst=0，MemToReg=0，RegWrite=1
- 无条件转移至 S0

S11：跳转 (Jump)

- 控制：PCSrc=10，PCWrite=1

- 无条件转移至 S0

9.5.3　硬连线控制器的电路实现

硬连线控制器电路结构

硬连线控制器由以下三部分构成：

- **状态寄存器 (State Register)**：存储 FSM 的当前状态（如 S0–S11），每个时钟上升沿根据组合逻辑输出更新状态。
- **PLA（可编程逻辑阵列，Programmable Logic Array)**：实现组合逻辑部分，包括两个子功能：
  - **输出逻辑**：根据当前状态产生本周期的所有控制信号（如 ALUSrcA、ALUOp 等）。
  - **次态逻辑**：根据当前状态 + 指令的 opcode 字段，计算下一个状态。
- **接口信号**：控制器接收数据通路的 opcode（来自 IR）和时钟信号，输出各种控制信号到数据通路。

工作流程：时钟上升沿 → 状态寄存器锁存新状态 → PLA 根据新状态和 opcode 输出控制信号 → 数据通路执行对应操作 → 等待下一个时钟沿。

9.6　微程序控制器 (Micro-Program Controller)

9.6.1　历史背景

- **1951 年**：M. V. Wilkes 提出微程序控制器的概念。
- **1964 年**：IBM 360 系列首次在商用机器上部署微程序控制器。
- **1970 年代起**：随着 VLSI 技术发展得到广泛应用。
- **现状**：大多数 CISC 处理器（如 x86）均采用微程序控制器。

9.6.2　基本原理与术语

微程序控制器核心术语

- **微程序 (Micro-program)**：由一系列微指令构成的序列，每条 ISA 指令对应一个微程序。
- **微指令 (Micro-instruction)**：一个 0/1 序列，由若干微命令组成；也称**控制字 (Control Word, CW)**。每条微指令对应 FSM 中的一个状态。
- **微命令 (Micro-command)**：控制部件发送给执行部件的控制信号（如寄存器的时钟使能信号、MUX 的选择信号）。
- **微操作 (Micro-operation)**：与微命令一一对应，是执行部件响应微命令所完成的最基本操作（类似数据通路中控制信号与数据流操作的关系）。
- **控制存储器 (Control Storage, CS)**：一块 ROM，存储所有微程序（即所有微指令的集合）。
- **微程序计数器 (μPC)**：类似 ISA 级别的 PC，指向控制存储器中下一条微指令的地址。

9.6.3　微程序控制器的实现架构

微程序控制器的主要组成部件：

- **μPC（微程序计数器）**：指向 CS 中当前执行的微指令地址。
- **控制存储器 (CS/ROM)**：存储全部微指令，通过 μPC 寻址。
- **微指令寄存器 (μIR)**：暂存从 CS 中取出的当前微指令。
- **起始/分支地址生成器**：根据 IR 中的 opcode 确定每条 ISA 指令对应微程序的起始地址，以及微指令内部的分支目标。
- **分支控制逻辑**：根据条件码 (Condition Code) 决定是否转移到分支微指令地址。

9.6.4　微程序控制器的两个核心问题

1. **微指令格式与微命令编码**：如何在有限的位宽内编码所有微命令?
   - **直接编码**（水平型微指令）：每一位直接对应一个微命令，控制灵活但指令字很长。
   - **间接编码**（垂直型微指令）：对微命令分组编码，指令字短但需要额外译码延迟。
2. **下一微指令地址的确定**：通常有两种方式：
   - **μPC 顺序加一**：类似普通程序计数器的顺序递增。
   - **微指令中显式存储下一地址**：微指令格式中包含跳转地址字段，支持灵活转移。

9.6.5　优缺点总结

微程序控制器的优缺点

优点：

- 显著降低控制器设计难度，尤其是对于复杂指令集 (CISC)。
- 灵活性高：修改微程序即可调整 ISA，无需重新布线。

缺点：

- 执行速度慢于硬连线控制器（访问 CS 的延迟）。
- 效率相对较低（引入额外存储和译码开销）。

9.7　多周期 CPU 的性能分析 (Performance Analysis)

9.7.1　CPI 计算

多周期 CPU 中，不同指令的 CPI 不同，需要用**加权平均 CPI** 来评估整体性能。
各类指令所需周期数：

指令类型	所需周期数
分支 (beq)、跳转 (j)	3
R 型、Store (sw)	4
Load (lw)	5

假设典型程序的指令分布：

- Load：25%
- Store：10%
- Branch：11%
- Jump：2%
- R 型：52%

计算加权平均 CPI：

$$\begin{aligned}\overline{\text{CPI}} &= (0.11 + 0.02) \times 3 + (0.52 + 0.10) \times 4 + 0.25 \times 5 \\ &= 0.13 \times 3 + 0.62 \times 4 + 0.25 \times 5 \\ &= 0.39 + 2.48 + 1.25 \\ &= \mathbf{4.12}\end{aligned}$$

9.7.2　关键路径与时钟周期

多周期 CPU 的时钟周期由**单个阶段中最长的操作**决定。假设各部件延迟为：

部件	延迟
存储器读/写	最长，主导关键路径
ALU 运算	较长
寄存器读/写	较短

关键路径分析：在多周期设计中，存储器读操作（取指阶段读指令存储器，或访存阶段读数据存储器）通常是最慢的单步操作，**主导时钟周期**。

在本课程的具体参数下：$T_c = 325\,\text{ps}$。

9.7.3　性能对比计算

例题：以 1000 亿条指令的程序为例，分别比较单周期和多周期 CPU 的执行时间。

多周期 CPU：

$$T_{\text{multi}} = 10^{11} \times 4.12 \times 325\,\text{ps} = 10^{11} \times 4.12 \times 325 \times 10^{-12}\,\text{s} \approx \mathbf{133.9}\,\text{s}$$

单周期 CPU（$T_c = 925\,\text{ps}$，CPI=1）：

$$T_{\text{single}} = 10^{11} \times 1 \times 925\,\text{ps} = 10^{11} \times 925 \times 10^{-12}\,\text{s} = \mathbf{92.5}\,\text{s}$$

反直觉的结论：单周期反而更快?

在此例中，**单周期 CPU（92.5s）比多周期 CPU（133.9s）执行更快**。

原因分析：

- 多周期缩短了时钟周期（925ps → 325ps），但代价是 CPI 从 1 增加到 4.12。
- 整体执行时间 = IC × CPI × T_c：$1 \times 925 < 4.12 \times 325$（$925 < 1339$）。
- **时序开销 (Sequencing Overhead)** 被多次支付：每个阶段都需要额外的寄存器写入开销，累积后抵消了缩短时钟周期带来的收益。

9.7.4　多周期设计的意义

尽管多周期 CPU 在简单实现下并不比单周期快，但它提供了一个**重要的设计视角**：

多周期的核心贡献

多周期将指令执行分解为**细粒度的阶段**，这为**流水线 (Pipelining)** 技术奠定了基础。

流水线的思路：在多周期的基础上，让多条指令的不同阶段**重叠执行**（类似工厂流水线），从而在不降低每条指令延迟的前提下，大幅提升**吞吐量 (Throughput)**——这将是下一讲（系统 II）的核心主题。

9.8　全章总结 (Summary)

1. **多周期 CPU 的动机**：单周期 CPU 的时钟周期受最慢指令（lw，925ps）约束；多周期通过缩短时钟周期（每周期只完成一个阶段）并允许简单指令更快完成，改善平均 CPI。

2. **数据通路变化**：合并存储器（统一指令/数据存储器）和 ALU（去掉多余加法器）；新增 5 个临时寄存器（IR、MDR、A、B、ALUout）实现阶段间数据传递。
3. **五个执行阶段**：IF（取指）→ ID（译码 + 读操作数）→ EX（执行）→ MEM（访存，仅 lw/sw）→ WB（写回）。不同指令在 EX 之后走不同路径，周期数为 3–5。
4. **控制信号随阶段变化**：同一信号在不同周期取值不同，因此控制器必须是时序逻辑，需记录当前阶段。
5. **两种控制器实现**：
   - **硬连线控制器**：FSM（S0–S11）+ PLA，适合 RISC，性能高但难以修改。
   - **微程序控制器**：将每条 ISA 指令翻译为存储在 CS/ROM 中的微指令序列，适合 CISC，灵活但效率较低。
6. **性能分析**：典型程序的加权平均 CPI ≈4.12，$T_c = 325\,\text{ps}$，执行时间 ≈133.9s > 单周期 92.5s（$T_c = 925\,\text{ps}$）。多周期实现未能超越单周期，原因是时序开销多次支付抵消了缩短时钟周期的收益。
7. **设计启示**：多周期揭示了指令的细粒度执行结构，直接引出流水线技术——通过让多条指令的不同阶段重叠执行，在不增加硬件的情况下大幅提升吞吐量。

> ”多周期是单周期到流水线的桥梁：它分解了问题，流水线则让多个问题同步推进。”

9.9　真题与巩固练习

做题要求

以闭卷状态完成：先标出题目所属考点，再写推导过程。画图题必须标明信号名、位宽、选择端和控制条件；计算题必须保留公式、中间值与单位。本节不附答案，便于自测。

Exercises

Exercises (Past Exams & Selected Practice)

1. **[Extra]** List the 5 new temporary registers added in a multi-cycle CPU and the role of each, and state the number of stages and the stage sequence required by each instruction class (beq, R-type, sw, lw).

2. **[Extra]** Given the cycle counts per instruction class in a multi-cycle design: branch/jump 3, R-type/Store 4, Load 5; and the instruction mix Load 25%, Store 10%, Branch 11%, Jump 2%, R-type 52%. Find the weighted average CPI.

3. **[Extra]** Let the multi-cycle design have $T_c = 325$ps, CPI $= 4.12$, and the single-cycle design have $T_c = 925$ps, CPI $= 1$. Find the time to execute 10^{11} instructions for each, and explain why the single-cycle design is actually faster in this example.

4. **[Extra]** Compare a hardwired control unit with a microprogrammed control unit: what are the implementation method, applicable scenario (RISC/CISC), and pros and cons of each?

考前检查清单

最后自测

1. 能否不查表完成补码、IEEE 754 和 RISC-V 立即数的编解码?
2. 能否从规格独立完成组合电路和时序电路的建模、化简、映射与验证?
3. 能否对一条指令说清每个阶段经过哪些部件、MUX 选什么、信号为何取 0/1?
4. 能否用同一套 CPU Time 模型分析单周期、多周期和 Amdahl 定律问题?
5. 是否已将错题按“概念错、编码错、连线错、符号/位宽错、计算错”分类重做?